



UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO  
GRAO EN ENXEÑARÍA INFORMÁTICA  
MENCIÓN EN COMPUTACIÓN



# **Sistema inteligente para la gestión, análisis y consulta conversacional de reuniones mediante un chatbot**

**Estudiante:** Esteban Suárez Calvo

**Dirección:** Enrique Fernández Blanco

A Coruña, julio de 2026.

*A mi familia, amigos y pareja*

### **Agradecimientos**

Me gustaría dar las gracias a mi familia por su apoyo durante toda esta larga etapa de estudio y de dedicación, durante el grado y también en los años anteriores.

A mi pareja Carolina, por todos los buenos momentos que hemos vivido juntos a lo largo de la carrera, siempre has estado ahí cuando te he necesitado, ha sido muy bonito tenerte a mi lado todos estos años. Agradecerte también las “clases particulares” que me impartiste antes de algún que otro examen, salvándome así de más de un suspenso.

A mi amigo y compañero de piso César, por haberme acompañado y apoyado durante la mayor parte de esta etapa, siendo de gran ayuda tu sentido del humor y nuestros maratones nocturnos de anime.

A mi tutor Enrique por tu rigor, apoyo y dedicación en este proyecto. Gracias no solo por la guía técnica sino también por tu cercanía, en especial por nuestras “breves” reuniones del TFG que, por arte de magia se alargaban hasta las tres horas hablando de mil cosas más; tanto que al volver a casa Carolina siempre me preguntaba si habíamos quedado para trabajar o para pasar el rato. Muchas gracias por tu tiempo y por tu labor como mentor.

Por último, también me gustaría agradecer al profesorado del Seminario Menor de la Asunción por su paciencia y dedicación durante los cuatro años anteriores a comenzar este grado universitario. En especial a Doña Rosalía por haber cultivado en mí el interés por las matemáticas.

## Resumen

Las reuniones constituyen una herramienta fundamental para la transmisión de información, la toma de decisiones y la coordinación en los equipos de trabajo. No obstante, el contenido tratado suele quedar limitado a actas manuales o notas personales, lo que dificulta la recuperación eficiente de la información. Por otro lado, aunque las grabaciones digitales son comunes, buscar datos específicos en ellas resulta un proceso lento y poco práctico.

Por ello, este proyecto propone el desarrollo de un sistema inteligente orientado a la gestión automatizada de grabaciones de reuniones. La plataforma integra un flujo que comienza con la transcripción del habla. A partir del texto generado, el sistema es capaz de extraer resúmenes automáticos de los puntos clave y permite al usuario realizar consultas en lenguaje natural sobre el contenido procesado. Esta última funcionalidad se apoya en una arquitectura [RAG](#), que dota al sistema de la capacidad de responder dudas específicas sobre cualquier sesión almacenada. Todo este sistema se materializa en una aplicación web que centraliza la gestión de las grabaciones, permitiendo al usuario visualizar las transcripciones y los resúmenes, así como interactuar con el asistente de forma intuitiva.

El valor diferencial de esta propuesta radica en el compromiso con la privacidad y la soberanía de los datos. A diferencia de las soluciones comerciales basadas en la nube, este sistema utiliza herramientas de código abierto y permite la ejecución de [LLM](#) de forma completamente local. De este modo, se garantiza que la información sensible de las reuniones no salga del entorno del usuario, eliminando además la dependencia de suscripciones y [APIs](#) externas de pago.

## Abstract

Meetings are a fundamental tool for information sharing, decision-making, and coordination within work teams. However, the discussed content is often limited to manual minutes or personal notes, hindering efficient information retrieval. Furthermore, although digital recordings are common, searching for specific data within them remains a slow and impractical process.

To address this, this project proposes the development of an intelligent system for the automated management of meeting recordings. The platform integrates a workflow starting with speech transcription. Based on the generated text, the system can extract automated summaries of key points and allows users to perform natural language queries on the processed content. This latter functionality is supported by a [RAG](#) architecture, providing the system with the capability to answer specific questions about any stored session. The entire



system is implemented as a web application that centralizes recording management, allowing users to view transcripts and summaries, as well as interact with the assistant through an intuitive interface.

The core value of this proposal lies in its commitment to privacy and data sovereignty. Unlike cloud-based commercial solutions, this system utilizes open-source tools and enables the execution of [LLM](#) locally. This ensures that sensitive meeting information remains within the user's environment, while also eliminating dependency on subscriptions and paid external [APIs](#).

**Palabras clave:**

- Inteligencia Artificial
- Procesamiento de lenguaje natural
- Generación aumentada por recuperación
- Voz a texto
- Privacidad
- Aplicación Web
- Código abierto

**Keywords:**

- Inteligencia Artificial
- Natural Language Processing
- Retrieval Augmented Generation
- Speech to Text
- Privacy
- Web Application
- Open Source

# Índice general

---

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducción</b>                                      | <b>1</b>  |
| 1.1      | Motivación . . . . .                                     | 1         |
| 1.2      | Objetivos . . . . .                                      | 2         |
| 1.3      | Estructura de la memoria . . . . .                       | 2         |
| <b>2</b> | <b>Estudio de Mercado</b>                                | <b>4</b>  |
| 2.1      | Estado de la cuestión . . . . .                          | 4         |
| 2.1.1    | Limitaciones en la obtención de información . . . . .    | 4         |
| 2.1.2    | Limitaciones en la recuperación de información . . . . . | 5         |
| 2.2      | Opciones existentes . . . . .                            | 5         |
| 2.2.1    | Asistentes de reuniones (SaaS) . . . . .                 | 5         |
| 2.2.2    | Soluciones Integradas . . . . .                          | 6         |
| 2.2.3    | Alternativas de código abierto . . . . .                 | 7         |
| 2.3      | Análisis comparativo . . . . .                           | 7         |
| 2.4      | Análisis de la problemática común . . . . .              | 8         |
| 2.5      | Análisis DAFO . . . . .                                  | 9         |
| <b>3</b> | <b>Fundamentos técnicos</b>                              | <b>10</b> |
| 3.1      | Conceptos fundamentales . . . . .                        | 10        |
| 3.1.1    | Procesamiento de audio: STT y Diarización . . . . .      | 10        |
| 3.1.2    | Modelos de Lenguaje de Gran Tamaño (LLM) . . . . .       | 10        |
| 3.1.3    | Model Context Protocol . . . . .                         | 11        |
| 3.1.4    | Generación Aumentada por Recuperación (RAG) . . . . .    | 12        |
| 3.1.5    | Bases de datos vectoriales y <i>Embeddings</i> . . . . . | 12        |
| 3.2      | Arquitectura del sistema . . . . .                       | 12        |
| 3.3      | Herramientas . . . . .                                   | 13        |
| 3.3.1    | <i>Frontend</i> . . . . .                                | 13        |
| 3.3.2    | <i>Backend</i> . . . . .                                 | 17        |

|          |                                                                                           |           |
|----------|-------------------------------------------------------------------------------------------|-----------|
| 3.3.3    | Herramientas de desarrollo y seguimiento . . . . .                                        | 20        |
| <b>4</b> | <b>Metodología y Planificación</b>                                                        | <b>22</b> |
| 4.1      | Metodología empleada . . . . .                                                            | 22        |
| 4.1.1    | Fundamentos: Scrum y Kanban . . . . .                                                     | 22        |
| 4.1.2    | Adaptación de Scrumban para un único desarrollador . . . . .                              | 23        |
| 4.1.3    | Gestión del código mediante Gitflow . . . . .                                             | 23        |
| 4.2      | Análisis de riesgos . . . . .                                                             | 24        |
| 4.3      | Planificación inicial . . . . .                                                           | 24        |
| 4.4      | Estimación de costes . . . . .                                                            | 26        |
| 4.4.1    | Recursos Humanos . . . . .                                                                | 26        |
| 4.4.2    | Recursos técnicos . . . . .                                                               | 27        |
| 4.4.3    | Cómputo total . . . . .                                                                   | 27        |
| 4.5      | Seguimiento y Desviaciones . . . . .                                                      | 27        |
| 4.5.1    | Actualización de los costes . . . . .                                                     | 28        |
| <b>5</b> | <b>Desarrollo</b>                                                                         | <b>30</b> |
| 5.1      | Sprint 1 . . . . .                                                                        | 30        |
| 5.1.1    | Estudio de mercado . . . . .                                                              | 30        |
| 5.1.2    | Especificación del proyecto . . . . .                                                     | 30        |
| 5.1.3    | Diseño de la interfaz . . . . .                                                           | 31        |
| 5.1.4    | Elección de herramientas . . . . .                                                        | 31        |
| 5.1.5    | Elección de la Licencia Open Source . . . . .                                             | 32        |
| 5.1.6    | Asignación de nombre comercial . . . . .                                                  | 32        |
| 5.2      | Sprint 2 . . . . .                                                                        | 32        |
| 5.2.1    | Diseño de la arquitectura del <i>backend</i> . . . . .                                    | 32        |
| 5.2.2    | Implementación . . . . .                                                                  | 35        |
| 5.3      | Sprint 3 . . . . .                                                                        | 38        |
| 5.3.1    | Implementación del asistente conversacional . . . . .                                     | 38        |
| 5.3.2    | Implementación de la autenticación de usuarios . . . . .                                  | 40        |
| 5.4      | Sprint 4 . . . . .                                                                        | 45        |
| 5.4.1    | Implementar ajustes de usuario . . . . .                                                  | 45        |
| 5.4.2    | Definición de la arquitectura a alto nivel del <i>frontend</i> de la aplicación . . . . . | 46        |
| 5.4.3    | Implementación de la autenticación . . . . .                                              | 49        |
| 5.4.4    | Implementación de la navegación . . . . .                                                 | 51        |
| 5.4.5    | Implementar subida de reuniones . . . . .                                                 | 51        |
| 5.5      | Sprint 5 . . . . .                                                                        | 54        |
| 5.5.1    | Vista del chat . . . . .                                                                  | 54        |

|          |                                                                 |            |
|----------|-----------------------------------------------------------------|------------|
| 5.5.2    | Lista de chats . . . . .                                        | 56         |
| 5.5.3    | Vista de reuniones . . . . .                                    | 57         |
| 5.5.4    | Vista de la transcripción y el resumen de una reunión . . . . . | 60         |
| 5.5.5    | Vista de ajustes de usuario . . . . .                           | 61         |
| 5.6      | Sprint 6 . . . . .                                              | 64         |
| 5.6.1    | Implementación del flujo SSE . . . . .                          | 64         |
| 5.6.2    | Internacionalización . . . . .                                  | 67         |
| 5.6.3    | Añadir tema oscuro a la aplicación . . . . .                    | 68         |
| 5.6.4    | Añadir <i>proxy</i> para conexiones seguras . . . . .           | 69         |
| 5.7      | Sprint 7 . . . . .                                              | 70         |
| <b>6</b> | <b>Conclusiones</b>                                             | <b>71</b>  |
| 6.1      | Consecución de objetivos y logros técnicos . . . . .            | 71         |
| 6.2      | Impacto formativo y crecimiento profesional . . . . .           | 72         |
| <b>7</b> | <b>Futuros desarrollos</b>                                      | <b>73</b>  |
| <b>A</b> | <b>Épicas</b>                                                   | <b>76</b>  |
| <b>B</b> | <b>Requisitos del Sistema</b>                                   | <b>79</b>  |
| <b>C</b> | <b>Casos de Uso</b>                                             | <b>83</b>  |
| <b>D</b> | <b>Mockups</b>                                                  | <b>92</b>  |
| <b>E</b> | <b>Vistas de la interfaz de usuario</b>                         | <b>98</b>  |
| <b>F</b> | <b>Guía de instalación</b>                                      | <b>115</b> |
| <b>G</b> | <b>Manual de usuario</b>                                        | <b>125</b> |
|          | <b>Lista de acrónimos</b>                                       | <b>145</b> |
|          | <b>Glosario</b>                                                 | <b>147</b> |
|          | <b>Bibliografía</b>                                             | <b>152</b> |

# Índice de figuras

---

|      |                                                                                                                                       |    |
|------|---------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1  | Diagrama de Gantt de la planificación inicial . . . . .                                                                               | 26 |
| 4.2  | Diagrama de Gantt del seguimiento del proyecto . . . . .                                                                              | 29 |
| 5.1  | Diagrama de Casos de Uso . . . . .                                                                                                    | 31 |
| 5.2  | Diagrama de secuencia a alto nivel de la aplicación . . . . .                                                                         | 33 |
| 5.3  | Diagrama de componentes del <i>backend</i> a alto nivel, estructurado por capas lógicas y dominios . . . . .                          | 34 |
| 5.4  | Captura de Swagger UI con los <i>endpoints</i> del dominio <i>meeting</i> . . . . .                                                   | 35 |
| 5.5  | Diagrama de componentes del <i>backend</i> . En azul se pueden apreciar los componentes implementados en este <i>sprint</i> . . . . . | 37 |
| 5.6  | Diagrama de secuencia del procesamiento de las reuniones ejecutado al recibir una petición HTTP POST <code>/meetings</code> . . . . . | 38 |
| 5.7  | Captura de Swagger UI con los <i>endpoints</i> del dominio <i>conversation</i> . . . . .                                              | 39 |
| 5.8  | Diagrama de componentes de <i>backend</i> . En color azul se resaltan los módulos implementados en esta sección . . . . .             | 41 |
| 5.9  | Captura de Swagger UI con los <i>endpoints</i> del dominio <i>auth</i> . . . . .                                                      | 42 |
| 5.10 | Diagrama de componentes del <i>backend</i> . En color azul se resaltan los módulos implementados en esta sección . . . . .            | 43 |
| 5.11 | Diagrama Entidad Relación de la base de datos . . . . .                                                                               | 44 |
| 5.12 | Diagrama de componentes del <i>backend</i> . En color azul se resaltan los módulos implementados en esta sección . . . . .            | 46 |
| 5.13 | Captura de Swagger UI con los <i>endpoints</i> del dominio <i>settings</i> . . . . .                                                  | 47 |
| 5.14 | Diagrama Entidad Relación de la base de datos . . . . .                                                                               | 48 |
| 5.15 | Diagrama de componentes del <i>frontend</i> a alto nivel . . . . .                                                                    | 49 |
| 5.16 | Vistas para un usuario no autenticado . . . . .                                                                                       | 50 |
| 5.17 | Comparativa de la interfaz de la aplicación según el estado del panel lateral . . . . .                                               | 52 |
| 5.18 | Modal de subir reuniones . . . . .                                                                                                    | 53 |

|      |                                                                                                   |    |
|------|---------------------------------------------------------------------------------------------------|----|
| 5.19 | Vistas del asistente conversacional en su estado inicial y durante una consulta.                  | 55 |
| 5.20 | Panel lateral con la lista de chats . . . . .                                                     | 56 |
| 5.21 | Vista de reuniones . . . . .                                                                      | 58 |
| 5.22 | Comparativa del listado de reuniones según el estado del filtro de búsqueda<br>por fecha. . . . . | 59 |
| 5.23 | Editar metadatos de una reunión . . . . .                                                         | 59 |
| 5.24 | Modal para confirmar la eliminación de una reunión . . . . .                                      | 59 |
| 5.25 | Vistas de transcripción y de resumen de una reunión . . . . .                                     | 61 |
| 5.26 | Diferentes paneles de la ventana de configuración del sistema. . . . .                            | 63 |
| 5.27 | Vista de subida de reuniones actualizada . . . . .                                                | 66 |
| 5.28 | Indicadores visuales de estado del asistente durante el procesamiento de la<br>consulta . . . . . | 67 |
| 5.29 | Vista de la interfaz de configuración general en sus variantes visuales . . . . .                 | 69 |

# Índice de tablas

---

|     |                                                                                                         |    |
|-----|---------------------------------------------------------------------------------------------------------|----|
| 2.1 | Matriz comparativa entre las herramientas del mercado y la solución propuesta.                          | 8  |
| 2.2 | Análisis DAFO del proyecto . . . . .                                                                    | 9  |
| 4.1 | Análisis de riesgos del proyecto y planes de mitigación. . . . .                                        | 25 |
| 4.2 | Coste de los recursos humanos . . . . .                                                                 | 27 |
| 4.3 | Comparativa de costes estimados iniciales frente a costes finales reales tras las desviaciones. . . . . | 29 |

# Introducción

---

En este capítulo se exponen las razones por las cuales se decidió realizar este proyecto, así como también los objetivos del mismo y un breve resumen sobre la estructura de este documento.

## 1.1 Motivación

Las reuniones son el punto clave donde los equipos de trabajo intercambian ideas, toman decisiones y coordinan sus tareas. Sin embargo, en el día a día de cualquier organización, el volumen de estas reuniones genera una cantidad de información tan grande que muchas veces acaba perdiéndose. Ser capaz de organizar y recuperar lo que se ha hablado es fundamental para que el trabajo en equipo sea realmente eficiente.

Tradicionalmente, para registrar lo que ocurre en una reunión se ha recurrido a las actas manuales o a notas personales. El problema de este método es que las notas dependen totalmente de la visión de quien las escribe, lo que hace que a menudo sean incompletas o contengan errores. Por otro lado, aunque hoy en día es muy común grabar las sesiones (especialmente con el auge del teletrabajo y plataformas como *Teams* [1] o *Zoom* [2]), estas grabaciones suelen quedar guardadas sin volver a ser consultadas. Buscar un dato concreto o una decisión específica en una grabación relativamente larga es un proceso lento y poco práctico si no se sabe exactamente en qué minuto se trató cada tema.

En este escenario, el gran avance de la [Inteligencia Artificial](#) y el [Procesamiento del Lenguaje Natural](#) permite plantear una solución a estos problemas. Actualmente, es posible automatizar tareas que antes eran manuales, como, por ejemplo, transcribir el audio a texto o resumir los puntos más importantes de una conversación. [3, 4]

Sin embargo, la mayoría de las herramientas comerciales que ofrecen estos servicios funcionan en la nube. Esto supone dos inconvenientes principales: el coste de las suscripciones y la falta de privacidad al tener que enviar audios con información sensible a servidores de



terceros (como se analizará en la Sección 2.3). Por este motivo, el presente proyecto nace con la idea de crear una herramienta que integre todo este proceso de forma local.

## 1.2 Objetivos

Por todo lo expuesto en la Sección 1.1, el objetivo principal de este proyecto es que cualquier usuario pueda transcribir sus reuniones, generar resúmenes y consultar dudas sobre ellas mediante un *chatbot*, garantizando en todo momento que la información no sale de su propio equipo y que no depende de servicios externos de pago.

Por ello, se definen los siguientes objetivos concretos:

1. Desarrollar un módulo capaz de extraer el contenido hablado de grabaciones en formato texto.
2. Desarrollar un módulo de generación automática de resúmenes relevantes a partir del contenido transcrito en el paso anterior.
3. Desarrollar un módulo para la realización de consultas sobre el contenido procesado mediante lenguaje natural.
4. Desarrollar una interfaz que facilite la interacción del usuario con el sistema.
5. Hacer que la solución propuesta sea fácilmente desplegable con docker.

## 1.3 Estructura de la memoria

A continuación, se detalla la estructura de la memoria, junto a una breve explicación sobre el contenido de cada uno de los diferentes apartados:

1. **Introducción:** El presente capítulo expone la motivación del proyecto y los objetivos del mismo.
2. **Estudio de mercado:** Se analizan las soluciones actuales, comentando sus ventajas, desventajas, la necesidad de desarrollar este proyecto y se presenta un análisis DAFO de este proyecto.
3. **Fundamentos técnicos:** Se detallan diferentes conceptos esenciales como los *Large Language Models* (LLMs) o la arquitectura *Retrieval Augmented Generation* (RAG), posteriormente se comenta de manera breve la arquitectura del sistema y finalmente se detallan las herramientas utilizadas.

4. **Metodología y planificación:** Se detalla la metodología utilizada y la planificación inicial del proyecto, abarcando los *sprints* identificados, la planificación temporal y los costes previstos. Además, también se realiza el seguimiento del proyecto.
5. **Desarrollo:** En esta sección se narra el desarrollo de cada uno de los *sprints* identificados al comienzo del proyecto.
6. **Conclusiones:** Se indican las conclusiones del proyecto, revisando tanto si los objetivos propuestos al comienzo del proyecto fueron cumplidos de manera satisfactoria como todo lo aprendido a lo largo del desarrollo.
7. **Futuros desarrollos:** Por último, se contemplan futuros desarrollos que puedan ser llevados a cabo en iteraciones siguientes.

# Estudio de Mercado

---

En esta sección se analizará el estado de la cuestión y las principales alternativas ya existentes a este proyecto. Posteriormente, se realizará un análisis **Debilidades Amenazas Fortalezas Oportunidades (DAFO)** sobre el proyecto propuesto.

Como bien se ha mencionado en la Sección 1.1, las reuniones constituyen el principal escenario en el que se toman decisiones y se coordinan equipos. Es por ello que resulta de gran interés para los miembros de un equipo poder acceder de manera eficaz a la información fruto de estos encuentros.

## 2.1 Estado de la cuestión

En primer lugar, es necesario entender claramente las limitaciones presentes tanto en la manera actual de recopilar la información como de recuperarla.

### 2.1.1 Limitaciones en la obtención de información

Existen tres formas principales de retener la información, cada una con sus limitaciones:

- **Actas manuales:** Aunque permiten anotar lo que cada integrante considera importante, esto es un arma de doble filo, pues se pueden pasar por alto detalles que a otros sí les parezcan relevantes. En reuniones dinámicas, este sistema es propenso a errores.
- **Transcripciones a mano:** Además del factor humano y el riesgo de errores, suelen requerir que un integrante se dedique exclusivamente a ello, lo que hace que el proceso resulte inviable en la mayoría de ocasiones.
- **Grabaciones:** Son completamente objetivas y requieren poco esfuerzo por parte de los integrantes. Sin embargo, ocupan mucho espacio en memoria, lo que resulta poco práctico para equipos que generan muchas horas de metraje.

### 2.1.2 Limitaciones en la recuperación de información

Además de las limitaciones a la hora de obtener la información, existen barreras críticas a la hora de intentar recuperarla de manera efectiva:

- **Actas manuales:** Típicamente son breves y constan únicamente de pequeños puntos críticos o breves resúmenes. Al faltar el contexto completo de la conversación, si se desea consultar los argumentos que llevaron a una decisión, la información puede resultar insuficiente.
- **Transcripciones:** Recuperar la información se vuelve una tarea compleja debido a que el volumen de texto aumenta considerablemente. Sin una herramienta que filtre el contenido, el usuario debe realizar una lectura exhaustiva para encontrar un dato concreto, lo que consume una gran cantidad de tiempo.
- **Grabaciones:** El problema se vuelve aún peor debido a la naturaleza lineal del formato. Buscar algo en concreto en horas de metraje resulta muy complicado si no se dispone de cierta información previa, como saber en qué minuto, de manera aproximada, se trata el tema de interés. Esto hace que realizar búsquedas en este tipo de información no sea del todo práctico.

## 2.2 Opciones existentes

A continuación se presentan las diferentes alternativas para resolver los problemas antes señalados.

### 2.2.1 Asistentes de reuniones (SaaS)

Gran parte de estas soluciones se materializan en la forma de asistentes de reuniones *Software as a Service* (SaaS).

#### Fireflies

**Fireflies** [5] constituye una plataforma que permite a los usuarios generar transcripciones y resúmenes de sus encuentros en tiempo real. Para facilitar su adopción, la herramienta ofrece una gran versatilidad de acceso a través de una extensión de *Chrome*, así como aplicaciones tanto para escritorio como para dispositivos móviles.

En cuanto a su modelo de negocio, Fireflies ofrece cuatro planes de suscripción que oscilan entre los 0 y los 39 \$/mes por usuario. Cada uno de estos niveles dispone de diferentes ventajas competitivas, principalmente centradas en los minutos de almacenamiento permitidos y en el número de *créditos* disponibles. Estos créditos son esenciales para realizar acciones avanzadas,

como por ejemplo consultar al asistente de **IA** cuestiones específicas sobre lo tratado durante el transcurso de la reunión.

### **Otter.ai**

**Otter.ai** [6] es una herramienta muy similar a la anterior, centrada en la generación de transcripciones, resúmenes automáticos y la interacción mediante un *chatbot*. Al igual que su competidor directo, ofrece soporte multiplataforma mediante extensiones de navegador y aplicaciones para móvil y escritorio.

Respecto a sus tarifas, dispone principalmente de tres planes comerciales que se sitúan entre los 0 y los 20 \$/mes. Las limitaciones más destacadas entre estos niveles residen en el número de ficheros de audio y vídeo que el usuario puede subir mensualmente, así como en la duración máxima de los mismos. Cabe destacar que la plataforma dispone de un plan más avanzado orientado a grandes corporaciones, el cual no es accesible de forma directa al público general, sino que requiere de una contratación a medida.

### **2.2.2 Soluciones Integradas**

Otras alternativas buscan integrar estos asistentes en plataformas ampliamente utilizadas para realizar reuniones telemáticas en el ámbito profesional.

#### **Microsoft Teams Premium**

Dentro del ámbito empresarial y académico, **Microsoft Teams** [1] es una de las plataformas más utilizadas para la coordinación de equipos. Esta herramienta ofrece el plan específico **Microsoft Teams Premium** [7], el cual integra funcionalidades avanzadas de **IA** como la toma automática de notas, la transcripción en directo y la generación de resúmenes inteligentes de la sesión. Su precio se sitúa en una franja competitiva de 8.70 €/mes por usuario, ofreciendo una integración total con el resto de la *suite* de productividad de la compañía.

#### **Zoom AI Companion**

**Zoom AI Companion** [8] permite a los usuarios generar transcripciones y notas de las reuniones, además de disponer de un *chatbot* para realizar consultas en lenguaje natural sobre el contenido tratado.

En términos de costes, se ofertan tres planes que varían entre los 0 y los 16 €/mes, aunque este precio se ve reducido si se opta por una facturación anual. Las diferencias más notables entre estos niveles son el límite de minutos de duración de cada reunión y la capacidad de integración con herramientas externas de terceros; por ejemplo, la funcionalidad de enviar automáticamente los resúmenes generados a canales de *Slack*.

### Google Meet

**Google Meet** [9] permite la generación de transcripciones automáticas y la toma de notas en tiempo real. A diferencia de las opciones anteriores, estas características no se comercializan como un producto aislado, sino que vienen incluidas dentro de los planes de **Google Workspace** [10]. Esto hace que resulte una opción de gran interés para organizaciones que ya dependen de otros servicios de la compañía para su gestión diaria.

#### 2.2.3 Alternativas de código abierto

##### Meetly

**Meetly** [11] es una aplicación de código abierto bajo [licencia MIT](#) que funciona como un asistente integral de reuniones. Su propuesta incluye la generación de transcripciones y resúmenes, permitiendo procesar tanto audio en tiempo real como archivos de vídeo y audio previamente grabados. Una de sus características más destacadas es su interfaz amigable, que permite a los usuarios editar manualmente los archivos generados. Además, ofrece flexibilidad técnica al permitir modificar el modelo de lenguaje utilizado, siempre que los proveedores sean compatibles con el [API de OpenAI](#) [12].

Sin embargo, el proyecto se divide en dos vertientes: **Meetly Community Edition**, que es la versión de código abierto, y **Meetly Pro**, una versión de pago que desbloquea funcionalidades adicionales. Entre estas ventajas exclusivas de la versión Pro se encuentra la posibilidad de modificar el *prompt* que utilizan los [LLM](#) para generar los resúmenes. Asimismo, una funcionalidad crítica denominada **Chat with Meetings** —que permite realizar consultas complejas sobre las reuniones— está reservada para la versión de pago y, en el momento de redactar este documento, todavía figura como *disponible pronto* en el repositorio oficial del proyecto [13].

### 2.3 Análisis comparativo

Para sintetizar el estudio de las alternativas existentes y justificar el valor de la solución propuesta en este proyecto, se ha realizado una comparativa técnica y funcional. Se han seleccionado cinco criterios clave que responden a las necesidades críticas de las organizaciones en la actualidad:

- **Despliegue:** Indica si la herramienta se ejecuta obligatoriamente en servidores de terceros (Nube) o si permite una ejecución en la infraestructura propia del usuario (Local).
- **Privacidad de los datos:** Mide el nivel de control y soberanía que tiene el usuario sobre sus grabaciones e información sensible.

- **Modelo de coste:** Determina si el acceso a las herramientas avanzadas requiere el pago de cuotas recurrentes.
- **Código abierto:** Disponibilidad del código fuente para su auditoría, modificación y personalización por parte de la comunidad.
- **Chat:** Capacidad de interactuar directamente con el contenido de las reuniones mediante lenguaje natural para extraer respuestas específicas.

En la Tabla 2.1 se presenta el contraste entre las principales soluciones comerciales, las alternativas de código abierto evaluadas y el presente proyecto.

| Solución                | Despliegue | Privacidad    | Coste       | Open Source | Chat              |
|-------------------------|------------|---------------|-------------|-------------|-------------------|
| Fireflies               | Nube       | Externa       | Suscripción | No          | Sí (con créditos) |
| Otter.ai                | Nube       | Externa       | Suscripción | No          | Sí (Premium)      |
| Teams Premium           | Nube       | Ecosistema MS | Suscripción | No          | Sí                |
| Zoom AI                 | Nube       | Externa       | Suscripción | No          | Sí                |
| Meetly (CE)             | Nube/Local | Media         | Gratuito    | Sí          | No (Solo Pro)     |
| La propuesta presentada | 100% Local | Total         | Gratuito    | Sí          | Sí                |

Tabla 2.1: Matriz comparativa entre las herramientas del mercado y la solución propuesta.

## 2.4 Análisis de la problemática común

Todas estas herramientas ofrecen características similares, pero disponen de una problemática común: son *software* privativo. Al depender de una entidad externa, el usuario queda expuesto a subidas de precios o a que la entidad limite los planes existentes, ya sea reduciendo la duración permitida de las reuniones o el número de procesos que se pueden ejecutar mensualmente.

Esta amenaza resulta altamente preocupante debido a la situación financiera de las empresas proveedoras de los LLMs. Por ejemplo, *OpenAI* es una entidad que actualmente reporta pérdidas en lugar de beneficios, lo que puede derivar en incrementos de precio a medio o largo plazo para compensar sus balances. Según proyecciones financieras, *OpenAI* predice pérdidas de aproximadamente 44.000 millones de \$ entre los años 2023 y 2028, a pesar de que se espera alcanzar los 14.000 millones de \$ de beneficios en el año 2029 [14].

Esta tendencia ya ha comenzado a materializarse en los últimos meses. En el ámbito de las herramientas de IA generativa aplicadas a la programación, diversos proveedores han encarecido sus planes o han restringido las condiciones de sus modalidades gratuitas [15].

Además de los riesgos económicos, existe la problemática de la soberanía de los datos. Para ciertas instituciones, supone un conflicto crítico que la información sensible tratada en

sus reuniones sea procesada en servidores de terceros, quedando supeditada a políticas de privacidad ajenas y fuera de su control directo.

## 2.5 Análisis DAFO

Para sintetizar el estudio realizado, se presenta el análisis DAFO del proyecto, donde se contrastan las capacidades de la solución propuesta frente al contexto del mercado actual.

| Fortalezas                                                                                                                                                                                                                          | Debilidades                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- Ejecución 100% local (Privacidad absoluta).</li> <li>- Sin costes de suscripción ni límites de uso por plan.</li> <li>- Control total sobre el flujo y almacenamiento de datos.</li> </ul> | <ul style="list-style-type: none"> <li>- Requiere <i>hardware</i> con potencia de cómputo (GPU/RAM).</li> <li>- Mayor complejidad inicial de instalación y despliegue.</li> <li>- Ciertos <i>modelos de pesos abiertos</i> pueden no ofrecer resultados óptimos en tareas específicas.</li> </ul> |
| Oportunidades                                                                                                                                                                                                                       | Amenazas                                                                                                                                                                                                                                                                                          |
| <ul style="list-style-type: none"> <li>- Demanda creciente de soberanía de datos en sectores sensibles.</li> <li>- Mejora constante y rápida de las capacidades de los modelos locales.</li> </ul>                                  | <ul style="list-style-type: none"> <li>- Competencia de grandes corporaciones con recursos masivos.</li> <li>- Evolución de las soluciones.</li> </ul>                                                                                                                                            |

Tabla 2.2: Análisis DAFO del proyecto

A partir del DAFO expuesto, se concluye que las fortalezas y las oportunidades superan con creces tanto las debilidades como las amenazas, por lo que se toma la decisión de llevar a cabo el desarrollo del proyecto.



# Fundamentos técnicos

---

En esta sección se abordan tanto los conocimientos técnicos que sirven de base para este proyecto como también las diferentes herramientas utilizadas para llevar a cabo el desarrollo del mismo, además de una justificación sobre la elección de las mismas.

## 3.1 Conceptos fundamentales

Para comprender el funcionamiento técnico de la aplicación, es necesario introducir los conceptos en los que se basa el procesamiento de la información, desde la captura del audio hasta la generación de respuestas inteligentes.

### 3.1.1 Procesamiento de audio: STT y Diarización

El primer paso del sistema consiste en transformar la señal de audio en texto estructurado. Este proceso se divide en dos tecnologías clave:

- ***Speech to Text (STT)***: Es la tecnología que permite la transcripción automática del habla. Utiliza modelos de aprendizaje profundo para mapear frecuencias sonoras con unidades lingüísticas (fonemas y palabras) [16].
- **Diarización de hablantes**: En una reunión con múltiples participantes, no basta con saber qué se dice, sino también quién lo dice. La diarización consiste en particionar el audio en segmentos según la identidad del locutor, permitiendo que la transcripción final tenga una estructura de diálogo organizada por turnos de palabra [17].

### 3.1.2 Modelos de Lenguaje de Gran Tamaño (LLM)

Los *Large Language Models (LLMs)* son modelos de inteligencia artificial entrenados con vastas cantidades de texto para comprender y generar lenguaje humano. Su arquitectura, basada habitualmente en *transformers* [18], les permite captar el contexto de una conversación

de forma mucho más profunda que los sistemas tradicionales. En este proyecto, los **LLMs** se encargan de dos tareas críticas: sintetizar la información en resúmenes ejecutivos y actuar como el motor de razonamiento para responder a las preguntas del usuario.

### 3.1.3 Model Context Protocol

El *Model Context Protocol* (MCP) [19] es un estándar abierto diseñado para centralizar y unificar la forma en que los **LLMs** se conectan de manera segura con fuentes de datos, herramientas y aplicaciones externas. Desarrollado originalmente por Anthropic, este protocolo surge con el objetivo de mitigar la fragmentación existente en el desarrollo de aplicaciones de Inteligencia Artificial, donde tradicionalmente cada herramienta o fuente de datos requería una implementación de un conector a medida o una abstracción *ad-hoc*.

Este protocolo consta de una arquitectura cliente-servidor:

- **Cliente:** Aplicaciones de **IA** que actúan como directores del flujo de trabajo. El cliente es el encargado de transferir el contexto del usuario hacia el modelo y de gestionar el ciclo de vida de la interacción.
- **Servidores:** Servicios que exponen ciertas herramientas, como leer o modificar archivos, ejecutar comandos, realizar búsquedas en internet, etc. Estos servidores interactúan directamente con el entorno local o con servicios de terceros para ofrecer tres tipos de recursos primarios:
  - *Prompts:* Plantillas predefinidas de texto estructurado que guían el comportamiento del **LLM**.
  - *Resources:* Datos contextuales de lectura, como pueden ser archivos de texto o esquemas de bases de datos.
  - *Tools:* Funciones que el **LLM** puede ejecutar.

El protocolo opera habitualmente sobre transportes estándar como flujos de entrada/salida estándar (*stdio*) en entornos locales o *Server Sent Events* en entornos de red.

A pesar de las ventajas arquitectónicas que ofrece **MCP** para desacoplar el motor de inferencia de las fuentes de datos, su correcto funcionamiento depende en gran parte de las capacidades de realizar llamadas a funciones del modelo subyacente. El **LLM** debe ser capaz de analizar correctamente la descripción de las herramientas expuestas por el servidor **MCP**, generar una petición perfectamente válida que respete el esquema requerido y procesar la respuesta de forma coherente.

### 3.1.4 Generación Aumentada por Recuperación (RAG)

La arquitectura *Retrieval Augmented Generation (RAG)* [20] es el núcleo que permite al LLM acceder al contenido de las reuniones. Su funcionamiento se basa en un flujo de tres pasos:

1. **Indexación:** Los textos de las reuniones se fragmentan en trozos pequeños y se convierten en *embeddings*.
2. **Recuperación:** Cuando el usuario realiza una consulta, el sistema no busca la pregunta directamente. En su lugar, un LLM procesa el texto original del usuario para generar un contenido sintético optimizado. Este nuevo texto es el que se utiliza para realizar la búsqueda por proximidad en la base de datos vectorial, localizando así los fragmentos de la reunión que guardan la mayor similitud semántica con la intención del usuario.
3. **Generación:** Los fragmentos recuperados se envían al LLM que genera la respuesta final junto con la pregunta original, sirviendo como contexto para que la respuesta sea precisa y esté basada exclusivamente en lo hablado en la reunión.

### 3.1.5 Bases de datos vectoriales y *Embeddings*

A diferencia de las bases de datos relacionales, las vectoriales están diseñadas para almacenar y buscar información según su “significado”. Esto se consigue mediante los *embeddings*: representaciones matemáticas donde palabras o frases con significados similares se sitúan cerca en un espacio multidimensional [21]. Esta tecnología es la que permite que el sistema entienda que una pregunta sobre “acuerdos económicos” está relacionada con un fragmento de la reunión donde se habló de “presupuesto”.

## 3.2 Arquitectura del sistema

Para el desarrollo de este proyecto se ha optado por un modelo de **arquitectura cliente-servidor**. Esta estructura permite separar la lógica de presentación de la lógica de negocio y el procesamiento pesado de datos, lo cual es fundamental en una aplicación que requiere una alta carga computacional para la ejecución de modelos de IA.

A continuación, se describen los dos pilares sobre los que se asienta el sistema:

- **Cliente (*frontend*):** Es la parte de la aplicación con la que el usuario interactúa directamente. Su función principal es la gestión de la interfaz, permitiendo la carga de archivos de audio, la visualización de los resultados y la interacción con el *chatbot*. Al delegar el procesamiento al servidor, el cliente puede ejecutarse de forma fluida en cualquier navegador web sin necesidad de disponer de *hardware* especializado.

- **Servidor (*backend*):** Se encarga de la gestión de la base de datos y de las interacciones con los modelos de *IA*.

La comunicación entre el *frontend* y el *backend* se realiza mediante un *API REST*, que actúa como el punto de entrada para todas las operaciones del sistema. Para adaptarse a las necesidades de una aplicación de *IA*, esta interfaz utiliza diferentes estrategias de transferencia de datos:

- **Peticiones estándar (*HTTP*) [22]:** La mayor parte de la interacción sigue el modelo tradicional de petición-respuesta. El cliente solicita una acción (como recuperar la lista de reuniones o la lista de chats) y el servidor responde con los datos correspondientes en formato *JSON*.
- **Flujos de eventos (*SSE*) [23]:** Integrado dentro del propio *API REST* [24], se utiliza el protocolo *SSE* para aquellas tareas que no son instantáneas. En lugar de cerrar la conexión, ciertos *endpoints* mantienen un canal abierto para enviar actualizaciones en tiempo real al usuario. En concreto, esto se utiliza para el endpoint de añadir reuniones, pues el procesamiento de estas puede ser bastante lento, y debido a que el usuario puede subir varias reuniones a la vez, utilizar *SSE* permite al *frontend* mostrar el estado de cada una de las reuniones.
- **Comunicación bidireccional (*WS*) [25]:** A diferencia de los anteriores, el módulo de chat utiliza *WS* para establecer una conexión persistente. Esto permite un intercambio de mensajes fluido y en tiempo real entre el usuario y el asistente, permitiendo que las respuestas de la *IA* se reciban de forma asíncrona.

Este enfoque permite que la aplicación sea eficiente: utiliza el estándar *REST* para la gestión de datos y reserva los canales de flujo y tiempo real para las funciones que realmente lo requieren.

### 3.3 Herramientas

A continuación, se detallan las diferentes herramientas empleadas durante el desarrollo del proyecto.

#### 3.3.1 *Frontend*

Para la implementación del *frontend*, se han utilizado las siguientes herramientas.

## React

**React** [26] es una librería de JavaScript orientada a la creación de interfaces de usuario de forma declarativa. Su filosofía de desarrollo se basa en la arquitectura de componentes reutilizables, lo que permite segmentar la interfaz en piezas independientes, facilitando así la escalabilidad y el mantenimiento del código a lo largo de todo el proyecto.

Aunque existen otras alternativas relevantes en el desarrollo de *Single Page Applications* (SPAs), como Angular [27] o Vue [28], se ha seleccionado **React** por los siguientes motivos:

- **Ecosistema y madurez:** Cuenta con la comunidad más extensa de desarrolladores, lo que garantiza una amplia disponibilidad de recursos, librerías de terceros y soporte técnico.
- **Curva de aprendizaje:** Comparada con *frameworks* más rígidos o complejos como Angular, React ofrece una curva de aprendizaje más suave, permitiendo una implementación más ágil de las funcionalidades del proyecto.

## TypeScript

TypeScript [29] es un *superset* de JavaScript que añade *tipado estático* al lenguaje, ofreciendo así una mejor experiencia de desarrollo, evitando errores y haciendo que el código sea más legible.

## Tailwind CSS

Tailwind CSS [30] es un *framework* de CSS que permite aplicar estilos de manera ágil y directa sobre los componentes de la interfaz, utilizando un conjunto de clases predefinidas que cubren prácticamente todas las necesidades de diseño. Se ha optado utilizar esta herramienta principalmente por la agilidad en el desarrollo y por su consistencia visual.

## React-markdown

React-markdown [31] es un componente de React que transforma texto en formato Mark-down a código HTML de forma segura. En este proyecto se utiliza para renderizar el contenido de los resúmenes de las reuniones y las respuestas del asistente conversacional, permitiendo que estos textos incluyan elementos como listas, títulos, texto en negrita o cursiva.

## Zustand

Zustand [32] es una librería de gestión de estado global para JavaScript que permite centralizar la información de la aplicación de forma externa al árbol de componentes. Su uso

principal en este proyecto es la gestión del estado de la sesión, el historial de mensajes y la configuración de las reuniones, evitando así el fenómeno del *prop drilling*.

Aunque existen otras alternativas consolidadas, la elección de Zustand se justifica por los siguientes motivos técnicos:

- **Frente a Redux Toolkit [33]:** A pesar de ser la herramienta más extendida, Redux requiere una gran cantidad de código repetitivo (*boilerplate*) para definir acciones, reductores y selectores. Zustand ofrece un *API* mucho más simplificada y directa, lo que permite reducir la complejidad del código y acelerar el tiempo de desarrollo.
- **Frente a React Context API [34]:** Aunque *Context* es una herramienta nativa de React, presenta dos inconvenientes. En primer lugar, cualquier cambio en el valor del contexto provoca el re-renderizado de todos los componentes consumidores, lo que puede afectar negativamente al rendimiento. En segundo lugar, el uso de múltiples contextos suele derivar en un “anidamiento de proveedores” (*provider nesting*) que ensucia la estructura del proyecto. Zustand soluciona esto funcionando como un *hook* externo, permitiendo que los componentes se suscriban solo a fragmentos específicos del estado.

### React Router

React Router [35] es una librería de navegación para aplicaciones React que permite definir y gestionar las rutas de la interfaz de usuario. Su integración en el proyecto es fundamental, ya que toda la aplicación se estructura en torno a un sistema de rutas que determina qué contenido se muestra en función de la *URL*.

### TanStack Query

TanStack Query [36] (anteriormente conocida como *React Query*) es una librería de JavaScript especializada en la gestión del estado del servidor. Su función principal es facilitar la sincronización entre el *frontend* y el *Application Programming Interface (API)*, encargándose de la recuperación, el almacenamiento en caché y la actualización de los datos provenientes del *backend*.

Su integración en el proyecto se justifica por los siguientes beneficios en la arquitectura del sistema:

- **Gestión automática de la caché:** Sin esta herramienta, el almacenamiento local de las respuestas del servidor (como la lista de reuniones o los resúmenes generados) debería realizarse de forma manual mediante `useEffect` y estados globales. TanStack Query automatiza este proceso, invalidando y refrescando los datos solo cuando es estrictamente necesario, lo que optimiza el tráfico de red.

- **Simplificación del código:** Reduce drásticamente la cantidad de código repetitivo (*boilerplate*) al manejar de forma nativa los estados de carga, error y éxito de cada petición. Esto permite que los componentes se centren en la lógica de interfaz en lugar de en la gestión del ciclo de vida de las promesas.

### Camelcase-keys

Camelcase-keys [37] es una librería que convierte las claves de un objeto de `snake_case` a `camelCase` de manera recursiva. Se utiliza en la capa de comunicación con el *backend* para transformar las respuestas del *API*, debido a que el servidor sigue la convención `snake_case` (propia de Python), mientras que el *frontend* utiliza `camelCase` (propia de TypeScript). Esta transformación evita la necesidad de definir manualmente asignaciones entre ambos formatos en cada componente, manteniendo el código más limpio.

### React i18next

React i18next [38] es un potente *framework* de *internacionalización* (*i18n*) para React basado en el ecosistema *i18next*. Su integración en el proyecto permite gestionar de forma centralizada y escalable todos los textos de la interfaz, facilitando la adaptación de la plataforma a diferentes idiomas.

### Vite

Para la gestión y construcción del *frontend*, se ha empleado **Vite** [39], pues es la herramienta más extendida en el desarrollo web moderno debido a su velocidad superior frente a soluciones anteriores.

### Vitest

Para asegurar el correcto funcionamiento del *frontend*, se ha optado por **Vitest** [40] como entorno de ejecución de pruebas. Al estar la interfaz de usuario desarrollada sobre **vite**, el uso de **Vitest** permite compartir la misma configuración de compilación, lo que simplifica significativamente la infraestructura de testeo y garantiza que el código se pruebe en un entorno idéntico al de ejecución.

Se ha preferido esta herramienta frente a otras opciones como **Jest** [41] debido a su rendimiento superior y a su soporte nativo de TypeScript y módulos de JavaScript modernos. Gracias a su capacidad de ejecución reactiva, los tests se relanzan instantáneamente ante cualquier cambio en el código, facilitando un flujo de desarrollo ágil y eficiente.

### React Testing Library

React Testing Library [42] es una librería de utilidades para probar componentes de React simulando la interacción del usuario. En lugar de centrarse en detalles de implementación interna, fomenta la escritura de pruebas que reflejan cómo un usuario real utilizaría la aplicación, interactuando con los elementos a través de su representación visual en el DOM.

En este proyecto, se utiliza junto con Vitest como parte de los tests de integración del *frontend*. Mientras que Vitest actúa como un entorno de ejecución (*test runner*) y proporciona funciones de aserción, React Testing Library se encarga de renderizar los componentes y de proporcionar métodos para localizar elementos en el DOM y simular eventos de usuario (por ejemplo clicks o escritura de texto).

### MSW

*Mock Service Worker* (MSW) [43] es una librería de simulación de APIs para pruebas que intercepta las peticiones HTTP a nivel de red, permitiendo que los tests de integración del *frontend* se ejecuten sin enviar peticiones al *backend* real.

En este proyecto, MSW se utiliza en todos los tests de integración del *frontend*, definiendo *handlers* específicos para cada *endpoint* de la aplicación que permiten simular tanto respuestas exitosas como errores del servidor.

#### 3.3.2 Backend

Para la implementación del *backend*, se han utilizado las siguientes herramientas.

### Python

En la fase de diseño del *backend*, se evaluaron diversos lenguajes de programación de alto nivel, tales como Java, Go, Python y JavaScript. Dada la naturaleza del proyecto, centrada en el *Procesamiento del Lenguaje Natural* (PLN) y la integración de LLM, el abanico de posibilidades se redujo a Python [44] y JavaScript (Node.js) [45].

Esta acotación se debe a que la gran mayoría de las librerías y *frameworks* de vanguardia para la interacción con modelos de IA están orientados a estos dos ecosistemas. Si bien estas herramientas actúan, en esencia, como capas de abstracción (*wrapper*) sobre llamadas a APIs de terceros, su uso es fundamental para acelerar la implementación, garantizar la mantenibilidad del código y gestionar flujos de trabajo complejos de forma estructurada.

De haber escogido JavaScript, el proyecto sería más homogéneo, pues tanto el *backend* como el *frontend* se habrían desarrollado en el mismo lenguaje. Sin embargo, se ha seleccionado **Python** [44] debido a su predominancia en el ecosistema de IA y Ciencia de Datos. Al



ser el estándar *de facto* en la comunidad científica, ofrece un soporte de librerías más maduro y extenso en comparación a JavaScript.

### FastAPI

En el ecosistema de Python existen principalmente tres *frameworks* orientados al desarrollo de aplicaciones web: **Flask** [46], **Django** [47] y **FastAPI** [48]. Tras haber considerado estas opciones, se ha seleccionado **FastAPI** debido a su suave curva de aprendizaje, el respaldo de una gran comunidad activa (siendo a día de hoy el *framework* más utilizado de los tres), su excelente documentación oficial y su alto rendimiento en la gestión de peticiones.

### Swagger UI

**Swagger UI** [49] es una herramienta integrada de forma nativa en FastAPI que permite visualizar la documentación del *backend* a través de una interfaz web interactiva. Basada en el estándar *OpenAPI* [50], esta herramienta permite interactuar directamente con el servidor enviando peticiones a los *endpoints*, lo que agiliza significativamente el proceso de desarrollo y la realización de pruebas funcionales de la aplicación.

### LangChain

**LangChain** [51] es un *framework* de código abierto diseñado para facilitar la creación de aplicaciones basadas en modelos de lenguaje y agentes inteligentes. Esta herramienta permite integrar diversos proveedores de modelos, tales como **Ollama** [52], **Anthropic** [53] y **OpenAI** [12], entre otros.

Aunque el desarrollo podría haberse realizado utilizando directamente las *APIs* de bajo nivel de cada proveedor como la de OpenAI (compatible con un gran abanico de servicios) o la de Anthropic, LangChain proporciona una interfaz unificada para interactuar con todos ellos. Esta abstracción permite que la incorporación de nuevos modelos sea un proceso sencillo, generalmente implementado mediante el patrón de diseño de factoría. Además, LangChain facilita enormemente la implementación de arquitecturas complejas como **RAG** o flujos de trabajo basados en *Model Context Protocol* (MCP), evitando la necesidad de gestionar manualmente los *Software Development Kit* (SDK) de bajo nivel de cada fabricante.

### Ollama

Ollama [52] es un servicio de código abierto que permite ejecutar múltiples **LLMs** de manera local. Es el proveedor de **LLMs** que se utiliza por defecto en la aplicación.

Aunque existen alternativas que pueden llegar a ofrecer un mayor rendimiento, como *vLLM* [54], Ollama resulta más sencillo de configurar y se pueden ejecutar los **LLMs** directa-

mente en la [CPU](#), o en dispositivos *Apple Silicon* (no ocurre lo mismo con vLLM) lo que hace que Ollama sea más versátil a la hora de que el usuario ejecute el programa en local.

### MongoDB

**MongoDB** [55] es una base de datos *NoSQL* orientada a documentos que permite almacenar información de manera flexible en formato *Binary JSON (BSON)*. Debido a que el historial de mensajes de una conversación entre el usuario y el agente inteligente es, por naturaleza, un dato semiestructurado o no estructurado, surge la necesidad de emplear una base de datos de este tipo en lugar de una relacional.

Dentro de las diferentes opciones que existen en el mercado, se ha seleccionado MongoDB debido a su gran flexibilidad de esquema, su capacidad de escalabilidad horizontal y su alto rendimiento. Asimismo, destaca por contar con una comunidad muy activa, una documentación detallada y una curva de aprendizaje muy suave.

### ChromaDB

Para la implementación del [RAG](#), es necesario el uso de una base de datos vectorial que permita indexar y recuperar eficientemente los fragmentos de información de las reuniones. En este proyecto se ha seleccionado **ChromaDB** [56] debido a que es de código abierto, se puede implementar fácilmente en un contenedor de docker y tiene una buena integración con LangChain.

### WhisperX

**Whisperx** [57] es una herramienta de código abierto que ofrece tanto transcripción de audio (utilizando internamente *whisper* [58]) como [diarización](#) de hablantes (utilizando internamente *pyannote-audio* [59]).

Es el servicio de diarización que se selecciona por defecto debido a que se ejecuta de forma local en el [backend](#), eliminando la necesidad de una [API Key](#) externa. Se ha decidido emplear esta herramienta por su naturaleza de código abierto, su facilidad de integración y la calidad de sus resultados en entornos multilingües.

### AssemblyAI

**AssemblyAI** es una herramienta propietaria que ofrece diversas utilidades de [IA](#). Entre sus productos destacan los servicios de [Speech-to-Text](#) [60], ya disponen de modelos tanto de transcripción como de diarización de alta precisión.

Se ha optado por añadir esta herramienta como una alternativa a **WhisperX**, la cual el usuario puede activar proporcionando su propia [API Key](#). La decisión de incluir este servi-

cio externo se debe a que, al ejecutarse íntegramente en la nube, permite que usuarios con recursos computacionales limitados puedan utilizar la aplicación.

**AssemblyAI** ofrece un [API](#) sencilla de utilizar y proporciona —en el momento de desarrollo de este proyecto— un crédito gratuito de 50 \$, facilitando la realización de pruebas exhaustivas sin costes iniciales.

### Pytest

Para realizar los tests del [backend](#) de la aplicación, se ha utilizado la librería **pytest** [61].

Aunque existen otras librerías ampliamente utilizadas para realizar tests en Python principalmente **unittest** [62], **pytest** ofrece un [API](#) más fácil de utilizar, con una sintaxis inspirada no tanto en **JUnit** [63], sino en la propia filosofía del lenguaje Python.

### 3.3.3 Herramientas de desarrollo y seguimiento

A continuación se detallan diversas herramientas utilizadas para la planificación, desarrollo y seguimiento del proyecto.

#### Docker y Docker Compose

**Docker** [64] es una plataforma de código abierto basada en la tecnología de contenedores que permite empaquetar una aplicación y todas sus dependencias en un entorno virtualizado, ligero, portable y reproducible.

Para la gestión de la arquitectura multicontenedor de este sistema, se ha empleado **Docker Compose** [65]. Esta herramienta de orquestación permite definir y ejecutar aplicaciones compuestas por varios servicios mediante un único fichero de configuración denominado `docker-compose.yml`.

#### Ngix Proxy Manager

Para garantizar la seguridad de las comunicaciones mediante el protocolo [HTTPS](#), se ha empleado **Ngix Proxy Manager** [66]. Esta herramienta actúa como un [proxy](#) inverso que redirige el tráfico entrante de forma segura hacia el [frontend](#) de la aplicación.

Se ha seleccionado esta opción entre las diversas alternativas disponibles debido a su amplia adopción en la comunidad y a la simplicidad que ofrece su interfaz para la gestión de certificados y la configuración de reglas de acceso. Su uso permite centralizar la seguridad de la infraestructura sin añadir complejidad excesiva al proceso de despliegue.

### DuckDNS

**DuckDNS** [67] es un servicio de *Domain Name Service* (DNS) que permite asociar una dirección *Internet Protocol* (IP) a un nombre estático bajo el sufijo `.duckdns.org`.

Se utiliza en conjunto a **Nginx Proxy Manager** para que el usuario se pueda conectar de manera segura a una URL determinada

### Git

**Git** [68] es el sistema de control de versiones distribuido más extendido actualmente. Se ha utilizado durante todo el ciclo de vida del proyecto para gestionar el historial de cambios del código fuente.

### GitHub

**GitHub** [69] es una plataforma de alojamiento en la nube que permite centralizar y gestionar repositorios de *Git*. Se ha utilizado como servidor remoto para asegurar el almacenamiento del código fuente, permitiendo un acceso estructurado a las diferentes versiones del proyecto y garantizando la integridad de los archivos.

### GitHub Projects

**GitHub Projects** [70] es la herramienta de gestión integrada en la GitHub que permite realizar la planificación y el seguimiento de las tareas del proyecto. Facilita la organización del flujo de trabajo mediante tableros *kanban* altamente personalizables. Su principal ventaja competitiva es la sincronización nativa con las *issues* del repositorio, lo que asegura que el estado del desarrollo y el tablero de tareas estén actualizados en tiempo real.

# Metodología y Planificación

---

Para que un proyecto de ingeniería resulte exitoso, este debe de contar tanto con unas metas claras como con una planificación realista que permita cumplirlas. Además, el proceso de desarrollo debe seguir una serie de prácticas enfocadas en que la calidad sea algo transversal en todo el ciclo de vida, aplicando buenas prácticas desde el principio para garantizar que el producto final sea adecuado y cumpla las expectativas iniciales.

En este capítulo se detalla la metodología de trabajo empleada durante el proceso de ingeniería del software, así como la planificación temporal del proyecto.

## 4.1 Metodología empleada

Las metodologías ágiles surgen con la idea de que el proceso de desarrollo sea mucho más flexible que en las metodologías tradicionales (como el desarrollo en cascada), concibiendo el producto final como el resultado de un proceso iterativo y de mejora continua. Para este proyecto, se ha optado por utilizar una aproximación híbrida denominada **Scrumban** [71], que combina la estructura iterativa de **Scrum** [72] con la visualización del flujo de trabajo de **Kanban** [73].

### 4.1.1 Fundamentos: Scrum y Kanban

Dentro del marco de las metodologías ágiles, **Scrum** [72] es una de las más extendidas. Consiste en dividir el trabajo en ciclos cortos y regulares denominados *sprints*. Al final de cada *sprint*, el objetivo es obtener un incremento de *software* que sea funcional y evaluable.

Por otro lado, **Kanban** [73] es un marco de trabajo que se centra en la gestión visual de las tareas. Utiliza un tablero dividido en columnas que identifican el estado de las tareas. Su principio fundamental es visualizar el estado del proyecto de un vistazo y limitar el trabajo en curso para evitar cuellos de botella. Para este proyecto, se han utilizado las siguientes columnas:

- *Por hacer*: Tareas que están identificadas y que se pueden desarrollar de manera inmediata.
- *En desarrollo*: Tareas que están en proceso de desarrollo.
- *Bloqueadas*: Tareas las cuales no pueden ser realizadas porque requieren que otra tarea sea realizada anteriormente.
- *Hechas*: Tareas que ya han sido realizadas.

#### 4.1.2 Adaptación de Scrumban para un único desarrollador

Debido a que estas metodologías están diseñadas para equipos compuestos por varios miembros, Scrumban ha sido adaptada para un entorno de desarrollador único modificando sus prácticas tradicionales del siguiente modo:

- **Unificación de roles**: En Scrum existen tres roles definidos: el *Product Owner* (define prioridades), el *Scrum Master* (gestiona el proceso) y el Equipo de Desarrollo. En este proyecto, todos estos roles se unifican.
- **Eliminación de las *Daily Stand-ups***: Debido a que el equipo estaba formado por un único integrante, se han eliminado las reuniones diarias.
- **Revisiones simplificadas**: Las reuniones formales de revisión (*Sprint Review*) y retrospectiva se han sustituido por tutorías y puntos de control periódicos. En estos encuentros se evalúa el incremento desarrollado y se ajusta la planificación de las siguientes iteraciones según las necesidades detectadas.

#### 4.1.3 Gestión del código mediante Gitflow

Además de emplear Scrumban para gestionar el proyecto, se ha aplicado la metodología **Gitflow** [74] para gestionar el código a lo largo de todo el desarrollo, asegurando así un mayor orden y trazabilidad de los cambios.

Esta metodología consiste en dividir el desarrollo del código en diferentes ramas, en concreto se han empleado las siguientes:

- **main** (principal): Típicamente es la rama que contiene el código en estado de producción. En este caso, únicamente se integraba código en esta rama cuando se completaba un hito importante.
- **dev** (desarrollo): Actúa como una rama de integración continua en la que se van integrando los cambios desarrollados en las diferentes tareas del proyecto.

- **feature/** (nuevas funcionalidades): Cada nueva tarea que añade una funcionalidad se corresponde con una rama **feature/**, de modo que no se mezclan diferentes funcionalidades en el historial del proyecto.
- **bugfix/** (solución de errores): Los diversos bugs que se han identificado a lo largo del desarrollo del proyecto han sido resueltos cada uno de ellos en este tipo de ramas, aislando así la solución de estos errores del resto del código.
- **enhance/** (mejoras del código): Aquellas tareas cuyo objetivo es únicamente mejorar el código desarrollado en el resto de ramas y que no añaden ninguna funcionalidad se han desarrollado en este tipo de ramas. Ejemplos de estas tareas pueden ser refactorizar un módulo o mejorar la documentación.

## 4.2 Análisis de riesgos

Durante la planificación del proyecto se han identificado una serie de riesgos que podrían llegar a comprometer ciertos aspectos del proyecto, tanto desde el punto de vista técnico como de desarrollo.

En la Tabla 4.1 se pueden observar los riesgos identificados. Estos son clasificados en las siguientes categorías:

- **Bajo:** Riesgos para los que existe una solución sencilla.
- **Medio:** Riesgos con posible efecto en plazos y/o funcionalidad del producto.
- **Alto:** Riesgos que pueden llegar a implicar un alto desvío en el camino crítico del proyecto.

## 4.3 Planificación inicial

A continuación, se muestran los diferentes *sprints* planificados inicialmente. Cabe destacar que, alineado con la metodología ágil y el uso de *Gitflow* detallado en secciones anteriores, ninguna de estas iteraciones se da por concluida hasta cumplir con un criterio de calidad estricto. Antes de cerrar un ciclo e iniciar el siguiente, se ejecutan las pruebas correspondientes, se depuran los errores encontrados y se realiza la integración del código (*merge*) hacia las ramas estables (*main*).

- **Sprint 1 (40h).** En esta primera iteración se realiza un estudio de mercado y se identifican y redactan las historias de usuario, los casos de uso y los requisitos del sistema.

| Descripción del riesgo                                                                                                           | Nivel | Plan de mitigación                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dificultad para estimar duraciones de tareas con poca experiencia previa                                                         | Medio | Descomponer las tareas complejas en unidades de trabajo más pequeñas y abarcables, lo que incrementa significativamente la precisión a la hora de estimar los tiempos de desarrollo |
| Cambios en los requisitos funcionales a lo largo del desarrollo                                                                  | Medio | Realizar revisiones tras cada sprint para poder identificar estos posibles cambios lo antes posible                                                                                 |
| Modelos abiertos no funcionen correctamente con el protocolo MCP                                                                 | Alto  | Modificar la interacción de los modelos con los datos para utilizar un sistema RAG en lugar del protocolo MCP                                                                       |
| Filtraciones de datos sensibles, como contraseñas de usuarios o API keys                                                         | Bajo  | Cifrar estos datos a la hora de almacenarlos, de modo que una filtración de la base de datos no permita a un atacante obtener esta información                                      |
| Solapamiento del desarrollo con el inicio del curso académico (estudios de máster) debido a retrasos acumulados en el cronograma | Alto  | Planificar una congelación estructurada y ordenada del proyecto en septiembre, dejando el progreso plenamente documentado para agilizar la retoma del desarrollo posteriormente.    |

Tabla 4.1: Análisis de riesgos del proyecto y planes de mitigación.

Asimismo, se diseñan los *mockups* de la interfaz de usuario y se definen las herramientas que compondrán el *stack* tecnológico. Por último, dada la relevancia del carácter de código abierto del proyecto, se evalúan las diferentes licencias de *software* libre para seleccionar la más adecuada para la publicación del repositorio.

- **Sprint 2 (40 horas).** En este *sprint* se diseña la arquitectura del *backend* del sistema, identificando las diferentes capas y dominios. También se desarrolla el procesamiento de las reuniones.
- **Sprint 3 (40 horas).** En este *sprint* se implementa el asistente conversacional utilizando el protocolo MCP y la autenticación de usuarios.
- **Sprint 4 (40 horas).** En este *sprint* se implementan los ajustes de usuario y se comienza a desarrollar la interfaz, añadiendo la autenticación, la navegación y la funcionalidad



de subir reuniones.

- **Sprint 5 (40 horas).** En este *sprint* se implementan el resto de funcionalidades de la interfaz:
  - Vista del chat.
  - Vista de la lista de reuniones.
  - Vista de la transcripción y del resumen de una reunión.
  - Vista de ajustes de usuario.
- **Sprint 6 (40 horas).** Pulir la interfaz, implementando un flujo *SSE* para la subida de reuniones, *WS* para la interacción con el chat, internacionalización y cambio entre tema claro y oscuro. También se implementa el *proxy* para que las conexiones del usuario con el *frontend* se realicen de manera segura.
- **Sprint 7 (80 horas).** Esta última iteración se reserva para documentación y redacción de la memoria.

En la Figura 4.1 se puede observar el diagrama de Gantt correspondiente a esta planificación inicial. Se observa que la duración estimada es de unos 3 meses, asumiendo que el alumno dedique unas 4 horas al día 5 días a la semana al proyecto.

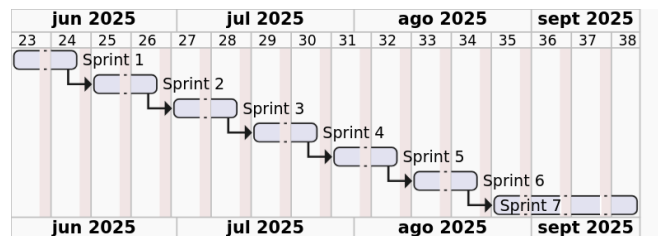


Figura 4.1: Diagrama de Gantt de la planificación inicial

## 4.4 Estimación de costes

En esta sección se realiza una estimación de los costes de este proyecto, teniendo en cuenta los costes debidos a recursos humanos y a recursos técnicos.

### 4.4.1 Recursos Humanos

En cuanto a los recursos humanos, este proyecto se desarrolla por dos integrantes. Los salarios se han calculado a partir de las tablas salariales del XIX Convenio colectivo estatal

de empresas de consultoría y tecnologías de la información [75]. En la Tabla 4.2 se pueden observar sus puestos, dedicación, coste por hora y coste total.

| Puesto                    | Dedicación | Coste por hora | Coste total |
|---------------------------|------------|----------------|-------------|
| Analista y programador    | 320 horas  | 13,39 €        | 4.284,8 €   |
| Jefe de proyecto software | 20 horas   | 16,16 €        | 323,20 €    |

Tabla 4.2: Coste de los recursos humanos

#### 4.4.2 Recursos técnicos

Sobre los recursos técnicos, para realizar este proyecto únicamente se cuenta con un ordenador Asus ROG Zephyrus G14 con las siguientes características técnicas:

- CPU AMD Ryzen 7 4800HS.
- GPU Nvidia GTX 1650ti con 4GB de VRAM.
- 24GB de RAM.
- SO Ubuntu 25.10.

La vida útil de un equipo de estas características se estima en unas 12000 horas, y su coste es de 1000€. Por lo tanto, teniendo en cuenta que va a ser utilizado durante las 320 horas de trabajo del alumno, se calcula un coste del equipo del siguiente modo:

$$\frac{1000 \text{ €}}{12000 \text{ horas}} \times 320 \text{ horas} = 26.67 \text{ €}$$

#### 4.4.3 Cómputo total

Por lo tanto, sumando tanto los costes del personal como los costes del equipo empleado, se obtiene que el coste estimado del proyecto es de 4.634,67 €.

### 4.5 Seguimiento y Desviaciones

A lo largo del desarrollo del proyecto, el cronograma y la estimación temporal planteados inicialmente sufrieron diversas alteraciones debido a la materialización de riesgos técnicos y organizativos. A continuación, se detallan las desviaciones respecto a la planificación original:

- **Bloqueo con el protocolo MCP:** Durante el *sprint* 3, a la hora de proporcionar el contenido de las reuniones como contexto a los LLMs, se trató de utilizar inicialmente el protocolo MCP. Sin embargo, a pesar de que los LLMs privativos, como los de OpenAI y los de Anthropic daban buenos resultados, los modelos de pesos abiertos como los de Qwen o Deepseek no eran capaces de llamar a las herramientas expuestas por este protocolo. Ante esta situación, y debido a que el soporte de modelos de pesos abiertos es vital para este proyecto, el desarrollo de esta iteración se alargó una semana (20 horas), avanzando hacia una arquitectura RAG.
- **Curva de aprendizaje en tecnologías frontend:** Debido a la falta de experiencia previa en el desarrollo frontend, el tiempo de implementación de estas tareas fue superior al estimado. En concreto, el *sprint* 4 sufrió un retraso de una semana adicional (20 horas).
- **Pausa por estudios de Máster:** Como consecuencia de los retrasos anteriores, el desarrollo se solapó con el inicio del curso académico en septiembre. Ante la falta de disponibilidad temporal, se tomó la decisión de, una vez terminado el *sprint* 4, congelar el desarrollo hasta el mes de febrero, debido a que la carga académica del máster es menor durante el segundo cuatrimestre, lo que posibilita dedicar tiempo a este proyecto.
- **Coste de reanudación y adaptación:** Tras retomar el proyecto en el mes de febrero, el ritmo de desarrollo fue reducido durante las primeras semanas. Esto produjo un retraso de 1 semana (20 horas) debido al periodo de adaptación necesario para familiarizarse nuevamente con el estado del código, la arquitectura y las decisiones de diseño tomadas meses atrás.
- **Desarrollo intermitente y segunda pausa por estudios de Máster:** Como consecuencia de desarrollar parte del proyecto en simultaneidad al máster, tanto el *sprint* 6 como el *sprint* 7 fueron desarrollados dedicándoles menos de 20 horas a la semana, por lo que, aunque el número de horas trabajadas no superó lo previsto inicialmente, sí tuvo como consecuencia retrasos en los plazos. Además, el desarrollo fue pausado otra vez desde principios de abril hasta mediados de mayo, debido tanto al incremento de la carga académica como al desarrollo de prácticas en empresa.

Para reflejar el impacto de estos acontecimientos, la Figura 4.2 muestra un diagrama de Gantt actualizado en el que se puede visualizar cómo se llevó a cabo realmente el desarrollo.

#### 4.5.1 Actualización de los costes

Todos estos retrasos implicaron que el número de horas de trabajo aumentó en 60. Como se detalla en la Sección 4.4.1, el coste por hora del desarrollador es de 13,39 €, lo que supone un incremento de 803,40 € con respecto al coste inicial.

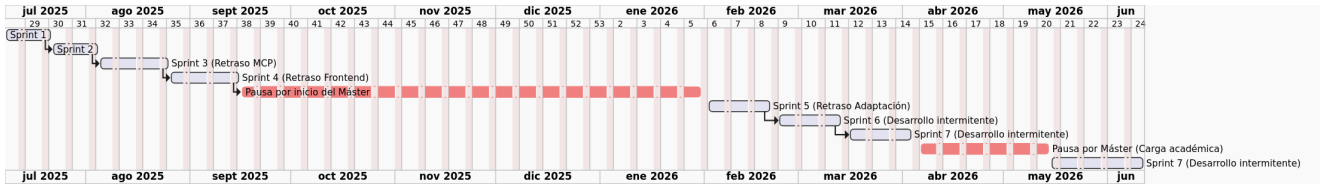


Figura 4.2: Diagrama de Gantt del seguimiento del proyecto

Además, se debe de computar también el incremento del coste asociado al equipo informático expuesto en la Sección 4.4.2, lo que supone un incremento de 5 €:

$$\frac{1000 \text{ €}}{12000 \text{ horas}} \times 60 \text{ horas} = 5 \text{ €}$$

En la Tabla 4.3 se muestra la comparativa entre los costes previstos en la Sección 4.4.3 y el costo real tras los retrasos expuestos en esta sección.

| Recurso                   | Coste estimado    | Coste real        | Incremento      |
|---------------------------|-------------------|-------------------|-----------------|
| Analista y programador    | 4.284,8 €         | 5.088,2 €         | 803,40 €        |
| Jefe de proyecto software | 323,20 €          | 323,20 €          | 0 €             |
| Equipo informático        | 26,67 €           | 31,67 €           | 5 €             |
| <b>TOTAL:</b>             | <b>4.634,67 €</b> | <b>5.443,07 €</b> | <b>808,40 €</b> |

Tabla 4.3: Comparativa de costes estimados iniciales frente a costes finales reales tras las desviaciones.

## Capítulo 5

# Desarrollo

---

En este capítulo se detalla el desarrollo del proyecto a lo largo de cada uno de los *sprints* mencionados en la planificación inicial del proyecto (Sección 4.3).

### 5.1 Sprint 1

Este primer *sprint* del proyecto se centró fundamentalmente en establecer las bases del proyecto, previo a pasar a la codificación. En concreto, se realizó el estudio de mercado, se redactó la especificación del proyecto, se realizó el diseño de la interfaz, se seleccionó la licencia a utilizar y el nombre comercial del proyecto.

#### 5.1.1 Estudio de mercado

El primer paso consistió en analizar el estado de la cuestión y evaluar las soluciones existentes en el mercado, lo que permitió comprender las fortalezas y debilidades de las mismas, identificando así cuál debía de ser el factor diferenciador de este proyecto con respecto al resto, su enfoque de código abierto, tal y como se detalla en el Capítulo 2.

#### 5.1.2 Especificación del proyecto

Una vez establecida la meta general de este proyecto (crear una plataforma de código abierto que permita a los usuarios recuperar información sobre reuniones mediante lenguaje natural) y con el fin de guiar el desarrollo de manera estructurada, se llevó a cabo la especificación del proyecto.

En primer lugar, se definieron las épicas del proyecto (Anexo A), en base a las cuales se redactaron los requisitos del sistema (Anexo B), dividiéndolos en *Requisito Funcional* (RF) y *Requisito No Funcional* (RNF). Con esta base, se modeló el comportamiento del sistema mediante casos de uso. En la Figura 5.1 se observa el diagrama de casos de uso, mientras que el detalle de cada uno de estos puede ser consultado en el Anexo C.

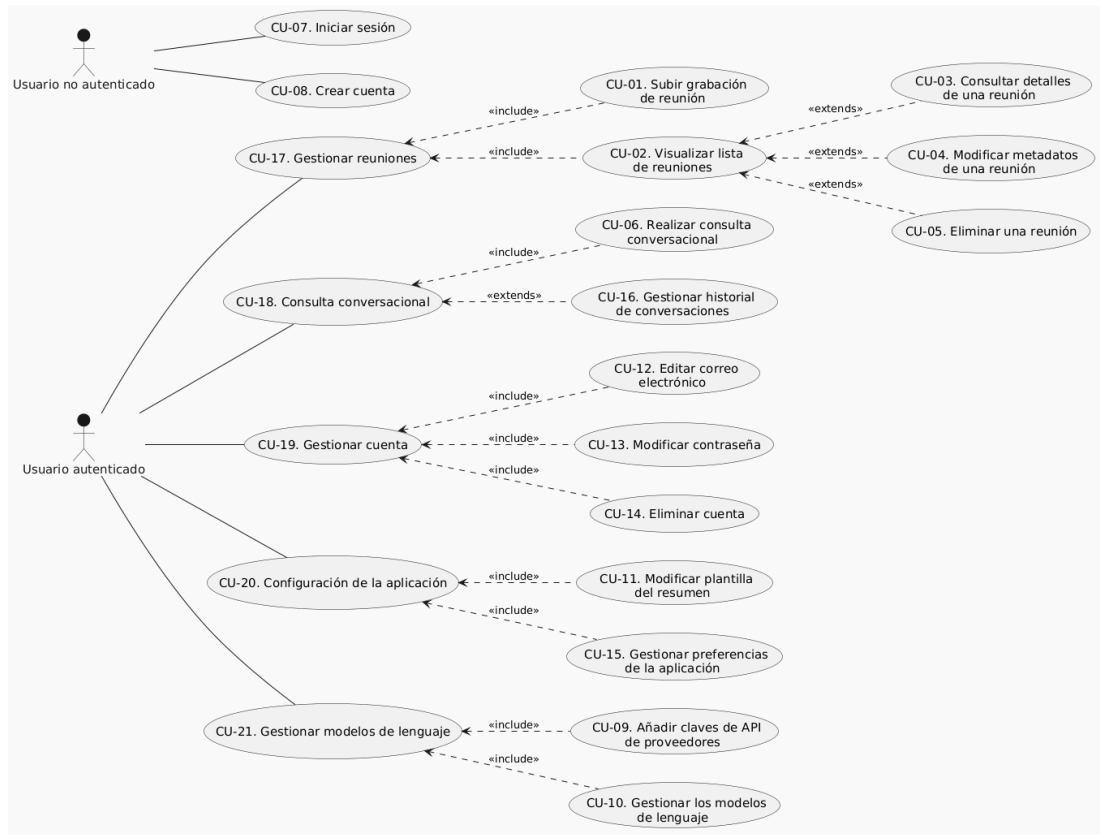


Figura 5.1: Diagrama de Casos de Uso

### 5.1.3 Diseño de la interfaz

Una vez redactados los documentos mencionados en la sección anterior, se procedió a realizar una fase de diseño de la interfaz de la aplicación.

Se priorizó diseñar la interfaz de modo que esta sea clara, minimalista y fácil de utilizar para personas no técnicas (*RNF-04*), apostando por una disposición similar a la de otras aplicaciones web en las que se interactúa con modelos de lenguaje, para resultar lo más familiar posible al usuario.

En el Anexo D se pueden visualizar los *mockups* desarrollados, los cuales se tomarán como referencia en iteraciones posteriores para implementar la interfaz.

### 5.1.4 Elección de herramientas

Teniendo ya definida la especificación del proyecto y realizado el diseño de la interfaz, se definieron tanto la arquitectura general del proyecto (Sección 3.2) como las herramientas a utilizar (Sección 3.3), haciendo especial hincapié en el uso de tecnologías de código abierto (*RNF-03*).

### 5.1.5 Elección de la Licencia Open Source

Dada la naturaleza del proyecto y su fuerte compromiso con la privacidad y el *software* de código abierto (*RNF-03* y *RNF-10*), se evaluaron diferentes licencias para la publicación del repositorio.

Se descartaron licencias permisivas como la MIT [76], ya que permiten la integración del código en *software* privativo cerrado.

La licencia seleccionada es la **GNU Affero General Public License v3 (AGPLv3)** [77]. Esta licencia de tipo *copyleft* garantiza que cualquier *software* derivado de este proyecto deba distribuirse bajo los mismos términos libres, de manera que el código queda protegido de apropiaciones privativas e incentiva el crecimiento colaborativo de la comunidad de código abierto.

La licencia AGPLv3 es una extensión sobre la licencia **GNU General Public License v3 (GPLv3)** [78], añadiendo una cláusula según la cual, cualquier entidad que desee hostear el servicio deberá de hacerlo publicando su código modificado. Por lo tanto, la elección de esta licencia protege al *software* de ser apropiado por entidades externas, garantizando que cualquier mejora retorne a la comunidad. Asimismo, al conservar la titularidad sobre los derechos de autor, otorga la libertad de comercializar este sistema en el futuro como un *SaaS* propio.

### 5.1.6 Asignación de nombre comercial

Como última tarea en este *sprint*, y con el objetivo de dotar a la herramienta de un nombre reconocible que pueda ser utilizado posteriormente como producto comercial, se ha seleccionado *Memento* como nombre de la aplicación. Este término proviene del latín, y se traduce al español como “*recuerda*” o “*acuérdate*”.

## 5.2 Sprint 2

Este segundo *sprint* tiene principalmente dos objetivos. En primer lugar, se busca diseñar la arquitectura del *backend* de la aplicación, identificando las diferentes capas y dominios. En segundo lugar, se pretende desarrollar el código necesario para realizar el procesamiento de las reuniones (*CU-01. Subir grabación de reunión*).

### 5.2.1 Diseño de la arquitectura del *backend*

Tal y como se detalló en la Sección 3.2, el *backend* de la aplicación consta de un *API REST*. Una de las características de este tipo de *API* es la definición de diferentes capas jerárquicas, lo que permite que el sistema sea más escalable, flexible, seguro y mantenible [79].

El número de capas a utilizar depende tanto de la propia naturaleza del proyecto como de las decisiones de diseño llevadas a cabo por los desarrolladores. En este caso, se han considerado las siguientes tres capas:

- **Routers:** Es la capa que expone los *endpoints* con los que se comunica el cliente.
- **Services:** Es la capa en la que reside la lógica de negocio de la aplicación.
- **Repositories:** Es la capa que abstrae las consultas a la base de datos.

En la Figura 5.2 se aprecia cómo se comunican estas capas.

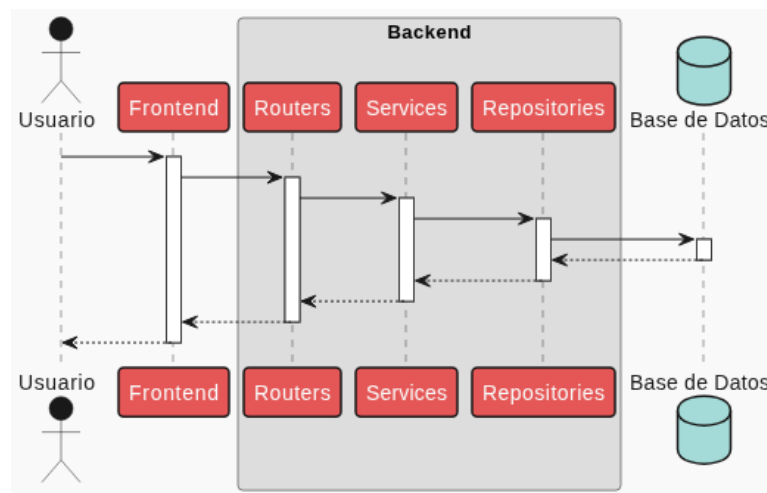


Figura 5.2: Diagrama de secuencia a alto nivel de la aplicación

Además de estas capas, se define también una serie de dominios separados, cada uno correspondiente a una serie de casos de uso (detallados en el Anexo C):

- **auth:** Este dominio se corresponde con los casos de uso *CU-07. Iniciar sesión*, *CU-08. Crear cuenta* y *CU-19. Gestionar cuenta*.
- **conversation:** Este dominio se corresponde con el caso de uso *CU-18. Consulta conversacional*.
- **meeting:** Este dominio se corresponde con el caso de uso *CU-17. Gestionar reuniones*.
- **settings:** Este dominio se corresponde con los casos de uso relacionados con la configuración, es decir, *CU-20. Configuración de la aplicación* y *CU-21. Gestionar modelos de lenguaje*.

En la Figura 5.3 se muestra el diagrama de componentes a alto nivel del *backend*, en el que se aprecian claramente estos cuatro dominios.



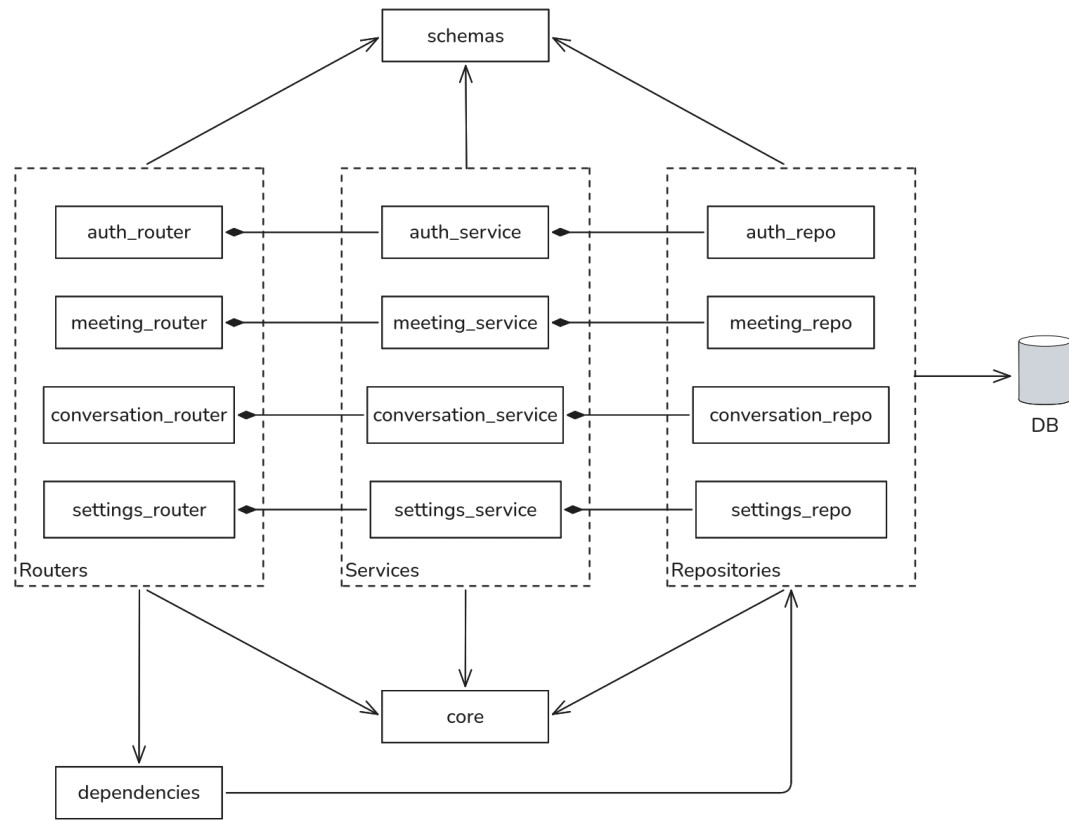


Figura 5.3: Diagrama de componentes del *backend* a alto nivel, estructurado por capas lógicas y dominios

### Desacoplamiento de la base de datos

Con el propósito de garantizar la extensibilidad del sistema y facilitar una eventual migración de la base de datos, la capa de persistencia se ha diseñado siguiendo el **Patrón Repositorio** (*repository*) [80]. Este patrón arquitectónico actúa como una interfaz mediadora entre la lógica de negocio y el acceso a los datos. Para ello, se define una clase abstracta que define las operaciones permitidas sobre los datos; esta clase es posteriormente heredada por clases concretas especializadas en un motor específico, como MongoDB o PostgreSQL.

Además, estas clases concretas se instancian desde un único punto centralizado y se inyectan al resto del código mediante inyección de dependencias (esto se realiza desde el componente `dependencies` que se puede observar en el diagrama de la Figura 5.3). Gracias a esta decisión, sustituir o añadir bases de datos en el futuro no tendrá ningún impacto en la lógica del negocio del sistema.

### 5.2.2 Implementación

Una vez definidas las capas y los dominios del *backend*, se procedió a implementar el procesamiento automatizado de las reuniones, además de otras funcionalidades necesarias para gestionar las mismas, las operaciones *Create Retrieve Update Delete* (CRUD).

#### Definición del API y los *endpoints*

En primer lugar se definen los *endpoints* a exponer a través de los *routers*. En la Figura 5.4 se pueden observar los endpoints implementados para el dominio de *meetings*, documentados mediante Swagger UI.

| Meetings |                              |                                         |
|----------|------------------------------|-----------------------------------------|
| GET      | /meetings                    | Retrieve all meetings                   |
| POST     | /meetings                    | Create meetings and process them        |
| GET      | /meetings/summary/{id}       | Retrieve the summary of a meeting       |
| GET      | /meetings/transcription/{id} | Retrieve the transcription of a meeting |
| PATCH    | /meetings/{id}               | Update a meeting                        |
| DELETE   | /meetings/{id}               | Delete a meeting                        |

Figura 5.4: Captura de Swagger UI con los *endpoints* del dominio *meeting*

El *endpoint* más crítico desarrollado en este sprint es POST `/meetings`, el cual se encarga de recibir los metadatos de la reunión (como el título y la fecha) junto al archivo de audio y llevar a cabo el procesamiento (transcripción, resumen y persistir resultados). El resto de *endpoints* implementados proporcionan operaciones *CRUD* necesarias para la gestión de los datos almacenados.

#### Arquitectura de la transcripción

Uno de los retos en esta iteración consistió en preparar el sistema para que este soporte múltiples motores de transcripción de manera limpia y escalable, respetando el principio Abierto Cerrado (*Open-Closed Principle*). Aunque tal y como se ha comentado en la planificación inicial (Sección 4.3), la selección de proveedores no se implementará hasta el *sprint* 4, al

añadir los ajustes de usuario (Sección 5.4.1), el *backend* debe ser diseñado desde el principio para soportar esta funcionalidad.

Para resolver esto, se aplicó el Patrón Estrategia (*Strategy*) [81], que consiste en definir una clase abstracta de la cual heredan las implementaciones concretas, en este caso, se define una clase abstracta `TranscriptionService`, de la cual heredan las clases concretas `AssemblyaiTranscriptionService` y `WhisperXTranscriptionService`.

En la Figura 5.5 se puede observar el diagrama de componentes, resaltando aquellos implementados en este *sprint*. En el módulo `services/meetings/transcription` se puede apreciar cómo se implementa el patrón estrategia para realizar la transcripción.

Para orquestar la creación de instancias de estos servicios concretos sin acoplar la lógica de negocio a una implementación específica, se aplicó el Patrón Factoría (*Factory*) [81]. De este modo, la capa de servicios delega la creación del transcriptor adecuado utilizando la factoría, encapsulando la lógica de decisión y reduciendo el acoplamiento del sistema.

En cuanto a la generación de resúmenes, este proceso se realiza interactuando con un *LLM*, facilitándole tanto la transcripción generada como una plantilla. Puesto que el usuario podrá en un futuro seleccionar diferentes modelos de lenguaje (los cuales pueden venir de diferentes proveedores), la interacción con estos modelos se realiza a través de la clase `BaseChatModel`, de la librería `LangChain`, la cual sirve como base para todos los *LLM* con los que se interactúa a través de esta librería; además, para la instanciación de los modelos se vuelve a hacer uso de una factoría (componente `llm_factory`). Para este *sprint*, el proveedor utilizado es Ollama (Sección 3.3.2), servicio el cual se añadió a la aplicación utilizando `Docker Compose`.

### Flujo de ejecución del procesamiento

El flujo de ejecución de todo este proceso se detalla en la Figura 5.6.

### Pruebas

Una vez implementados los *endpoints* mostrados en la Figura 5.4 se realizaron diferentes pruebas para comprobar el correcto funcionamiento de los mismos.

En primer lugar, se llevaron a cabo diferentes pruebas de manera manual a través de `Swagger UI`, comprobando el correcto funcionamiento del sistema.

Posteriormente, se definieron tests automatizados haciendo uso de `pytest`. En concreto, se implementaron tanto tests unitarios y de integración. Los tests unitarios permiten comprobar correctamente el funcionamiento de cada módulo de manera aislada (por ejemplo comprobar que `WhisperX` se ejecuta correctamente en la *GPU* siempre y cuando haya memoria disponible en la misma, y en caso contrario, se dispone a utilizar la *CPU*); los tests de integración permiten comprobar que todo el flujo de un *endpoint* en concreto funciona tal y como debería.

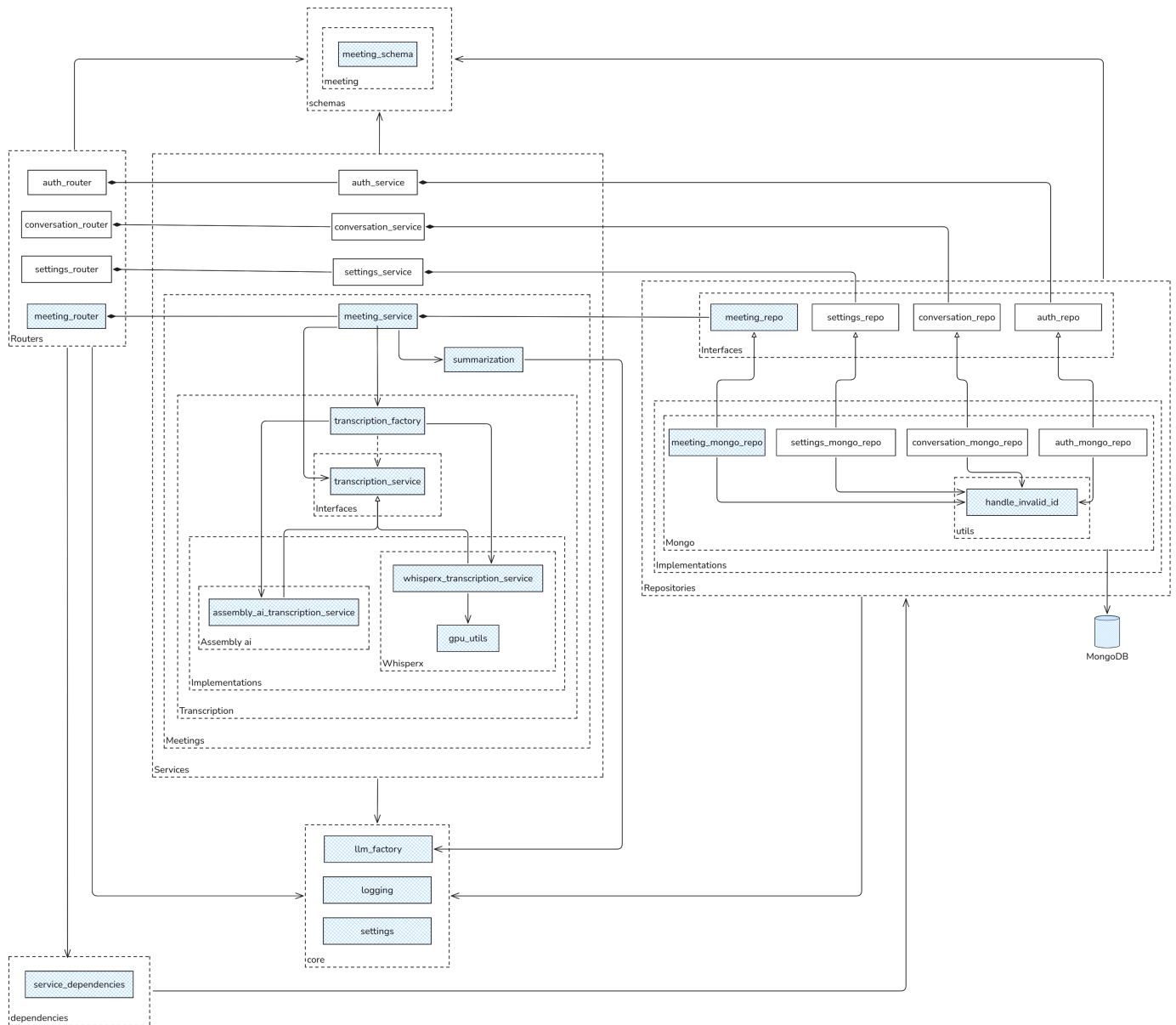


Figura 5.5: Diagrama de componentes del *backend*. En azul se pueden apreciar los componentes implementados en este *sprint*

La implementación de pruebas automatizadas resulta fundamental para el ciclo de vida del proyecto. A medida que el proyecto escale en las siguientes iteraciones, contar con esta *suite* permite detectar errores de forma sencilla, garantizando la estabilidad del código y facilitando su mantenibilidad.

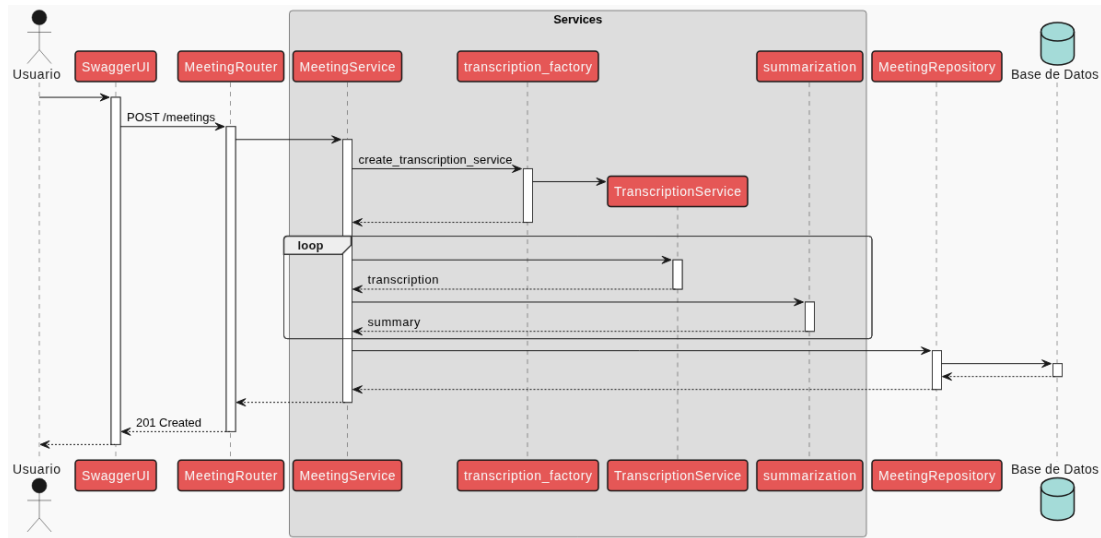


Figura 5.6: Diagrama de secuencia del procesamiento de las reuniones ejecutado al recibir una petición `HTTP POST /meetings`

### 5.3 Sprint 3

Los objetivos de este tercer *sprint* se centran en implementar dos partes del sistema. Por un lado, se implementa la lógica de autenticación en el *backend*, cubriendo tanto el *CU-07. Iniciar sesión* como el *CU-08. Crear cuenta*. Por otro lado, se implementa el asistente conversacional en el *backend*, implementando el *CU-18. Consulta conversacional*.

#### 5.3.1 Implementación del asistente conversacional

Siguiendo con los dominios presentados en la Sección 5.2.1, en esta tarea se busca implementar el dominio *conversation*, implementando así el *CU-18. Consulta conversacional*.

#### Definición del API y los endpoints

En la Figura 5.7 se muestran los endpoints implementados, correspondientes todos al dominio de *conversations*.

Destacar que los métodos POST serán sustituidos en el *sprint* 6 (Sección 5.6) por una comunicación cliente-servidor a través de *WebSockets (WS)*, tal y como se expuso en la Sección 4.3.

Al igual que ocurría en el *sprint* anterior, se pondrá el foco únicamente en comentar la implementación de ciertos endpoints (en este caso los POST, puesto que son los que manejan la interacción con los LLMs), debido a que el resto de operaciones únicamente proporcionan operaciones *CRUD* necesarias para la gestión de los datos.

| Conversations Manage conversations |                          |                                                        |
|------------------------------------|--------------------------|--------------------------------------------------------|
| GET                                | /conversations           | Retrieve all conversations metadata                    |
| POST                               | /conversations           | Create a new conversation                              |
| POST                               | /conversations/{id}/chat | Send a message to the chatbot and get a response       |
| GET                                | /conversations/{id}      | Retrieve a dialogue between the user and the assistant |
| PUT                                | /conversations/{id}      | Update the metadata of a conversation                  |
| DELETE                             | /conversations/{id}      | Delete a conversation                                  |

Figura 5.7: Captura de Swagger UI con los *endpoints* del dominio conversation

### Implementación utilizando MCP

Para la implementación del asistente inteligente, el diseño inicial contemplaba el uso del *Model Context Protocol* (MCP). La premisa era exponer diferentes herramientas que permitiesen al LLM acceder a la información de tanto las transcripciones de las reuniones como de los resúmenes estructurados.

Sin embargo, durante esta fase de desarrollo se materializó uno de los riesgos identificados en la planificación del proyecto (Sección 4.2, Tabla 4.1): la limitación de los modelos de pesos abiertos para operar correctamente con el protocolo MCP.

Mientras que los modelos privativos de OpenAI y Anthropic gestionaban correctamente las llamadas a las funciones necesarias para recuperar reuniones, los modelos de pesos abiertos ejecutados a través de Ollama (como Qwen o Deepseek) presentaron deficiencias críticas. Estos modelos, en muchas ocasiones, no fueron capaces de identificar correctamente cuándo debían utilizar estas herramientas, y en caso de sí hacerlo correctamente, las llamadas que efectuaban no eran válidas (por ejemplo introducían parámetros incorrectos a las funciones, o en la cadena de pensamiento de los modelos se identificaba que había que utilizar una herramienta determinada, pero luego no la llamaban).

Ante esta situación, y debido a la importancia de poder ejecutar los modelos de pesos abiertos en local expuesta a lo largo del Capítulo 2, se llevó a cabo el plan de mitigación previsto (Sección 4.2, Tabla 4.1): abandonar la aproximación MCP e implementar una arquitectura RAG.

### Implementación de la Arquitectura RAG

Al utilizar una arquitectura *Retrieval Augmented Generation* (RAG) en el flujo de la interacción con el asistente, se delega la responsabilidad de la búsqueda a la propia lógica de negocio, en lugar de tener que depender de manera tan directa en las capacidades de los propios modelos. Esta implementación se realizó utilizando LangChain, siguiendo el siguiente flujo:

- **Indexación:** Tras la generación de los resúmenes en el dominio de reuniones, el texto de las transcripciones y los resúmenes se fragmenta (*chunking*) y se transforma en representaciones vectoriales (*embeddings*). Estos vectores se almacenan en una base vectorial (concretamente ChromaDB), permitiendo realizar búsquedas basadas en similitud semántica.
- **Recuperación:** Cuando el usuario envía una consulta al asistente, un LLM procesa esta consulta generando un texto que posteriormente se utilizará para realizar una búsqueda semántica sobre la base de datos vectorial
- **Generación:** Se le envía a un LLM la consulta original del usuario junto al resultado obtenido tras el paso anterior.

En la Figura 5.8 se muestra el diagrama de componentes del *backend*, resaltando aquellos implementados en esta sección. Como se puede apreciar, el componente rag accede a la base de datos vectorial (ChromaDB). Esto lo realiza a través de la librería LangChain, utilizando la clase abstracta VectorStore, lo que permite acceder a la base de datos sin conocer el detalle de la misma; esto implica que en un futuro se podría cambiar esta sin requerir la modificación del funcionamiento del módulo.

### Pruebas

Al igual que en el *sprint* anterior, se llevaron a cabo pruebas manuales con Swagger UI y se definieron tests tanto unitarios como de integración para este módulo.

#### 5.3.2 Implementación de la autenticación de usuarios

El segundo objetivo de este *sprint* consiste en implementar la autenticación de usuarios.

### Definición del API y los *endpoints*

En la Figura 5.9 se muestran los *endpoints* implementados en esta sección.

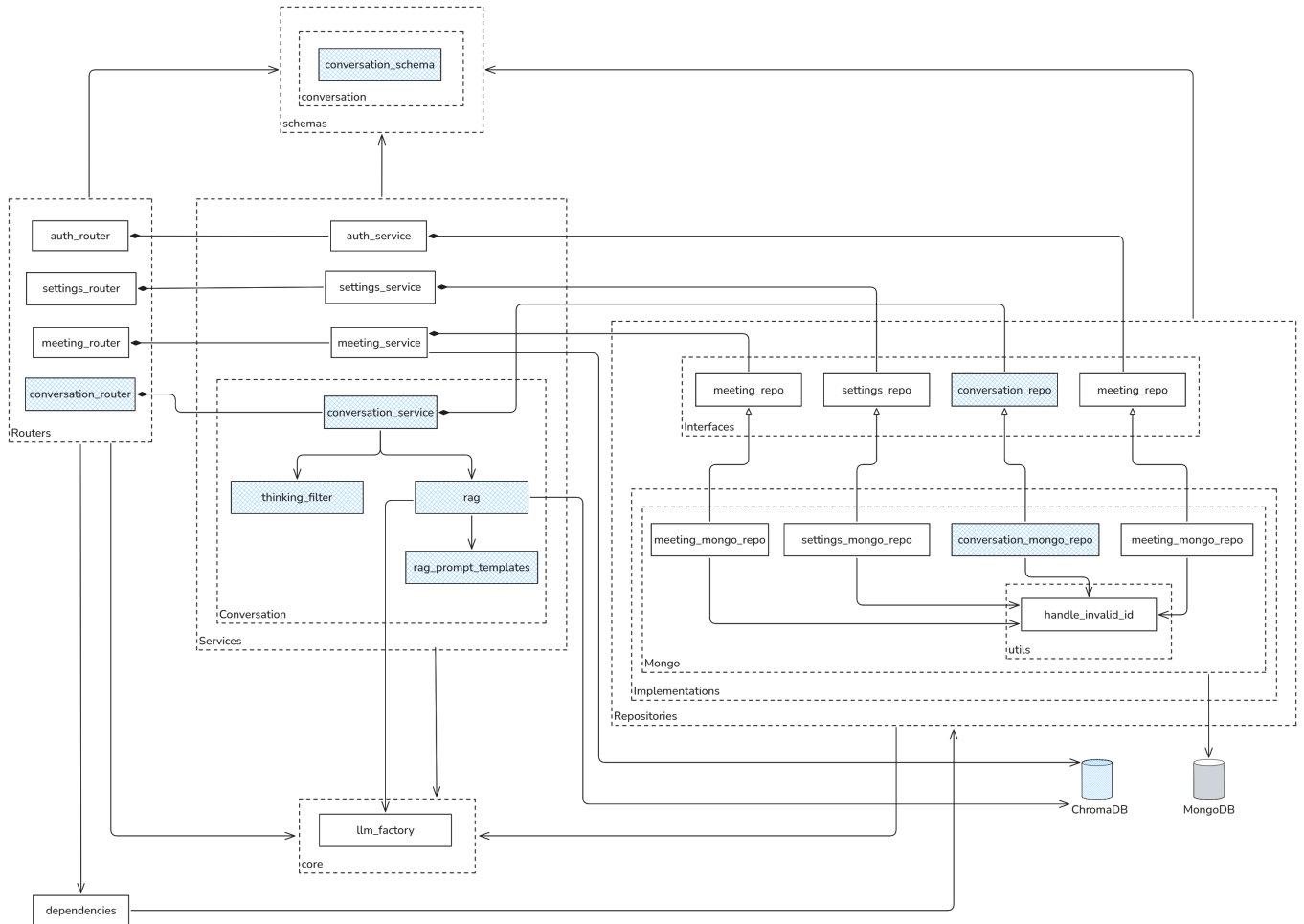


Figura 5.8: Diagrama de componentes de *backend*. En color azul se resaltan los módulos implementados en esta sección

## Implementación

En la Figura 5.10 se observa el diagrama de componentes del sistema, destacando aquellos implementados en esta sección.

La autenticación de usuarios se implementó siguiendo el estándar **OAuth 2.0** [82], utilizando contraseñas de usuario para obtener un *JSON Web Token (JWT)*, el cual será requerido por los *endpoints* que requieren que el usuario esté autenticado.

De este modo se implementa un mecanismo de autenticación sin estado, puesto que el *backend* no requiere almacenar sesiones activas en memoria ni realizar consultas a bases de datos para validar la identidad de un usuario en cada petición, optimizando así el uso de recursos técnicos del sistema.

A continuación se detallan las fases de este proceso:



## Authentication User authentication, registration and account management

|        |                       |                        |
|--------|-----------------------|------------------------|
| POST   | /auth/register        | Register               |
| POST   | /auth/token           | Login For Access Token |
| GET    | /auth/me              | Get Current User Info  |
| GET    | /auth/deletion-policy | Get Deletion Policy    |
| PATCH  | /auth/username        | Change Username        |
| PATCH  | /auth/password        | Change Password        |
| DELETE | /auth                 | Delete Account         |

Figura 5.9: Captura de Swagger UI con los *endpoints* del dominio auth

1. **Inicio de sesión:** Cuando el usuario introduce sus credenciales (correo electrónico y contraseña) al realizar un inicio de sesión, el *frontend* envía una petición POST `/auth/login` al *backend*, el cual la procesa realizando las siguientes comprobaciones:
  - (a) Busca al usuario en la base de datos a través de su correo electrónico.
  - (b) Verifica la integridad de la contraseña aplicando un algoritmo de *hashing* seguro.
  - (c) Si la validación es correcta, el sistema genera un *JWT* firmado con una clave secreta simétrica guardada en el servidor. Este *token* incluye en su *payload* el identificador del usuario (correo electrónico en este caso) en el campo `sub` y una marca de tiempo que define su periodo de expiración, `exp`, lo que limita la ventada de vulnerabilidad en caso de intercepción.
2. **Transmisión del *token* y acceso protegido:** Una vez devuelto el *JWT* al *frontend*, este lo almacena de manera segura. A partir de ese momento, cualquier petición dirigida a un *endpoint* protegido debe incorporar obligatoriamente el *JSON Web Token (JWT)* en la cabecera de la petición mediante un formato estandarizado: `Authorization: Bearer <token>`.

Para proteger los *endpoints* que requieren privilegios de acceso, se ha hecho uso del sistema de inyección de dependencias de FastAPI. Al recibir una petición en un *endpoint* protegido, el *framework* intercepta el flujo y ejecuta un módulo de seguridad que realiza lo siguiente:

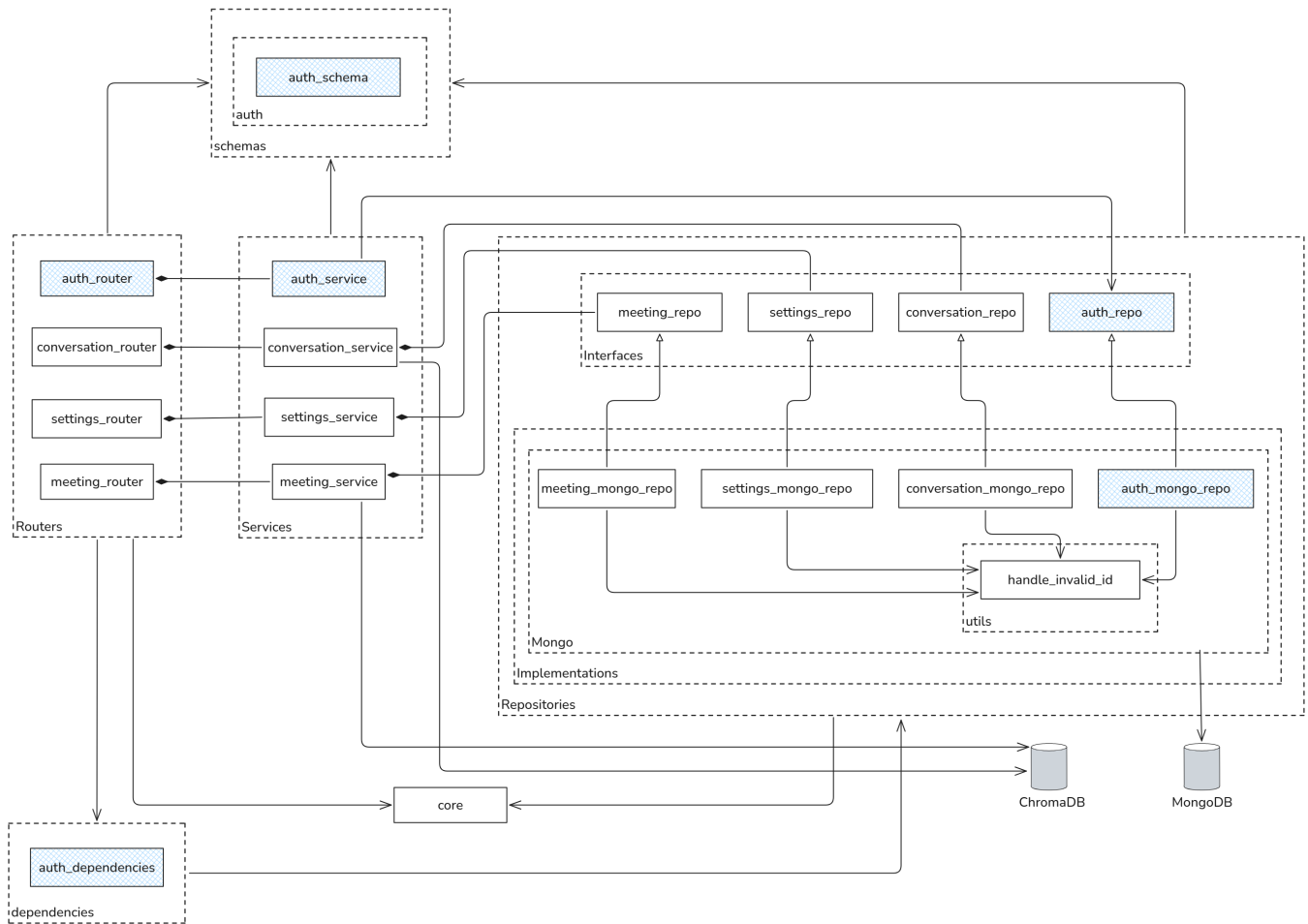


Figura 5.10: Diagrama de componentes del *backend*. En color azul se resaltan los módulos implementados en esta sección

- **Integridad y origen:** Verifica que la firma del *JWT* coincide con la clave secreta del servidor, garantizando que el *token* no ha sido manipulado por terceros.
- **Vigencia:** Comprueba que la fecha y hora actual no supera el tiempo de expiración.
- **Extracción de contexto:** Si el *token* es válido, se extrae el correo electrónico del *payload*. Este dato se inyecta directamente como una dependencia a la función del *endpoint* correspondiente, de modo que la capa de servicios sabe con certeza qué usuario ha realizado la petición, lo cual es necesario para poder realizar consultas a las bases de datos.

Esta implementación se realizó siguiendo la guía oficial de FastAPI [83].

Otro detalle a comentar con respecto a la autenticación es que el método `DELETE /auth` no elimina directamente la cuenta del usuario, sino que la marca para que esta sea eliminada

en un momento determinado en el futuro. Este periodo de retención y un correo electrónico de contacto se pueden obtener mediante el *endpoint* GET /auth/deletion-policy.

### Organización de la base de datos

Tal y como se expone en la Sección 3.3.2, la base de datos seleccionada es **MongoDB**, puesto que resulta idónea para almacenar las conversaciones con el sistema inteligente.

Por simplicidad, y dado que no existen relaciones complejas entre las entidades, se ha tomado la decisión de utilizar MongoDB tanto para las conversaciones como para el resto de entidades.

En cuanto a los datos almacenados, destacar que no se almacenan las contraseñas como tal, sino las contraseñas cifradas utilizando un algoritmo de hash unidireccional, evitando así exponer las contraseñas reales si un atacante consigue acceder a la base de datos, previniendo así uno de los riesgos identificados en el análisis de riesgos (Tabla 4.1, Sección 4.2).

En la Figura 5.11 se muestra el diagrama Entidad Relación de la base de datos.

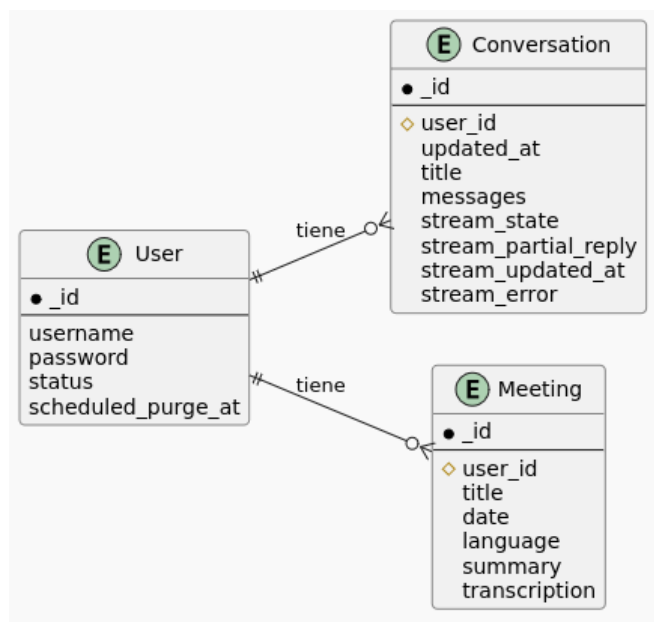


Figura 5.11: Diagrama Entidad Relación de la base de datos

### Pruebas

De manera análoga a las pruebas para el resto de dominios comentados anteriormente (reuniones y conversaciones), se realizaron pruebas manuales a través de Swagger UI y se definieron tests unitarios y de integración.

## 5.4 Sprint 4

Este *sprint* tiene dos objetivos. Por la parte del *backend* se implementa el dominio de ajustes de usuario, implementando el *CU-20. Configuración de la aplicación* y el *CU-21. Gestionar modelos de lenguaje*. Por la parte del *frontend*, se inicia el desarrollo de la interfaz, definiendo la arquitectura a alto nivel e implementando las vistas de *CU-07. Iniciar sesión*, *CU-08. Crear cuenta*, el sistema de navegación y la interfaz para subir reuniones (*CU-01. Subir grabación de reunión*).

### 5.4.1 Implementar ajustes de usuario

En esta última tarea correspondiente al *backend*, se implementa el dominio de `settings`.

En la Figura 5.12 se detallan los módulos implementados en esta tarea. También se puede apreciar cómo se desgranar los módulos `settings_router` y `settings_service` en cuatro componentes, de modo que el código quede mejor organizado; no se realiza esta división en el repositorio puesto que, como se comentó en la Sección 5.2.1, se está implementando el patrón repositorio, por lo que si se dividiere `settings_repository` en varios repositorios diferentes, se estaría permitiendo que los ajustes se distribuyesen en bases de datos diferentes, lo cual, por simplicidad, no se permite en este proyecto.

#### Definición del API y los *endpoints*

En la Figura 5.13 se muestran los *endpoints* identificados para los ajustes de usuario, desglosados en los diferentes routers que componen al `settings_router`.

#### Implementación

Como se puede observar, los *endpoints* identificados para esta etapa tienen una implementación bastante directa, puesto que ninguno cuenta con una lógica de negocio más allá de realizar comprobaciones básicas (como por ejemplo verificar que las *API Keys* introducidas sean válidas), o cifrar las *API Keys* antes de almacenarlas.

#### Organización de la base de datos

Se define una nueva colección en la base de datos para poder almacenar todos estos ajustes. En la Figura 5.14 se puede observar el diagrama Entidad Relación de la base de datos incluyendo esta nueva colección.

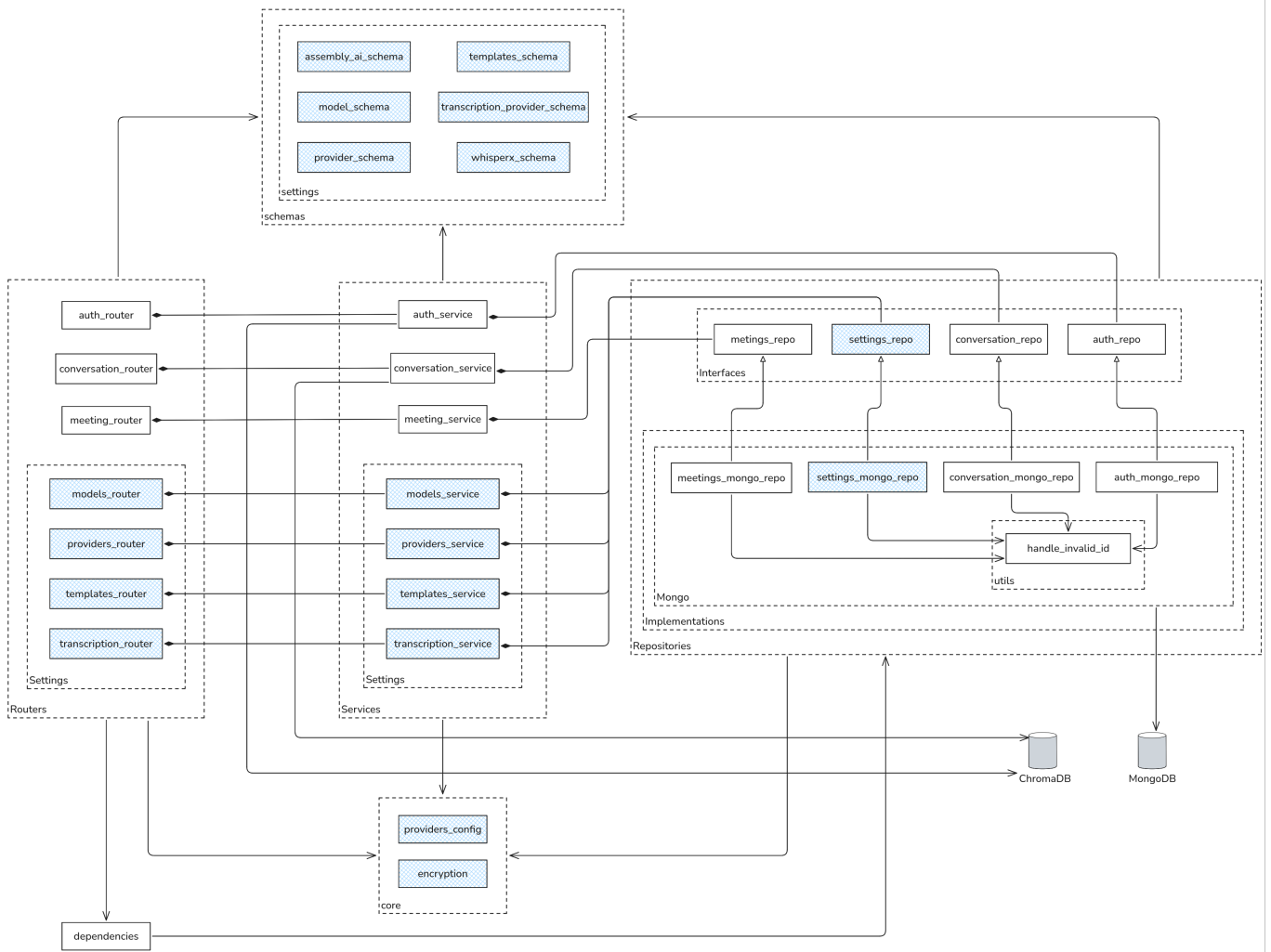


Figura 5.12: Diagrama de componentes del *backend*. En color azul se resaltan los módulos implementados en esta sección

## Pruebas

Al igual que en iteraciones anteriores, se realizaron pruebas manuales a través de Swagger UI y se implementaron tests unitarios y de integración.

### 5.4.2 Definición de la arquitectura a alto nivel del *frontend* de la aplicación

Antes de comenzar con la implementación del *frontend*, se definió una arquitectura en capas, con el objetivo de mantener una separación clara de responsabilidades y facilitar el mantenimiento del código.

En la Figura 5.15 se muestra la organización general, que se estructura en los siguientes

| Settings - Providers <small>Manage LLM providers and API keys</small>      |                                                           |                         |
|----------------------------------------------------------------------------|-----------------------------------------------------------|-------------------------|
| GET                                                                        | /settings/providers                                       | Get Providers           |
| POST                                                                       | /settings/providers/{provider_name}/api-key               | Add Provider Api Key    |
| DELETE                                                                     | /settings/providers/{provider_name}/api-key               | Delete Provider Api Key |
| Settings - Models <small>Manage model configurations</small>               |                                                           |                         |
| GET                                                                        | /settings/models/available                                | Get Available Models    |
| GET                                                                        | /settings/models/chat                                     | Get Chat Model          |
| PUT                                                                        | /settings/models/chat                                     | Update Chat Model       |
| GET                                                                        | /settings/models/summary                                  | Get Summary Model       |
| PUT                                                                        | /settings/models/summary                                  | Update Summary Model    |
| GET                                                                        | /settings/models/retrieval                                | Get Retrieval Model     |
| PUT                                                                        | /settings/models/retrieval                                | Update Retrieval Model  |
| POST                                                                       | /settings/models/pull                                     | Pull Model              |
| Settings - Transcription <small>Manage transcription configuration</small> |                                                           |                         |
| GET                                                                        | /settings/transcription/providers                         | Get Providers           |
| POST                                                                       | /settings/transcription/providers/{provider_name}/api-key | Add Provider Api Key    |
| DELETE                                                                     | /settings/transcription/providers/{provider_name}/api-key | Delete Provider Api Key |
| PATCH                                                                      | /settings/transcription/active-provider                   | Set Active Provider     |
| GET                                                                        | /settings/transcription/available-options                 | Get Available Options   |
| GET                                                                        | /settings/transcription/languages                         | Get Supported Languages |
| GET                                                                        | /settings/transcription/configuration                     | Get Configuration       |
| PATCH                                                                      | /settings/transcription/configuration                     | Update Configuration    |
| Settings - Templates <small>Manage templates settings</small>              |                                                           |                         |
| GET                                                                        | /settings/templates/default-prompt                        | Get Default Prompt      |
| GET                                                                        | /settings/templates/prompt                                | Get User Prompt         |
| PUT                                                                        | /settings/templates/prompt                                | Update User Prompt      |

Figura 5.13: Captura de Swagger UI con los *endpoints* del dominio settings

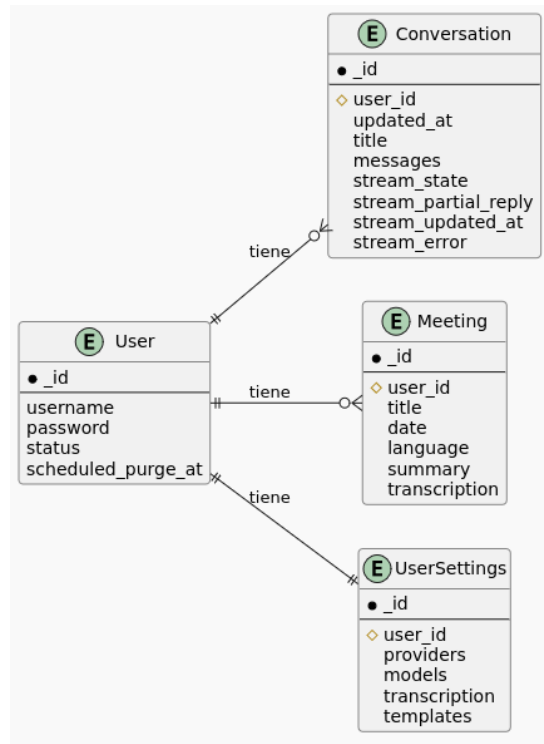


Figura 5.14: Diagrama Entidad Relación de la base de datos

niveles:

- `pages/`: Capa superior que define las diferentes páginas de la aplicación. Cada página es un componente de React que orquesta subcomponentes y se asocia a una ruta mediante React Router.
- `components/`: Capa intermedia que contiene los componentes reutilizables de la interfaz, divididos en dos niveles:
  - **Componentes de funcionalidad**: Agrupados por dominio (`chat/`, `meetings/`, `settings/`, `auth/`). Cada grupo encapsula la lógica y la presentación específica de esa funcionalidad.
  - **Componentes de *User Interface* (UI) primitivos** (`ui/`): Elementos genéricos y reutilizables como botones, campos de entrada, diálogos modales o indicadores de error. No están acoplados a ninguna funcionalidad concreta.
- `stores/`: Capa de gestión del lado del cliente. Utiliza Zustand para almacenar información global de la interfaz.

- `api/`: Capa de comunicación con el *backend*. Centraliza toda la interacción con el servidor.
- `hooks/`: Se definen *hooks* personalizados para lógica reutilizable.
- `utils/`: Módulo transversal que proporciona funcionalidades comunes al resto de capas.

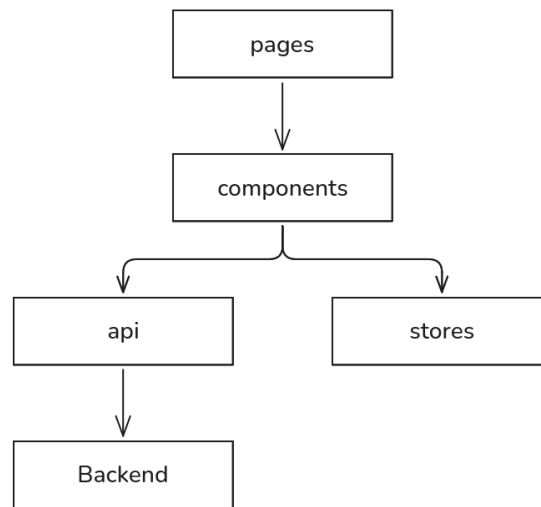


Figura 5.15: Diagrama de componentes del *frontend* a alto nivel

### 5.4.3 Implementación de la autenticación

Una vez implementada la autenticación en la parte del *backend* en el *sprint* anterior (Sección 5.3.2), es necesario desarrollar la contraparte en el *frontend* para que el usuario pueda iniciar sesión o registrarse.

#### Implementación

La autenticación en el *frontend* consta de tres elementos principales: el almacenamiento del *JWT*, la gestión del estado de autenticación y la protección de rutas.

El estado de autenticación se almacena mediante una *store* de Zustand con dos valores booleanos. El primero de ellos, `isUserAuth`, indica si el usuario está autenticado y controla qué rutas son o no son accesibles. El segundo, `authInitialized`, evita que la interfaz se muestre brevemente con el estado incorrecto al cargar la página: al iniciar la aplicación, si existe un *JWT* en el `localStorage`, se verifica su validez mediante una petición `GET /auth/me` antes de mostrar cualquier contenido.



Para proteger las rutas se definieron dos *guardianes de ruta* utilizando React Router. El primero, `ProtectedRoute`, redirige al usuario a la página de inicio de sesión si es que este no está autenticado, protegiendo así las rutas principales de la aplicación. El segundo, `PublicOnlyRoute`, redirige al usuario a la página principal si este ya ha iniciado sesión, impidiendo que acceda a las páginas de autenticación estando ya autenticado.

Cuando el `JWT` expira, cualquier petición a un *endpoint* protegido devuelve un código 401 `Unauthorized`. La capa de comunicación detecta esta respuesta y lanza un evento personalizado `unauthorized`. Un componente escucha este evento y, al recibirlo, elimina el `JWT` almacenado, limpia la caché de `TanStack Query`, actualiza el estado de autenticación y redirige al usuario a la página de inicio de sesión. Este diseño desacopla la lógica de comunicación de la interfaz de usuario, permitiendo que cualquier petición pueda desencadenar el cierre de sesión sin necesidad de modificar los componentes involucrados con dicha petición.

En la Figura 5.16 se observan las vistas de página principal para usuarios no autenticados, la vista de inicio de sesión y la vista de registro.



Figura 5.16: Vistas para un usuario no autenticado

## Pruebas

Además de pruebas manuales a través de la interfaz, se implementaron tests de integración que cubren los formularios de inicio de sesión y registro, los guardianes de las rutas y el manejo de sesiones expiradas.

#### 5.4.4 Implementación de la navegación

Tal y como se puede observar en los *mockups* de la interfaz de usuario (Anexo D), la navegación entre las diferentes vistas de la aplicación se realiza principalmente mediante un panel lateral, recordando así al diseño de otras aplicaciones web para interactuar con LLMs, como pueden ser ChatGPT, Gemini o Claude.

##### Implementación

Aunque visualmente todos los elementos de navegación del panel lateral son muy similares, se distinguen dos tipos de componentes diferentes:

- **Button**: Se utilizan botones para mostrar los modales de subir reuniones y de configuración.
- **NavLink**: Se utilizan “NavLinks” (componentes proporcionados por la librería React Router, Sección 3.3.1) para navegar entre las diferentes vistas de la aplicación.

En la Figura 5.17 se observa cómo se ve implementada la barra lateral en la aplicación.

Destacar que también se ha añadido la funcionalidad de abrir y cerrar este panel lateral, ya que, aunque no estaba representado en los *mockups* iniciales (Anexo D), sí es algo necesario para que la aplicación se pueda utilizar de manera cómoda en pantallas pequeñas. El estado del panel lateral (abierto, cerrado) se almacena en una *store* de Zustand (Sección 3.3.1), puesto que al abrirse y cerrarse, no solo se modifica el propio panel lateral, sino que también desplaza el contenido de la página.

##### Pruebas

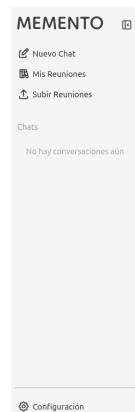
Para esta primera tarea en el *frontend*, únicamente se realizaron pruebas manuales a través de la propia interfaz, debido a que la funcionalidad implementada es tan sencilla que no requiere (al menos en un principio) tests automatizados.

#### 5.4.5 Implementar subida de reuniones

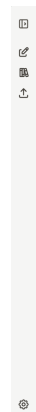
Esta tarea consiste en implementar el modal que permite al usuario subir sus reuniones al sistema. Este se corresponde con la vista 9 - *Vista de subir reuniones* de los *mockups* (Anexo D).

Como se puede observar en dicha imagen, el usuario podrá subir una o más reuniones de manera simultánea, añadiendo, para cada una de ellas, los siguientes campos:

- Título.



(a) Panel lateral abierto



(b) Panel lateral cerrado

Figura 5.17: Comparativa de la interfaz de la aplicación según el estado del panel lateral

- Fecha.
- Idioma (opcional).
- Número de hablantes (opcional).
- Archivo de audio.

### Implementación

Esta funcionalidad se implementó como un modal que contiene un formulario dinámico. Este formulario permite al usuario subir varias reuniones en una única operación, en lugar de tener que enviarlas todas de manera individual.

El estado de este formulario se gestiona a través de un array de objetos en el estado local. Cada elemento del array representa una reunión y se identifica mediante un *Universally Unique Identifier (UUID)*.

Antes del envío se comprueba que cada reunión disponga de título, fecha e idioma; una vez los datos han sido validados, se construye una nueva carga `multipart/form-data` que contiene, por un lado, los metadatos de las reuniones serializados como `JSON` y, por otro lado, los archivos de audio. Esta carga se envía luego al *backend* a través de una petición `POST /api/meetings` y se lleva a cabo el flujo visto en la Sección 5.2.2.

En la Figura 5.18 se muestra el modal implementado. Se puede observar que el diseño es ligeramente diferente con respecto al mostrado en los *mockups* (Anexo D), puesto que se han modificado los botones de *descartar* y *cancelar* por imágenes en formato *Scalable Vector Graphics* (SVG), haciendo que la interfaz sea más clara y minimalista. De todas formas, esta vista presenta algunos inconvenientes, por lo que en iteraciones posteriores se verá modificada (ver *sprint* 6, Sección 5.6).

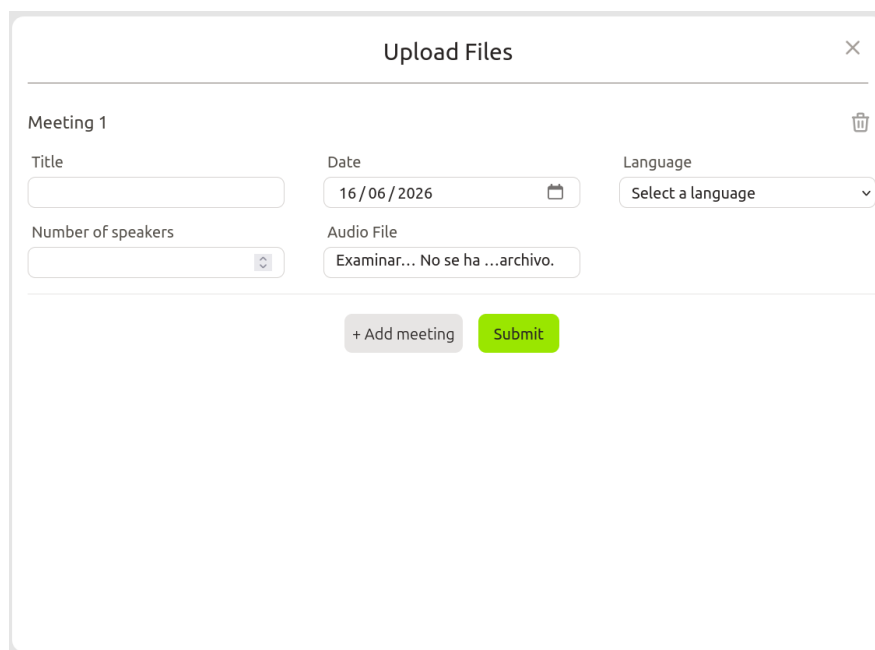
The image shows a modal window titled "Upload Files" with a close button (X) in the top right corner. Inside the modal, there is a section labeled "Meeting 1" with a trash icon to its right. Below this label, there are four input fields: "Title" (a text box), "Date" (a date picker showing "16 / 06 / 2026"), "Language" (a dropdown menu with "Select a language" and a downward arrow), and "Number of speakers" (a spinner box). Below these fields is an "Audio File" section with a text box containing "Examinar... No se ha ...archivo." and a file selection icon. At the bottom of the modal, there are two buttons: "+ Add meeting" and "Submit".

Figura 5.18: Modal de subir reuniones

## Pruebas

Para esta funcionalidad, además de realizar pruebas a mano a través de la propia interfaz, también se implementaron tests unitarios para verificar la correcta validación de los campos del formulario, así como tests de integración para cubrir el renderizado del formulario dinámico, la adición y la eliminación de reuniones, la correcta visualización de los errores y el flujo completo del envío mediante `POST` a un *backend* mockeado (*mocking*).

## 5.5 Sprint 5

El objetivo de este *sprint* es implementar las funcionalidades restantes del *frontend*, en concreto:

- **Vista del chat**, completando la interacción con el asistente (CU-06. *Realizar consulta conversacional*) y la gestión del historial de conversaciones (CU-16. *Gestionar historial de conversaciones*).
- **Gestión del historial de conversaciones**. Se completa el panel lateral de navegación, implementado en el *sprint* 4 (Sección 5.4.4) añadiendo la lista de chats, desde la que se puede tanto navegar a los diferentes chats como también editar su título o eliminarlos, implementando así la parte del *frontend* relativa al CU-16. *Gestionar historial de conversaciones*.
- **Vista de las reuniones**, implementando la interfaz para CU-02. *Visualizar lista de reuniones*, CU-04. *Modificar metadatos de una reunión* y CU-05. *Eliminar una reunión*.
- **Vista de la transcripción y el resumen de una reunión** (CU-03. *Consultar detalles de una reunión*).
- **Vista de ajustes de usuario**, implementando la interfaz para CU-19. *Gestionar cuenta*, CU-20. *Configuración de la aplicación* y CU-21. *Gestionar los modelos de lenguaje*.

### 5.5.1 Vista del chat

En este apartado se incluye tanto la funcionalidad de crear un nuevo chat como la de interactuar con un chat ya existente, debido a su similitud.

#### Implementación

Las vistas de *chat* y *nuevo chat* se conciben como vistas muy similares, aunque no iguales. La vista de *nuevo chat* reside en la ruta `chats`, mientras que la vista de *chat* reside en la ruta `chats/<chat_id>`.

La secuencia de la creación de un chat es la siguiente:

1. El usuario envía un primer mensaje desde la vista *nuevo chat*.
2. El *frontend* envía esta petición al *backend*.
3. El *backend* ejecuta el flujo comentado en la Sección 5.3.1, devolviendo la respuesta generada por el LLM y el identificador del chat.

4. El *frontend* se redirige a la ruta `chats/<chat_id>` correspondiente con el identificador del chat recibido.

Destacar que, para evitar que la página se quede congelada mientras se espera la respuesta, se muestra un *spinner* y un texto indicando así al usuario que los datos están siendo procesados por el *backend*. Esto será mejorado posteriormente en el *sprint* 6 (Sección 5.6), puesto que se sustituirá la comunicación entre el *frontend* y el *backend* a través de peticiones POST por una comunicación bidireccional utilizando *WebSockets* (WS).

En la Figura 5.19 se pueden observar las vistas de *nuevo chat* y de *chat*.

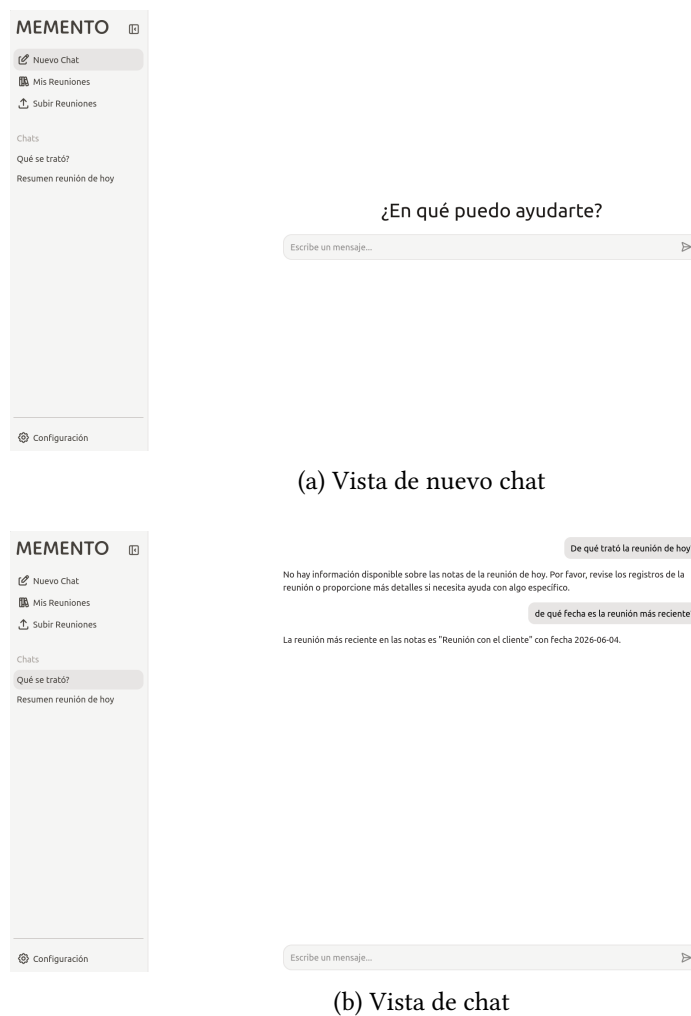


Figura 5.19: Vistas del asistente conversacional en su estado inicial y durante una consulta.

## Pruebas

Además de comprobar el correcto funcionamiento de esta funcionalidad a través de la interfaz, se implementaron pruebas de integración para comprobar que el flujo comentado anteriormente funciona correctamente.

### 5.5.2 Lista de chats

Para permitir al usuario acceder a las diferentes conversaciones que este ha tenido, editar su título y eliminarlas, se añadió la lista de chats a la barra lateral de la aplicación.

## Implementación

La lista de conversaciones se muestra en la barra lateral, se puede apreciar un ejemplo en la Figura 5.20. Esta lista se obtiene del *backend* mediante TanStack Query y se muestran ordenados por fecha de última actualización, de más reciente a más antigua.

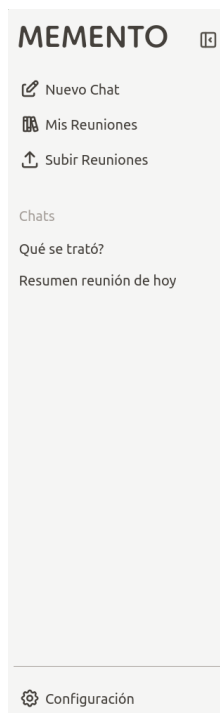


Figura 5.20: Panel lateral con la lista de chats

Cada uno de los elementos de la lista contiene:

- Un enlace a la ruta `/chats/<chat_id>` con el título de la conversación.
- Un menú desplegable con las operaciones de renombrar y eliminar.

La edición del título se realiza directamente sobre el elemento de la lista. Al seleccionar la opción de renombrar, el campo texto se habilita y se selecciona automáticamente el contenido para que el usuario pueda modificarlo. Los cambios se guardan al presionar la tecla `Enter` o al hacer clic fuera del campo. La eliminación, por su parte, se realiza de manera inmediata al seleccionar la opción correspondiente.

Destacar que, aquellas reuniones cuyos títulos son demasiado largos, como para ser mostrados de manera completa, si el usuario coloca el cursor sobre este, se puede visualizar el título completo.

### Pruebas

Se realizaron tests de integración para verificar el correcto funcionamiento de la lista de conversaciones. En concreto, se comprobó que la lista se renderiza correctamente tras la carga, que se muestra un mensaje de error si el `API` falla y que se indica adecuadamente cuando no hay conversaciones. También se probaron las funcionalidades de renombrar y eliminar reuniones.

#### 5.5.3 Vista de reuniones

En este apartado se implementa la vista de la lista de reuniones, así como las funcionalidades relacionadas dentro de esta propia vista:

- Buscar reuniones por título.
- Filtrar por rango de fecha.
- Ordenar la lista según diferentes criterios (más recientes, más antiguas, orden alfabético y orden alfabético inverso).
- Editar título y fecha de reuniones.
- Eliminar reuniones.
- Paginación.

### Implementación

La vista de reuniones se estructura en torno a tres capas bien diferenciadas: el estado de los filtros, el procesamiento de los datos y la interfaz de usuario.

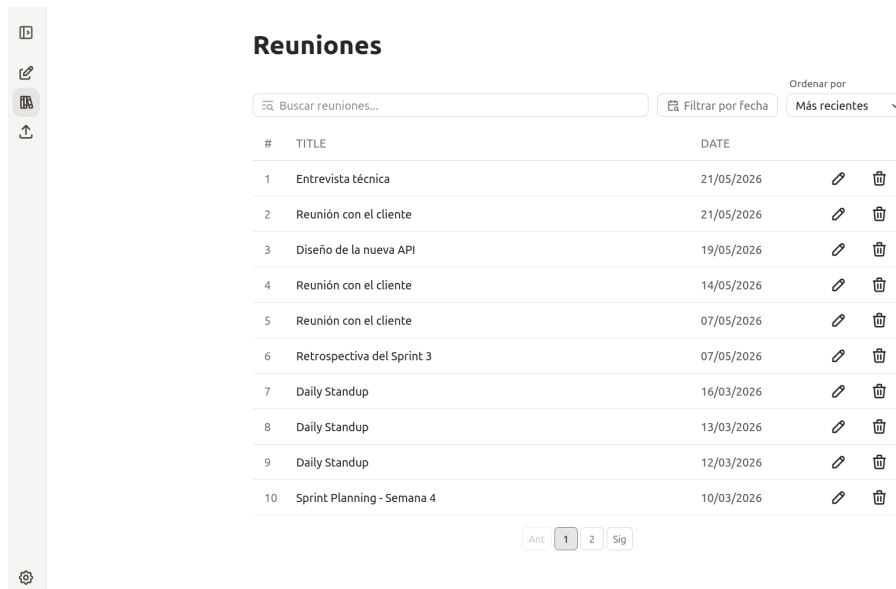
El estado de la vista se gestiona a través de un `store` de Zustand. Este `store` almacena el término de búsqueda, los filtros de fecha (desde/hasta), el criterio de ordenación, la página



actual y el número de reuniones mostradas por página (el cual se configura desde la vista de ajustes generales, como se verá en la Sección 5.5.5).

El procesamiento de los datos se realiza íntegramente en el *frontend* a través de un *hook* personalizado. Este *hook* recibe el listado completo de reuniones (obtenido mediante TanStack Query) y aplica secuencialmente el filtrado por título (utilizando un algoritmo de *fuzzy matching*) el filtro por rango de fechas, la ordenación según criterio seleccionado y, finalmente, la paginación. La paginación se implementa mediante el troceado del array resultante.

En la Figura 5.21 se muestra la interfaz de usuario para esta vista. Se observa que se dispone de un botón para poder mostrar/ocultar los filtros de fecha y, en caso de estar activos, se indica visualmente (Figura 5.22).



| #  | TITLE                      | DATE       |  |  |
|----|----------------------------|------------|--|--|
| 1  | Entrevista técnica         | 21/05/2026 |  |  |
| 2  | Reunión con el cliente     | 21/05/2026 |  |  |
| 3  | Diseño de la nueva API     | 19/05/2026 |  |  |
| 4  | Reunión con el cliente     | 14/05/2026 |  |  |
| 5  | Reunión con el cliente     | 07/05/2026 |  |  |
| 6  | Retrospectiva del Sprint 3 | 07/05/2026 |  |  |
| 7  | Daily Standup              | 16/03/2026 |  |  |
| 8  | Daily Standup              | 13/03/2026 |  |  |
| 9  | Daily Standup              | 12/03/2026 |  |  |
| 10 | Sprint Planning - Semana 4 | 10/03/2026 |  |  |

Ant 1 2 Sig

Figura 5.21: Vista de reuniones

Cada reunión se representa como una fila en la lista, mostrando el número de índice, el título, la fecha y los botones para editar y eliminar dicha reunión. La edición se realiza directamente sobre la fila, al hacer clic en el botón de editar, los campos de título y fecha se convierten en elementos de entrada modificables, con botones para confirmar o cancelar los cambios (Figura 5.23). Al confirmar, se envía una petición PATCH al *backend* únicamente con los campos modificados. La eliminación, por su parte, requiere una confirmación a través de un modal (Figura 5.24).

Por último, destacar que tanto la edición como la eliminación utilizan *optimistic updates* de TanStack Query, actualizando la interfaz de forma inmediata y revirtiendo los cambios en caso de error.

## Reuniones

Buscar reuniones...

Filtrar por fecha

Ordenar por Más recientes

Desde dd/mm/aaaa Hasta dd/mm/aaaa Limpiar

| # | TITLE              | DATE       |  |  |
|---|--------------------|------------|--|--|
| 1 | Entrevista técnica | 21/05/2026 |  |  |

(a) Filtro de fecha desactivado

## Reuniones

Buscar reuniones...

Filtrar por fecha

Ordenar por Más recientes

Desde 01/04/2026 Hasta 03/06/2026 Limpiar

| # | TITLE              | DATE       |  |  |
|---|--------------------|------------|--|--|
| 1 | Entrevista técnica | 21/05/2026 |  |  |

(b) Filtro de fecha activado

## Reuniones

Buscar reuniones...

Filtrar por fecha

Ordenar por Más recientes

| # | TITLE              | DATE       |  |  |
|---|--------------------|------------|--|--|
| 1 | Entrevista técnica | 21/05/2026 |  |  |

(c) Filtro de fecha activado y colapsado

Figura 5.22: Comparativa del listado de reuniones según el estado del filtro de búsqueda por fecha.

|   |                    |            |  |  |
|---|--------------------|------------|--|--|
| 1 | Entrevista técnica | 21/05/2026 |  |  |
|---|--------------------|------------|--|--|

Figura 5.23: Editar metadatos de una reunión

¿Seguro que deseas eliminar esta reunión?

Entrevista técnica - 21/05/2026

Cancelar Eliminar

Figura 5.24: Modal para confirmar la eliminación de una reunión

## Pruebas

Para la vista de reuniones, además de pruebas manuales a través de la interfaz, se implementaron tests unitarios y de integración.

En los tests unitarios se comprueba el funcionamiento del *hook* de filtrado, verificando la búsqueda difusa por título, el filtro por rango de fechas, la ordenación y el cálculo de páginas, así como el *store* de Zustand.

En los tests de integración se probó el correcto funcionamiento de los componentes de esta vista, cubriendo los distintos estados de la misma (cargando datos, lista vacía y error), la búsqueda y el filtrado desde la interfaz, la ordenación, la paginación con navegación entre páginas y las operaciones de edición y eliminación.

### 5.5.4 Vista de la transcripción y el resumen de una reunión

Desde la vista de reuniones implementada anteriormente, el usuario puede acceder a la vista de transcripción/resumen de la reunión. En concreto, el usuario accede a la vista de la transcripción haciendo clic en el título de la reunión, y desde ahí, el usuario puede alternar entre transcripción y resumen o volver a la lista de reuniones.

## Implementación

La implementación de estas vistas es bastante directa, el título de la reunión funciona como un NavLink, y al hacer clic el usuario pasa de la vista `/meetings` a estar la vista `/meetings/<meeting_id>/transcription`.

Como se puede observar en la Figura 5.25, ambas vistas son prácticamente idénticas. El componente que las implementa es el mismo, variando únicamente el contenido, el título de la vista y el destino del NavLink que permite alternar entre ellas.

Destacar que, en el caso del resumen, es el propio *backend* el que le da estructura de conversación al texto, dividiendo el diálogo en diferentes párrafos y añadiendo etiquetas HTML para que los identificadores de los hablantes aparezcan en negrita.

Para el caso de la transcripción, el contenido se renderiza haciendo uso de la librería *react-markdown*, que transforma texto en formato Markdown a HTML. Los estilos sobre este contenido se aplican mediante selectores arbitrarios de Tailwind CSS sobre los elementos generados, permitiendo que listas, títulos y párrafos mantengan una apariencia coherente con el resto de la interfaz.

## Pruebas

Aparte de las pruebas manuales realizadas a través de la interfaz, se definieron tests de integración para comprobar el correcto renderizado de los estados de carga, la visualización del



(a) Vista de transcripción

(b) Vista de resumen

Figura 5.25: Vistas de transcripción y de resumen de una reunión

contenido, el manejo de errores y la navegación entre las diferentes vistas. No se consideró necesaria la inclusión de tests unitarios adicionales puesto que los componentes implementados se limitan a renderizar contenido.

### 5.5.5 Vista de ajustes de usuario

La última tarea realizada en este *sprint* consiste en implementar las diferentes vistas de ajustes de usuario que se muestran en los *mockups* (Anexo D).

## Implementación

Los ajustes de usuario se implementan como un diálogo modal que se abre desde el botón de configuración, situado en la parte inferior de la barra lateral. El diálogo se estructura en dos columnas: una de navegación lateral, que permite seleccionar entre las diferentes categorías

de ajustes, y un área de contenido donde se muestran los controles correspondientes a la categoría seleccionada.

A continuación se detallan las categorías de ajustes, cuyas vistas correspondientes se pueden visualizar en la Figura 5.26:

- **General:** Permite configurar el tema de la interfaz (claro, oscuro o seguir la configuración del sistema), el idioma de la aplicación y el número de reuniones mostradas por página.
- **Proveedores:** Gestiona los proveedores de modelos de lenguaje, incluyendo la adición y la eliminación de [APIKeys](#), así como la descarga de los modelos disponibles en Ollama.
- **Transcripción:** Configura el proveedor de transcripción, pudiendo elegir entre WhisperX, que se ejecuta localmente y permite ajustar diferentes hiperparámetros; o AssemblyAI, que se ejecuta en la nube y requiere un [API Key](#).
- **Resúmenes:** Permite editar la plantilla del [prompt](#) utilizado para generar los resúmenes, con opción de reestablecer el valor por defecto, así como seleccionar el [LLM](#) encargado de esta tarea.
- **Chat:** Configura los modelos utilizados tanto para generar las respuestas como para generar el texto que se utilizará como [query](#) para recuperar información de los documentos sobre la base de datos vectorial.
- **Cuenta:** Permite editar diferentes aspectos de la cuenta de usuario, como su correo electrónico o la contraseña, también permite al usuario cerrar su sesión y eliminar su cuenta. Destacar que todos estos cambios requieren que el usuario introduzca su contraseña para poder llevarse a cabo, y en el caso de cambiar la contraseña, la nueva contraseña debe ser introducida dos veces, evitando así errores por parte del usuario.

Los ajustes que dependen del [backend](#) se gestionan mediante TanStack Query, mientras que las opciones que únicamente dependen del [frontend](#) (como cambiar el idioma o el tema) se gestionan a través de la librería [i18next](#) para el caso del idioma y una [store](#) de Zustand con persistencia para el resto de casos.

## Pruebas

Para las vistas de ajustes, además de verificar el correcto funcionamiento de manera manual a través de la interfaz, también se implementaron pruebas de integración para cada una de las nuevas categorías del diálogo de configuración.

(a) Generales

(b) Proveedores

(c) Transcripción

(d) Resumen

(e) Chat

(f) Cuenta

Figura 5.26: Diferentes paneles de la ventana de configuración del sistema.

En concreto, se verificó la correcta visualización de los formularios, la validación de los campos, el correcto envío de cambios al servidor y el manejo de errores. También se añaden tests unitarios para comprobar el correcto funcionamiento de la *store* de Zustand que modifica el tema de la aplicación.

## 5.6 Sprint 6

Este *sprint* tiene como objetivo realizar diversas mejoras sobre el proyecto desarrollado a lo largo de los *sprints* anteriores, de modo que se garantice una buena experiencia de usuario. Además, también se deben de modificar las conexiones entre el usuario y el *frontend* para que estas se realicen de manera segura. De este modo, en lugar de implementar nuevos casos de uso, el desarrollo de este *sprint* tiene como objetivo cubrir requisitos no funcionales. En concreto, las tareas a realizar son las siguientes:

- Implementación de un flujo *Server Sent Events* (SSE) para la subida de reuniones.
- Sustituir comunicación a través de peticiones POST por una conexión con *WebSockets* (WS) para la interacción con el chat.
- Añadir internacionalización, ofreciendo al usuario la posibilidad de utilizar la interfaz en castellano y en inglés.
- Añadir tema oscuro a la aplicación, seleccionando por defecto el tema del sistema.
- Añadir un *proxy* para que las conexiones entre el usuario y el *frontend* se realicen de manera segura.

Se observa que la implementación de los flujos SSE, WS y la implementación de la internacionalización y el cambio de idioma mejoran notablemente la experiencia de usuario, cumpliendo con la necesidad de una interfaz intuitiva (RNF-04). Además, se configuró un *proxy* para que las conexiones entre usuario y *frontend* sean seguras (RNF-09).

### 5.6.1 Implementación del flujo SSE

La subida de reuniones (implementada en el *backend* en el *sprint* 2, la Sección 5.2.2; e implementada en el *frontend* en el *sprint* 4, Sección 5.4.5) se realizaba mediante simples peticiones POST, en las que el usuario subía una serie de reuniones, y hasta que el procesamiento de todas estas terminaba, el usuario no podía consultar el estado de cada una de estas reuniones, es decir, si ya han sido procesadas, si están a la espera de ser procesadas, si ya han sido procesadas, o si una de ellas no ha podido ser procesada. Para solucionarlo, se introdujeron *Server Sent Events* (SSE) en esta comunicación.

#### Implementación

El mecanismo de esta comunicación es el siguiente: el *frontend* sigue realizando una petición POST con los archivos y los metadatos, pero el *backend* responde con un flujo continuo

de eventos en el que se indica cuál es el estado de la petición, en lugar de recibir toda esta información al terminar la petición.

Para realizar la implementación de esta comunicación, se utilizó la librería *sse-starlette* en el *backend* para generar el flujo de eventos, mientras que en el *frontend* se implementó un lector del *stream*, y se modificó la vista de subir reuniones para incluir la información sobre el estado de las mismas. Además, también se reestructuró el diseño del *frontend* para mantener un estilo más acorde al resto de la interfaz.

En concreto, los eventos añadidos fueron los siguientes:

- **JobStarted**: Indica que se ha recibido la petición y que ha comenzado el flujo de procesamiento de las reuniones.
- **MeetingProcessingStarted**: Indica que una reunión determinada ha comenzado a ser procesada.
- **MeetingProcessingSucceeded**: Indica que una reunión concreta ha sido correctamente procesada.
- **MeetingProcessingFailed**: Indica que el procesamiento de una reunión ha fallado.
- **JobFinished**: Indica que el procesamiento de las reuniones ha finalizado.

En la Figura 5.27 se puede observar el modal de subida de reuniones actualizado para poder mostrar el estado de una reunión; en concreto, se aprecia que en esta imagen, las reuniones 1 y 2 han sido procesadas correctamente, mientras que la reunión 3 no ha podido ser procesada correctamente, la 4 está siendo procesada, y por último, la 5 está a la espera de ser procesada. También se puede apreciar en la imagen que el diseño de este modal ha sido modificado con respecto al implementado originalmente en el *sprint* 4 (Sección 5.4.5, Figura 5.27), de modo que su aspecto sea más coherente con el resto de la interfaz.

## Pruebas

Para verificar el correcto funcionamiento del flujo *SSE* se implementaron tests de integración en el *backend* que verifican la secuencia de eventos emitidos durante el procesamiento de las reuniones. En el *frontend*, los tests de integración del formulario de subida simulan la respuesta *SSE* completa, verificando que la interfaz procesa correctamente el resultado final y maneja adecuadamente los casos de error.



Subir Reuniones ×

| # | ESTADO | TÍTULO                              | FECHA      | ARCHIVO                                                |
|---|--------|-------------------------------------|------------|--------------------------------------------------------|
| 1 | ✓      | Daily Standup                       | 02/06/2026 | Examinar audio_prueb... <span>▼</span> <span>🗑️</span> |
| 2 | ✓      | Reunión con el cliente              | 04/06/2026 | Examinar audio_prueb... <span>▼</span> <span>🗑️</span> |
| 3 | 🔴      | Seguimiento estudiante en prácticas | 10/06/2026 | Examinar audio_prueb... <span>▼</span> <span>🗑️</span> |
| 4 | 🔄      | Daily Standup                       | 03/06/2026 | Examinar audio_prueb... <span>▼</span> <span>🗑️</span> |
| 5 | 🕒      | Daily Standup                       | 04/06/2026 | Examinar audio_prueb... <span>▼</span> <span>🗑️</span> |

+ Añadir reunión Subiendo...

Figura 5.27: Vista de subida de reuniones actualizada

### Implementación de la conexión mediante WS

La interacción con el asistente conversacional (implementada en el *backend* en el *sprint* 3, Sección 5.3.1; e implementada en el *frontend* en el *sprint* 5, Sección 5.5.1) se realizaba originalmente mediante peticiones POST. El usuario enviaba un mensaje y debía esperar a que el LLM generase la respuesta completa antes de recibirla, lo que podía llegar a tardar bastantes segundos sin que el usuario tuviese información sobre el progreso. Para solucionarlo, se sustituyó esta comunicación por una comunicación mediante *WebSockets* (WS).

### Implementación

El mecanismo de esta comunicación es el siguiente:

- El *frontend* inicia la conexión al enviar un mensaje al *backend*, enviando el JWT del usuario como parámetro de autenticación.
- Una vez establecida la conexión, el *frontend* envía el mensaje, junto a un identificador del chat (en caso de ser una nueva conversación, este identificador será nulo).
- El *backend* procesa la solicitud y devuelve eventos a medida que avanza el flujo de generación de la respuesta, permitiendo que la interfaz se actualice en tiempo real.

Los eventos que devuelve el *backend* son los siguientes:

- `conversation_created`: Se genera cuando el mensaje corresponde a una nueva conversación, devolviendo el identificador de la misma.
- `retrieving`: Indica que el sistema está buscando información relevante en la base de datos vectorial para contextualizar su respuesta (Figura 5.28a).

- `thinking_start` y `thinking_end`: Delimitan el periodo en el que el modelo está generando su cadena de pensamiento (Figura 5.28b).
- `token`: Contiene texto acumulado de la respuesta del asistente, permitiendo que la interfaz muestre el mensaje a medida que se está generando.
- `title`: Se genera cuando el *backend* ha determinado un título para la conversación.
- `done`: Indica que la respuesta ha sido completada y que la conexión se cerrará.
- `error`: Se genera cuando se produce un error, cerrando la conexión.

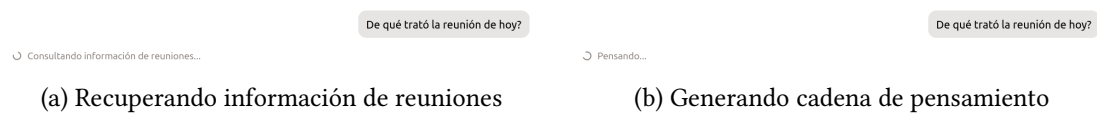


Figura 5.28: Indicadores visuales de estado del asistente durante el procesamiento de la consulta

Adicionalmente, se implementó un filtro en el *backend* para evitar enviar al *frontend* las cadenas de pensamiento.

Otra característica añadida al introducir la conexión *WS* es la generación automática del título, el cual es generado automáticamente por el *LLM* seleccionado para el usuario para la conversación.

## Pruebas

Para comprobar el correcto funcionamiento de la conexión mediante *WS* se implementaron tests de integración tanto para el *backend* como para el *frontend*.

En el *backend*, los tests de integración verifican que tanto la creación de conversaciones como el envío de mensajes a través de *WS* funcionan correctamente, incluyendo el *streaming* de la respuesta del asistente y el manejo de errores. También se implementaron tests unitarios para comprobar el correcto funcionamiento del filtrado de la cadena de pensamiento.

En el *frontend*, los tests de integración verifican el comportamiento completo de la vista de chat: carga del historial, envío de mensajes, visualización progresiva de la respuesta, y también los indicadores visuales de procesamiento de recuperación de contexto y pensamiento del modelo. También se comprueba el correcto funcionamiento de la actualización del título mediante un test unitario.

### 5.6.2 Internacionalización

Para que la aplicación pueda ser utilizada en varios idiomas, se añadió internacionalización. En concreto, la aplicación está disponible en inglés y en español.

## Implementación

La internacionalización se implementó utilizando la librería *react-i18next*. Los textos que se muestran en la interfaz se definen en ficheros **JSON** separados por idioma, organizados en cada uno de estos ficheros por categorías funcionales (autenticación, barra lateral, reuniones, ajustes, etc.).

El idioma se detecta automáticamente al cargar la aplicación, primero se comprueba si hay un idioma seleccionado en el `localStorage` del navegador que el usuario haya escogido anteriormente, en caso de no haberlo, se utiliza el idioma configurado en el navegador. El usuario puede cambiar el idioma desde la vista de ajustes generales, y el cambio se aplica inmediatamente a toda la interfaz sin necesidad de recargar la página.

Destacar que si el usuario está utilizando la plantilla de resúmenes por defecto, cuando cambia el idioma, también se cambia la plantilla por la del idioma que selecciona. Esto únicamente ocurre si la plantilla seleccionada es la plantilla por defecto, evitando así eliminar las plantillas modificadas por los usuarios.

## Pruebas

Dada la simplicidad de esta tarea y a que la internacionalización se implementa a través de una librería externa, se decidió contar únicamente con pruebas manuales para la internacionalización.

### 5.6.3 Añadir tema oscuro a la aplicación

Para ofrecer una experiencia visual al usuario más cómoda se implementó el tema oscuro. La aplicación permite seleccionar tres modos: *claro*, *oscuro* y *sistema*.

## Implementación

La implementación se basa en el modo oscuro que incluye Tailwind CSS. Esta implementación permite añadir atributos de Tailwind CSS precedidos por un `dark:`, los cuales se activarán dinámicamente únicamente para el tema oscuro.

Para la implementación del modo *sistema*, el *frontend* se suscribe al *media query prefers-color-scheme* del navegador. De esta forma, si el usuario cambia el tema de su navegador mientras la aplicación está abierta, el tema se actualiza automáticamente.

En la Figura 5.29 se puede apreciar un ejemplo de una misma vista en tema claro y oscuro, y en el Anexo E se pueden observar todas las vistas de la interfaz, tanto en tema claro como oscuro.

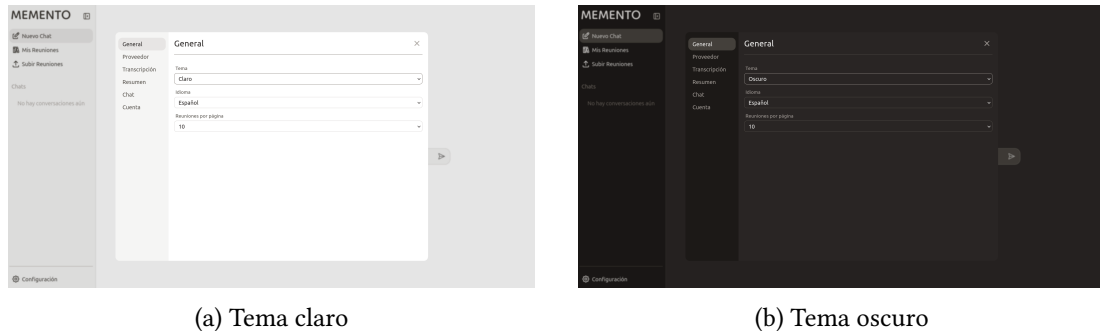


Figura 5.29: Vista de la interfaz de configuración general en sus variantes visuales

## Pruebas

Se implementaron tests unitarios para comprobar el funcionamiento del tema *sistema*.

### 5.6.4 Añadir *proxy* para conexiones seguras

Para que el acceso a la aplicación se realice de manera segura, se añadió un *proxy* que actúa como puerta de entrada a la aplicación.

## Implementación

Este *proxy* se añadió a través de *nginx-proxy-manager*, servicio el cual se añadió a la configuración de Docker Compose. Este servicio expone los puertos 80 (HTTP) y 443 (HTTPS), y se encarga de la obtención y la renovación automática de certificados SSL mediante Let's Encrypt.

El *proxy* recibe las peticiones entrantes y las redirige al *frontend*. Destacar que el *proxy* está configurado de tal modo que cualquier petición HTTP recibida se redirige automáticamente a la petición HTTPS, de modo que toda la comunicación entre el usuario y la aplicación esté siempre cifrada.

Se utiliza también el dominio `esteban-suarez-tfg.duckdns.org`, proporcionado por el servicio DuckDNS. La configuración del *proxy* se realiza a través de la interfaz de administración de *nginx-proxy-manager*, accesible únicamente desde el propio servidor.

Adicionalmente, fue necesario configurar Vite para que el *frontend* permitiese conexiones desde este dominio.

## Pruebas

El correcto funcionamiento del *proxy* se verificó de manera manual, comprobando que se podía acceder al *frontend* de la aplicación mediante el dominio.

## 5.7 Sprint 7

Tal y como se comentó en la planificación inicial (Sección 4.3), este último *sprint* se reservó para llevar a cabo la documentación del proyecto, redactando así el manual de usuario (Anexo G), la guía de instalación (Anexo F) y a la redacción del presente documento.

# Conclusiones

---

Tras el diseño, desarrollo y evaluación del sistema propuesto, se concluye que el producto final cumple de manera satisfactoria todas las especificaciones y objetivos definidos en la fase inicial del proyecto. Se ha logrado construir una plataforma funcional, robusta y competitiva que da respuesta directa a las problemáticas identificadas en el estudio de mercado (Capítulo 2): privacidad, seguridad y dependencia económica presentes en las soluciones comerciales actuales.

La consecución de este proyecto permite extraer una serie de conclusiones que se estructuran en dos grandes bloques: consecución de objetivos y logros técnicos, y el impacto formativo y crecimiento profesional.

### 6.1 Consecución de objetivos y logros técnicos

Desde el punto de vista técnico, el sistema desarrollado demuestra que es viable construir una alternativa de código abierto con ejecución local a los asistentes de reuniones privados. Entre los hitos técnicos más destacables cabe mencionar:

- **Procesamiento avanzado de reuniones multiusuario:** Se ha integrado con éxito un flujo que permite no solo transcribir las reuniones, sino también de realizar diarización de hablantes. Esto permite que el sistema identifique y separe las intervenciones de varios usuarios en una misma reunión, lo que aporta un contexto vital para la posterior generación de resúmenes estructurados basados en plantillas configurables por el usuario.
- **Extracción de conocimiento mediante lenguaje natural:** Gracias a la integración de *Large Language Models (LLMs)*, el usuario puede realizar consultas mediante lenguaje natural a un asistente conversacional sobre las reuniones procesadas.

- **Arquitectura robusta, escalable y extensible:** Se ha diseñado una arquitectura cliente servidor claramente desacoplada. La implementación de los patrones estrategia, repositorio y factoría aseguran que el sistema sea fácilmente extensible en el futuro, permitiendo integrar nuevas bases de datos o proveedores de inteligencia artificial sin alterar la lógica de negocio central.

## 6.2 Impacto formativo y crecimiento profesional

Más allá de los resultados técnicos, la realización de este Trabajo de Fin de Grado ha supuesto un pilar fundamental en mi desarrollo como informático. Como estudiante de la mención en Computación, mi trayectoria académica me ha proporcionado una fuerte base en ámbitos como la *Inteligencia Artificial*; sin embargo, considero que mi formación como graduado en Ingeniería Informática habría quedado incompleta sin el desarrollo de este proyecto, puesto que carecía de conocimientos en áreas transversales y altamente demandadas, como la gestión de proyectos software y el desarrollo web.

Para superar este reto y llevar a cabo la interfaz de usuario, ha sido necesario un aprendizaje intensivo en tecnologías *frontend* modernas. En concreto, el dominio de React requirió la realización de un curso formativo de más de 70 horas de duración (*React - The Complete Guide* [84]). Esto me ha permitido diseñar e implementar una *Single Page Application (SPA)* funcional y responsiva, complementando mi perfil de Computación con habilidades más propias de la mención en Ingeniería de Software.

Adicionalmente, este proyecto me ha obligado a salir de mi zona de confort para investigar y aplicar tecnologías de vanguardia, tales como la arquitectura *Retrieval Augmented Generation (RAG)* o el *Model Context Protocol (MCP)*.

Por último, el desarrollo de “Memento” me ha enfrentado a problemas reales propios del mundo laboral. Un ejemplo claro fue el obstáculo técnico encontrado al integrar el protocolo MCP con LLMs de pesos abiertos. Al constatar la incompatibilidad e incapacidad de estos modelos locales para operar con dicho protocolo, fue necesario pivotar hacia una implementación basada en RAG. Aprender a evaluar alternativas viables, readaptar la planificación y solucionar problemas en un entorno tecnológico inestable ha aportado una experiencia con un valor incalculable de cara a mi futura inserción en el mundo laboral.

# Futuros desarrollos

---

A pesar de haber cumplido con los objetivos planteados al inicio del proyecto, su naturaleza permite plantear diversas líneas de trabajo para futuras iteraciones.

En primer lugar, se podría mejorar la experiencia de usuario de la aplicación web añadiendo funcionalidades ligeramente más avanzadas. Por ejemplo, sería de gran utilidad permitir la cancelación de la subida y procesamiento de reuniones en curso. Asimismo, la interacción con el asistente conversacional podría enriquecerse ofreciendo la opción de detener al modelo mientras está generando una respuesta, o la posibilidad de editar mensajes previamente enviados, tal y como ocurre en otras plataformas similares como ChatGPT o Claude. Otra mejora con respecto a la interacción con el asistente conversacional sería incluir citas a los documentos consultados, de modo que desde el propio chat el usuario pueda ver exactamente qué documentos y qué partes de estos documentos han sido tomados como referencia para generar las respuestas, como ocurre en plataformas como NotebookLM.

Por otro lado, el sistema tiene el potencial de evolucionar más allá de un asistente conversacional para convertirse en un gestor integral de conocimiento. Resultaría muy útil que el usuario pudiese añadir notas personales a las reuniones o modificar manualmente los resúmenes generados por la aplicación. En línea con esto último, una mejora sustancial sería permitir la asignación de nombres reales a cada uno de los hablantes identificados en la diarización. Esto habilitaría al usuario para realizar consultas específicas sobre lo expuesto por un participante en concreto. Esta línea de trabajo futuro podría incluso llevar al desarrollo de herramientas MCP que permitan a los LLMs acceder no sólo a transcripciones, resúmenes y las notas de los usuarios anteriormente mencionadas, sino también permitirles acceder a aplicaciones externas, como al calendario o a correos electrónicos.

Para optimizar la administración de las reuniones procesadas, sería conveniente implementar herramientas de organización más robustas, como la capacidad de asignar etiquetas a las reuniones o la opción de estructurarlas mediante directorios, también se debería permitir seleccionar varias reuniones para poder eliminarlas de manera conjunta, en lugar de tener que



proceder de forma individual. Del mismo modo, la adaptación de la interfaz para dispositivos móviles permitiría al usuario acceder a sus reuniones en cualquier momento y lugar.

Desde un punto de vista arquitectónico, otra vía de desarrollo consistiría en hacer el *backend* de la aplicación más escalable mediante una integración de una arquitectura de microservicios. De este modo, la alta carga podría ser distribuida de manera eficiente entre diferentes dispositivos.

La consecución de todas estas vías de desarrollo permitiría en un futuro convertir esta herramienta en un producto comercial bajo un modelo *SaaS*, capaz de competir directamente con las alternativas comerciales existentes actualmente en el mercado.

# **Apéndices**

Apéndice A

## Épicas

---

# Épicas

## Gestión de reuniones

1. **Subida de grabaciones.** Como usuario, quiero subir archivos de audio para que la aplicación los procese.
2. **Validación del formato.** Como usuario, quiero que el sistema me avise si intento subir un archivo en un formato no válido, para evitar errores.
3. **Listado de reuniones.** Como usuario, quiero ver un listado de todas las reuniones que he procesado, para acceder fácilmente a su resumen o transcripción.
4. **Filtrado y ordenación.** Como usuario, quiero filtrar y ordenar mis reuniones por fecha o título, para encontrar rápidamente las que necesito.
5. **Consulta de detalles.** Como usuario, quiero seleccionar una reunión para ver su transcripción y resumen, para consultar su contenido.
6. **Edición de metadatos.** Como usuario, quiero editar el título fecha de una reunión, para mantenerlas organizadas.
7. **Eliminación de reuniones.** Como usuario, quiero eliminar reuniones que ya no necesito, para mantener únicamente las relevantes.

## Procesamiento de grabaciones

1. **Transcripción automática.** Como usuario, quiero que la aplicación transcriba automáticamente el contenido de mis grabaciones, para poder leer la conversación.
2. **Resumen estructurado.** Como usuario, quiero que la aplicación genere automáticamente un resumen de la reunión organizado según una plantilla, para entender rápidamente lo tratado.
3. **Estado del procesamiento.** Como usuario, quiero que el sistema me informe del estado del procesamiento en cada fase, para saber si ha finalizado correctamente.
4. **Reintento ante errores.** Como usuario, quiero poder repetir el proceso de transcripción o resumen si ocurre un error, para no perder la grabación.

## Consulta conversacional e historial

1. **Realizar preguntas sobre reuniones.** Como usuario, quiero hacer preguntas en lenguaje natural sobre una o varias reuniones, para obtener información puntual sin leer toda la transcripción.
2. **Respuestas con referencias.** Como usuario, quiero que el modelo me indique de qué reuniones ha obtenido la información, para verificar su precisión.
3. **Historial de la conversación.** Como usuario, quiero ver el historial de mis preguntas y respuestas dentro de una misma conversación, para no perder el hilo.
4. **Historial de conversaciones.** Como usuario, quiero visualizar el listado de mis conversaciones previas, para retomarlas cuando lo necesite.
5. **Reanudar conversación.** Como usuario, quiero reanudar una conversación anterior, para continuar preguntando sobre el mismo tema.
6. **Renombrar conversación.** Como usuario, quiero renombrar una conversación, para identificarla fácilmente.
7. **Eliminar conversación.** Como usuario, quiero eliminar una conversación, para mantener solo las que me son útiles.

## Autenticación

1. **Crear cuenta.** Como usuario no autenticado, quiero registrarme con correo electrónico y contraseña, para acceder a la aplicación.
2. **Iniciar sesión.** Como usuario no autenticado, quiero iniciar sesión con mis credenciales, para acceder a mis reuniones y conversaciones.

## Gestión de cuenta

1. **Modificar correo electrónico.** Como usuario, quiero cambiar mi correo electrónico asociado a la cuenta, para mantener mis datos actualizados.
2. **Modificar contraseña.** Como usuario, quiero cambiar mi contraseña, por seguridad.
3. **Eliminar cuenta.** Como usuario, quiero eliminar mi cuenta y todos mis datos asociados, para dejar de usar la aplicación.

## Configuración

1. **Añadir claves de API.** Como usuario, quiero añadir claves de API de diferentes proveedores, para poder utilizar sus modelos de lenguaje.
2. **Seleccionar modelo de lenguaje.** Como usuario, quiero seleccionar qué modelo de lenguaje utilizar entre los disponibles, para elegir el que mejor se adapte a mis necesidades.
3. **Ajustar hiperparámetros.** Como usuario, quiero ajustar la temperatura y el número máximo de tokens del modelo, para controlar el estilo de las respuestas.
4. **Personalizar plantilla de resumen.** Como usuario, quiero modificar la plantilla del resumen generado, para adaptar su estructura a mis preferencias.
5. **Restaurar plantilla por defecto.** Como usuario, quiero restaurar la plantilla del resumen a su valor por defecto, si no estoy satisfecho con los cambios.
6. **Preferencias de la aplicación.** Como usuario, quiero cambiar el tema visual, el idioma y el número de elementos por página, para adaptar la aplicación a mis preferencias.

## Orquestación e integración

1. **Encadenar los procesos de subida, transcripción y resumen.** Como desarrollador, quiero que una vez el usuario suba una grabación, se inicie automáticamente el flujo de procesamiento completo.
2. **Automatizar el envío de datos al LLM.** Como desarrollador, quiero que los resúmenes se almacenen en un formato consultable por el modelo, para poder usarlos en las preguntas.
3. **Registrar errores en los flujos de procesamiento.** Como desarrollador, quiero que el sistema detecte errores en cada paso del flujo y los notifique, para facilitar la depuración.
4. **Modularidad en los servicios.** Como desarrollador, quiero que cada componente (transcripción, resumen y consulta) pueda ejecutarse como un servicio independiente, para facilitar pruebas y mantenimiento.
5. **Intercambio de datos mediante APIs o archivos.** Como desarrollador, quiero que los módulos se comuniquen mediante APIs o intercambio de archivos, para garantizar la independencia entre los mismos.

**Apéndice B**

# **Requisitos del Sistema**

---

# Requisitos

## Requisitos Funcionales

### Gestión de reuniones

| ID    | Requisito Funcional                                                                                                                    |
|-------|----------------------------------------------------------------------------------------------------------------------------------------|
| RF-01 | El sistema debe permitir al usuario subir archivos de audio en formatos ampliamente utilizados como .wav, .mp3 o .mp4                  |
| RF-02 | El sistema debe validar el formato y tamaño del archivo antes de iniciar la subida                                                     |
| RF-03 | El sistema debe mostrar un mensaje de éxito o error tras la subida del archivo                                                         |
| RF-05 | El sistema debe permitir eliminar reuniones ya procesadas                                                                              |
| RF-06 | El sistema debe mostrar un listado de todas las reuniones que el usuario ha procesado, con metadatos básicos (título, fecha, duración) |
| RF-07 | El sistema debe permitir filtrar y ordenar el listado de reuniones por fecha o título                                                  |
| RF-08 | El sistema debe permitir al usuario seleccionar una reunión para ver su transcripción y el resumen                                     |
| RF-09 | El sistema debe permitir editar metadatos de la reunión (título, fecha)                                                                |

### Procesamiento de grabaciones

| ID    | Requisito Funcional                                                                                                    |
|-------|------------------------------------------------------------------------------------------------------------------------|
| RF-10 | El sistema debe iniciar automáticamente el proceso de transcripción una vez se haya subido una grabación correctamente |
| RF-11 | El sistema debe utilizar un modelo de transcripción para generar texto a partir del audio                              |
| RF-12 | El sistema debe generar un resumen automático de la transcripción utilizando un modelo de lenguaje y una plantilla     |
| RF-13 | El sistema debe permitir repetir el proceso de transcripción o resumen si ocurre un error o si el usuario lo solicita  |
| RF-14 | El sistema debe informar al usuario del estado del procesamiento en cada fase                                          |

### Consulta conversacional e historial

| ID    | Requisito Funcional                                                                                                        |
|-------|----------------------------------------------------------------------------------------------------------------------------|
| RF-15 | El sistema debe permitir al usuario introducir consultas en lenguaje natural sobre una o varias reuniones                  |
| RF-16 | El sistema debe proporcionar como contexto los resúmenes de las reuniones al modelo de lenguaje                            |
| RF-17 | El sistema debe devolver la respuesta generada por el modelo y mostrarla al usuario en la interfaz de chat                 |
| RF-18 | El sistema debe registrar las preguntas realizadas y sus respuestas                                                        |
| RF-19 | El sistema debe mostrar el historial de preguntas y respuestas realizadas sobre cada reunión, agrupadas por sesión de chat |

| ID    | Requisito Funcional                                                                 |
|-------|-------------------------------------------------------------------------------------|
| RF-20 | El sistema debe permitir al usuario visualizar el listado de conversaciones previas |
| RF-21 | El sistema debe permitir al usuario reanudar una conversación anterior              |
| RF-22 | El sistema debe permitir al usuario renombrar una conversación                      |
| RF-23 | El sistema debe permitir al usuario eliminar una conversación previa confirmación   |

## Autenticación y gestión de cuenta

| ID    | Requisito Funcional                                                                   |
|-------|---------------------------------------------------------------------------------------|
| RF-24 | El sistema debe permitir a un usuario registrarse con correo electrónico y contraseña |
| RF-25 | El sistema debe permitir a un usuario iniciar sesión con sus credenciales             |
| RF-26 | El sistema debe permitir al usuario modificar su correo electrónico                   |
| RF-27 | El sistema debe permitir al usuario modificar su contraseña                           |
| RF-28 | El sistema debe permitir al usuario eliminar su cuenta                                |

## Configuración

| ID    | Requisito Funcional                                                                                                   |
|-------|-----------------------------------------------------------------------------------------------------------------------|
| RF-29 | El sistema debe permitir al usuario añadir claves de API de diferentes proveedores de modelos de lenguaje             |
| RF-30 | El sistema debe permitir al usuario seleccionar el modelo de lenguaje a utilizar entre los disponibles                |
| RF-31 | El sistema debe permitir al usuario ajustar hiperparámetros del modelo (temperatura y número de tokens máximo)        |
| RF-32 | El sistema debe permitir al usuario personalizar la plantilla del resumen generado                                    |
| RF-33 | El sistema debe permitir restaurar la plantilla del resumen a su valor por defecto                                    |
| RF-34 | El sistema debe permitir al usuario cambiar el tema visual de la aplicación                                           |
| RF-35 | El sistema debe permitir al usuario cambiar el idioma de la aplicación                                                |
| RF-36 | El sistema debe permitir al usuario configurar el número de elementos mostrados por página en el listado de reuniones |

## Requisitos No Funcionales

| ID     | Requisito No Funcional                                                                                  |
|--------|---------------------------------------------------------------------------------------------------------|
| RNF-01 | El sistema debe ser modular y permitir cambiar fácilmente los modelos de transcripción o LLM utilizados |
| RNF-02 | El sistema debe garantizar la privacidad de las grabaciones y su contenido                              |
| RNF-03 | El sistema debe estar desarrollado con tecnologías open source                                          |



| ID     | Requisito No Funcional                                                                                                      |
|--------|-----------------------------------------------------------------------------------------------------------------------------|
| RNF-04 | La interfaz debe ser intuitiva y fácil de usar para personas no técnicas                                                    |
| RNF-05 | El sistema debe estar debidamente documentado                                                                               |
| RNF-06 | El sistema debe almacenar logs o registros de los pasos realizados para cada grabación                                      |
| RNF-07 | Las claves de API deben almacenarse cifradas en la base de datos                                                            |
| RNF-08 | El sistema debe cumplir con la normativa de protección de datos aplicable para la eliminación de cuentas y datos personales |
| RNF-09 | Las conexiones entre el usuario y el sistema deben de realizarse de manera segura                                           |
| RNF-10 | El proyecto desarrollado debe ser open source                                                                               |

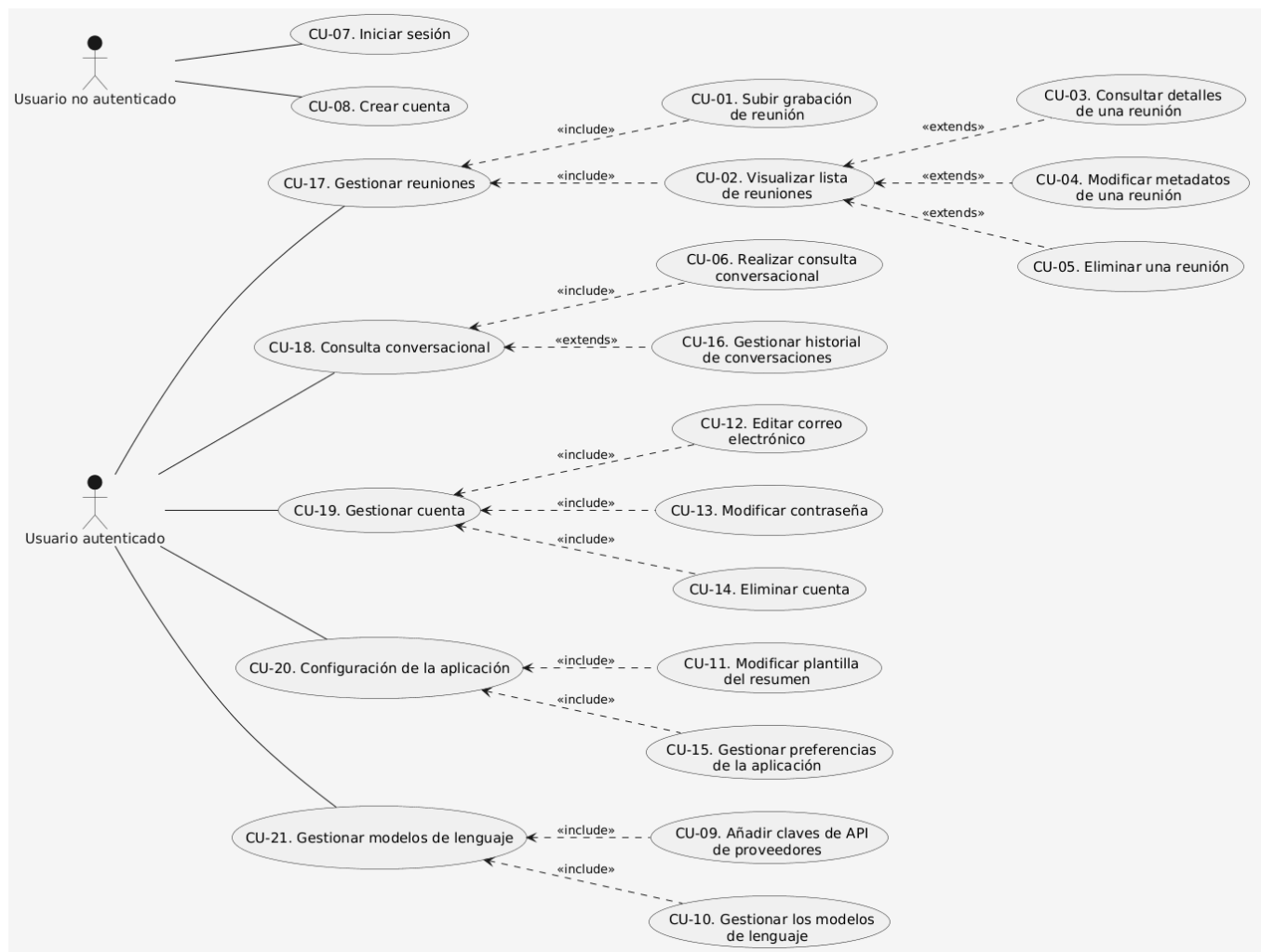
Apéndice C

## Casos de Uso

---

# Casos de Uso

## Diagrama de Casos de Uso



## Casos de uso del sistema

### Nivel 1 (resumen)

#### CU-17: Gestionar reuniones

**Descripción:** Agrupa las funcionalidades principales para incorporar, localizar y administrar reuniones procesadas.

**Precondición:** Usuario autenticado.

**Postcondición:** N/A.

#### CU-18: Consulta conversacional

**Descripción:** Permite interactuar con el contenido de las reuniones mediante consultas en lenguaje natural y mantener el contexto en conversaciones.

**Precondición:** Usuario autenticado.

**Postcondición:** N/A.

#### CU-19: Gestionar cuenta

**Descripción:** Gestiona los datos de acceso y la baja de la cuenta del usuario.

**Precondición:** Usuario autenticado.

**Postcondición:** N/A.

## **CU-20: Configuración de la aplicación**

**Descripción:** Permite personalizar aspectos de la aplicación que afectan a la experiencia de uso.

**Precondición:** Usuario autenticado.

**Postcondición:** N/A.

## **CU-21: Gestionar modelos de lenguaje**

**Descripción:** Habilita y configura los modelos de lenguaje disponibles en la aplicación.

**Precondición:** Usuario autenticado.

**Postcondición:** N/A.

## **Nivel 2 (detalle)**

### **CU-01: Subir grabación de reunión**

**Descripción:** Permite al usuario subir un archivo de audio para que el sistema lo procese automáticamente y almacene los datos extraídos.

**Precondición:** N/A.

#### **Secuencia normal:**

1. Usuario selecciona "Subir archivos".
2. Sistema muestra una ventana de selección.
3. Usuario selecciona los archivos que quiere subir y rellena los campos de metadatos (título y fecha).
4. Sistema valida los campos.
5. Sistema inicia la subida y muestra el progreso.
6. Sistema transcribe el contenido de la reunión.
7. Sistema genera un resumen estructurado del contenido de la reunión.

**Postcondición:** N/A.

#### **Excepciones:**

- 4a. *Formato/tamaño inválido:*
  - 4a1. Sistema muestra mensaje de error "Formato no válido" y vuelve al paso 2.

### **CU-02: Visualizar lista de reuniones**

**Descripción:** Permite al usuario ver el listado de reuniones procesadas, ordenarlas, filtrarlas y buscarlas.

**Precondición:** Existe al menos una reunión procesada.

#### **Secuencia normal:**

1. Usuario accede a la vista de "Mis reuniones".
2. Sistema muestra todas las reuniones con título, fecha y estado.
3. Opcionalmente, el usuario puede aplicar filtros o búsquedas.
4. Opcionalmente, el usuario puede editar los metadatos de una reunión. Ver CU-04.

5. Opcionalmente, el usuario puede consultar los detalles de una reunión. Ver *CU-03*.

**Postcondición:** N/A.

**Excepciones:**

- 3a. *Búsqueda sin resultados:*
  - 3a1. Sistema muestra el mensaje "No se han encontrado reuniones".

### **CU-03: Consultar detalles de una reunión**

**Descripción:** Permite al usuario visualizar la transcripción completa y el resumen estructurado de una reunión específica.

**Precondición:** La reunión seleccionada debe haber completado su procesamiento.

**Secuencia normal:**

1. Sistema muestra la transcripción/resumen de la reunión seleccionada.
2. Opcionalmente, el usuario puede intercambiar el documento que esté visualizando entre transcripción y resumen.

**Postcondición:** N/A.

**Excepciones:** N/A.

### **CU-04: Modificar metadatos de una reunión**

**Descripción:** Permite al usuario modificar información asociada a una reunión, como el título, etiquetas o fecha.

**Precondición:** N/A.

**Secuencia normal:**

1. Usuario modifica los campos deseados.
2. Usuario confirma los cambios.
3. Sistema valida los datos e implementa los cambios.

**Postcondición:** N/A.

**Excepciones:**

- 2a. *Usuario cancela la acción.*
  - 2a1. Sistema cierra el cuadro de confirmación y no realiza ningún cambio.
- 3a. *Error de validación:*
  - 3a1. Sistema muestra mensaje de error indicando qué campos no son válidos.
- 3b. *Error de actualización.*
  - 3b1. Sistema muestra mensaje "No se pudo actualizar la reunión".

### **CU-05: Eliminar una reunión**

**Descripción:** Permite al usuario eliminar una reunión procesada que ya no desea conservar.

**Precondición:** Existe al menos una reunión procesada.

**Secuencia normal:**

1. Usuario pulsa el botón "Eliminar" correspondiente a una reunión de la lista.

2. Sistema solicita la confirmación.
3. Usuario confirma la acción.
4. Sistema elimina la reunión y actualiza la lista.

**Postcondición:** N/A.

**Excepciones:**

- *3a. Usuario cancela la acción:*
  - 3a1. Sistema cierra el cuadro de confirmación y no realiza ningún cambio.
- *4a. Error al eliminar la reunión:*
  - 4a1. Sistema muestra el mensaje "No se pudo eliminar la reunión" y la conserva.

## **CU-06: Realizar consulta conversacional**

**Descripción:** Permite al usuario hacer preguntas en lenguaje natural sobre una o varias reuniones, recibiendo respuestas generadas por el modelo de lenguaje con referencias al contenido usado. La conversación queda almacenada en el historial para poder retomarla posteriormente.

**Precondición:** El usuario debe tener al menos una reunión procesada y una conversación activa.

**Secuencia normal:**

1. Usuario escribe la pregunta en la interfaz de chat.
2. Sistema envía la petición al LLM.
3. Sistema recibe una respuesta y la muestra.
4. Sistema registra la interacción en el historial.

**Postcondición:** N/A.

**Excepciones:** N/A.

## **CU-07: Iniciar sesión**

**Descripción:** Permite a un usuario no autenticado autenticarse.

**Precondición:** N/A.

**Secuencia normal:**

1. Usuario introduce sus credenciales.
2. Sistema valida las credenciales del usuario.

**Postcondición:** N/A.

**Excepciones:**

- *2a. Las credenciales no son correctas:*
  - 2a1. Sistema muestra "Credenciales incorrectas".

## **CU-08: Crear cuenta**

**Descripción:** Permite a un usuario no autenticado crear una cuenta para así poder autenticarse.

**Precondición:** N/A.

**Secuencia normal:**

1. Usuario introduce sus credenciales (correo electrónico y contraseña).

2. Sistema valida que los datos introducidos sean válidos.
3. Sistema comprueba que el correo electrónico introducido no esté asociado a otra cuenta.

**Postcondición:** N/A.

**Excepciones:**

- *2a. Las credenciales introducidas no son válidas:*
  - 2a1. Sistema muestra el error correspondiente (ej.: las contraseñas no coinciden, la contraseña no es segura) y vuelve al paso 1.
- *3a. El correo electrónico introducido ya está asociado a otra cuenta:*
  - 3a1. Sistema muestra mensaje de "el correo electrónico introducido ya está asociado a otro usuario" y vuelve al paso 1.

## **CU-09: Añadir claves de API de proveedores**

**Descripción:** Permite al usuario añadir claves de API de diferentes proveedores, de tal modo que pueda utilizar los modelos de lenguaje ofertados por estos proveedores.

**Precondición:** N/A.

**Secuencia normal:**

1. El usuario introduce la clave de API del proveedor.
2. El sistema valida la clave introducida.
3. El sistema cifra y almacena la clave.
4. El sistema registra el proveedor como disponible.

**Postcondición:** N/A.

**Excepciones:**

- *2a. La clave introducida no es válida:*
  - 2a1. El sistema muestra un mensaje indicando que la clave introducida no es válida.

## **CU-10: Gestionar los modelos de lenguaje**

**Descripción:** Permite al usuario modificar los modelos de lenguaje utilizados, así como valores de hiperparámetros (temperatura y número de tokens máximos de la respuesta).

**Precondición:** N/A.

**Secuencia normal:**

1. Sistema muestra los modelos de lenguaje disponibles, así como los valores que pueden tomar los hiperparámetros.
2. Usuario selecciona el modelo de lenguaje que desea utilizar, así como los valores que desea para los hiperparámetros.
3. Usuario presiona "guardar".
4. Sistema almacena los datos seleccionados por el usuario.

**Postcondición:** N/A.

**Excepciones:**

- *3a. Usuario presiona "cancelar":*
  - 3a1. Se aborta el CU.

## CU-11: Modificar la plantilla del resumen generado

**Descripción:** Permite al usuario modificar la plantilla del resumen generado automáticamente por un modelo de lenguaje.

**Precondición:** N/A.

**Secuencia normal:**

1. Usuario modifica el texto de la plantilla.
2. Usuario selecciona "guardar".
3. Sistema actualiza la plantilla.
4. Opcionalmente, el usuario puede seleccionar "restaurar plantilla por defecto" y volver así al paso 2.

**Postcondición:** N/A.

**Excepciones:**

- 2a. Usuario presiona "cancelar":
  - 2a1. Se aborta el CU.

## CU-12: Editar correo electrónico

**Descripción:** Permite al usuario modificar su correo electrónico.

**Precondición:** N/A.

**Secuencia normal:**

1. Usuario introduce su nuevo correo electrónico y su contraseña.
2. Sistema comprueba que la contraseña introducida sea correcta y que el nuevo correo no esté siendo utilizado por otro usuario.
3. Sistema aplica el cambio de correo electrónico.

**Postcondición:** N/A.

**Excepciones:**

- 2a. La contraseña introducida no es válida:
  - 2a1. Sistema muestra mensaje de error y se vuelve al paso 1.
- 2b. El nuevo correo introducido está siendo utilizado por otro usuario:
  - 2b1. Sistema muestra mensaje de error y se vuelve al paso 1.

## CU-13: Modificar contraseña

**Descripción:** Permite al usuario modificar su contraseña.

**Precondición:** N/A.

**Secuencia normal:**

1. Usuario introduce su contraseña actual y la nueva contraseña.
2. Sistema comprueba que la contraseña actual es correcta y que la nueva contraseña es válida.
3. Sistema aplica el cambio de contraseña.

**Postcondición:** N/A.

**Excepciones:**



- 2a. *La contraseña actual introducida no es correcta:*
  - 2a1. Sistema muestra mensaje de error y se vuelve al paso 1.
- 2b. *La nueva contraseña introducida no es válida:*
  - 2b1. Sistema muestra mensaje de error y se vuelve al paso 1.

## **CU-14: Eliminar cuenta**

**Descripción:** Permite al usuario eliminar su cuenta.

**Precondición:** N/A.

**Secuencia normal:**

1. Sistema muestra mensaje indicando que los datos se conservarán durante un número determinado de días antes de ser eliminados definitivamente, además de un correo electrónico de contacto al que el usuario puede escribir para pedir que se eliminen antes o que estos le sean transferidos.
2. Usuario introduce su contraseña.
3. Sistema comprueba la contraseña, marca la cuenta como eliminada y programa la eliminación de la cuenta para la fecha correspondiente.

**Postcondición:** N/A.

**Excepciones:**

- 3a. *La contraseña introducida no es correcta:*
  - 3a1. El sistema muestra mensaje de error y se vuelve al paso 2.

## **CU-15: Gestionar preferencias de la aplicación**

**Descripción:** Permite al usuario personalizar el tema visual (claro/oscuro/sistema), el idioma y el número de elementos mostrados por página en la vista de reuniones.

**Precondición:** N/A.

**Secuencia normal:**

1. Usuario accede a la sección de preferencias.
2. Sistema muestra las opciones disponibles (tema, idioma, elementos por página).
3. Usuario modifica los ajustes deseados.
4. Sistema aplica y persiste las preferencias.

**Postcondición:** N/A.

## **CU-16: Gestionar historial de conversaciones**

**Descripción:** Permite al usuario visualizar sus conversaciones previas, reanudarlas, renombrarlas o eliminarlas.

**Precondición:** Existe al menos una conversación en el historial.

**Secuencia normal:**

1. Usuario accede a la sección de conversaciones.
2. Sistema muestra el listado de conversaciones previas con título y fecha.
3. Opcionalmente, el usuario selecciona una conversación para reanudarla. Ver CU-06.
4. Opcionalmente, el usuario renombra una conversación.

5. Opcionalmente, el usuario elimina una conversación.

**Postcondición:** N/A.

**Excepciones:**

- 5a. *Usuario cancela la eliminación:*
  - 5a1. Sistema cierra el cuadro de confirmación y no realiza ningún cambio.
- 5b. *Error al eliminar la conversación:*
  - 5b1. Sistema muestra mensaje "No se pudo eliminar la conversación" y la conserva.

Apéndice D

# Mockups

---

Bienvenido a la aplicación

Iniciar sesión

Registrarse

1 - Página inicial (no autenticado)

Iniciar sesión

Correo electrónico

xxxxxx@xxx.xx

Contraseña

xxxxxxxxxxxx

Iniciar sesión

2 - Vista de iniciar sesión

Crear cuenta

Correo electrónico

xxxxxx@xxx.xx

Contraseña

xxxxxxxxxxxx

Confirmar contraseña

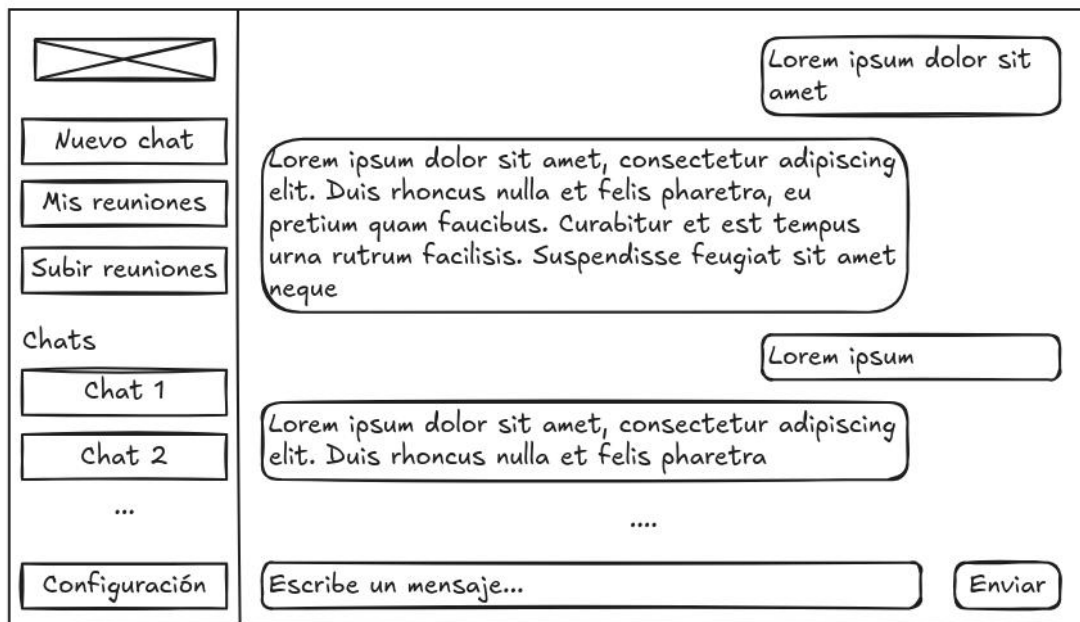
xxxxxxxxxxxx

Crear cuenta

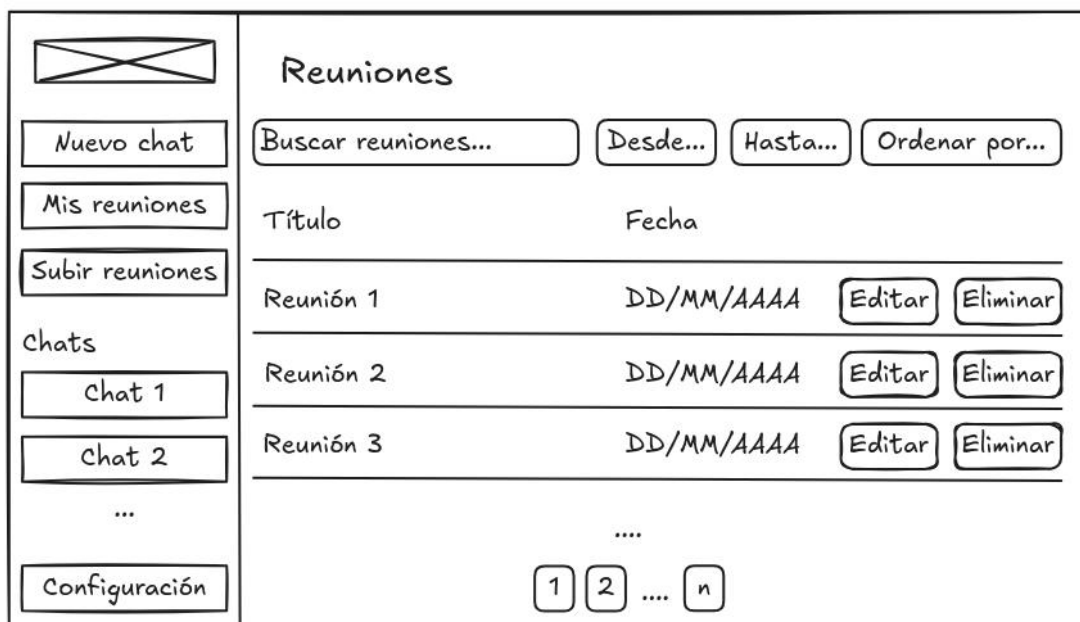
3 - Vista de registro



#### 4 - Página inicial (autenticado)



#### 5 - Vista de Chat



#### 6 - Vista de reuniones

Nuevo chat

Mis reuniones

Subir reuniones

Chats

Chat 1

Chat 2

...

Configuración

Título de la reunión - DD/MM/AAAA

Resumen

Ver transcripción

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis rhoncus nulla et felis pharetra, eu pretium quam faucibus. Curabitur et est tempus urna rutrum facilisis. Suspendisse feugiat sit amet neque at sagittis.

Etiam quis velit ac turpis tincidunt dignissim tincidunt vitae turpis. Pellentesque dui purus, malesuada ac dictum eget, scelerisque vel let inte

Pellentesque id dui euismod, faucibus metus id, lobortis metus. Nulla tincidunt rhoncus metus, sed elementum nunc pellentesque et.

## 7 - Vista del resumen de una reunión

Nuevo chat

Mis reuniones

Subir reuniones

Chats

Chat 1

Chat 2

...

Configuración

Título de la reunión - DD/MM/AAAA

Transcripción

Ver resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis rhoncus nulla et felis pharetra, eu pretium quam faucibus. Curabitur et est tempus urna rutrum facilisis. Suspendisse feugiat sit amet neque at sagittis.

Etiam quis velit ac turpis tincidunt dignissim tincidunt vitae turpis. Pellentesque dui purus, malesuada ac dictum eget, scelerisque vel let inte

Pellentesque id dui euismod, faucibus metus id, lobortis metus. Nulla tincidunt rhoncus metus, sed elementum nunc pellentesque et.

## 8 - Vista de la transcripción de una reunión

Nue

Mis

Subir

Chats

C

C

Config

Subir reuniones

Cancelar

Reunión 1

Título

Idioma

Fecha

Número de hablantes

Descartar

Seleccionar archivo

Reunión 2

Título

Idioma

Fecha

Número de hablantes

Descartar

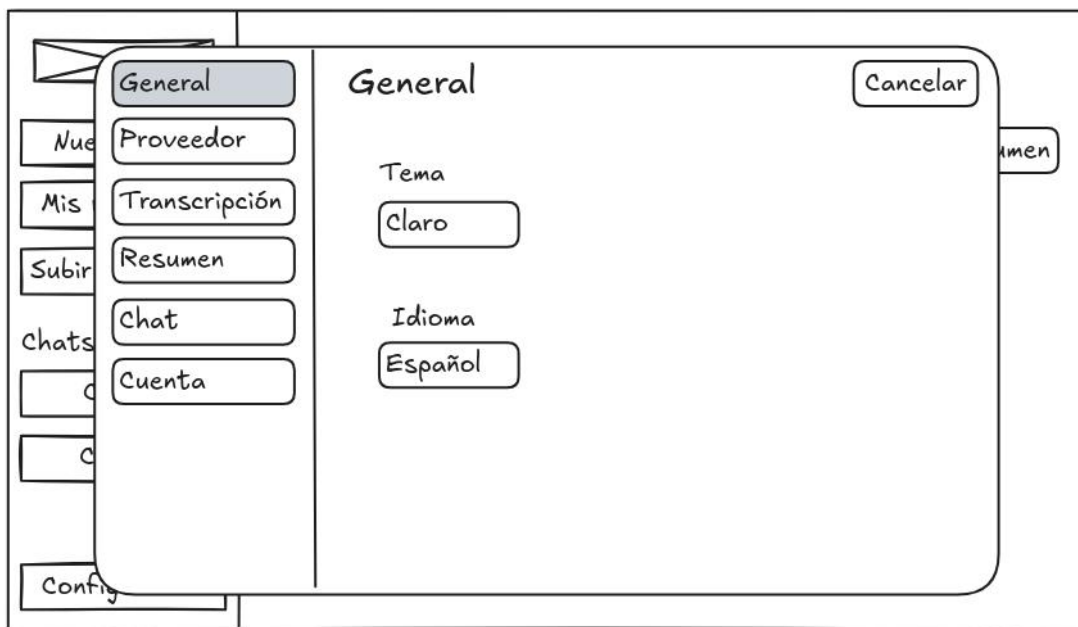
Seleccionar archivo

...

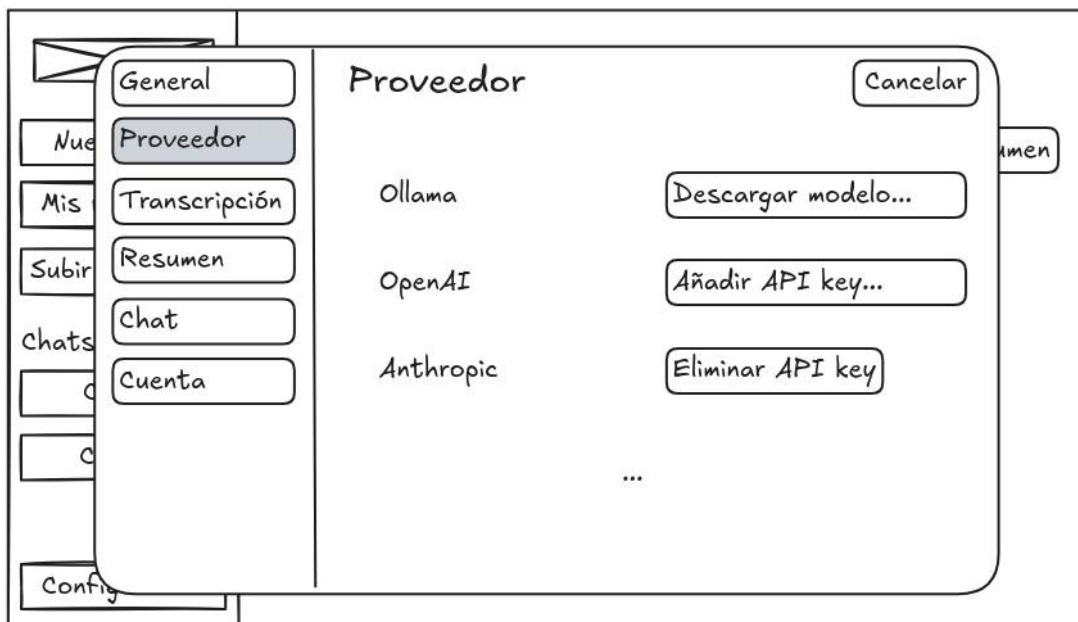
Añadir reunión

Subir reuniones

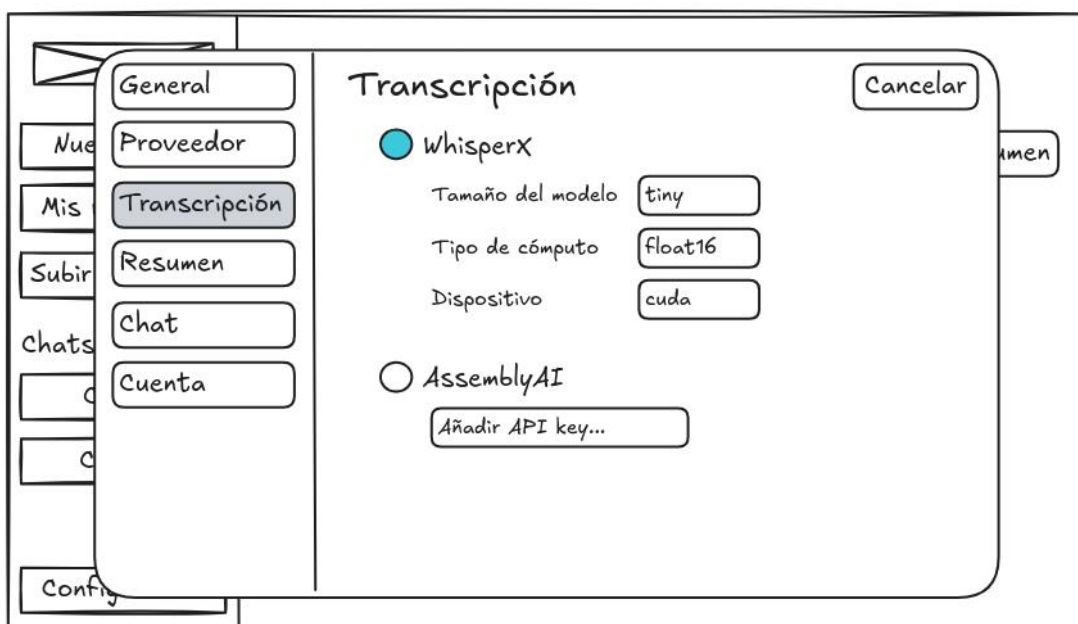
## 9 - Vista de subir reuniones



10.1 - Vista de configuración (general)



10.2 - Vista de configuración (proveedor)



10.3 - Vista de configuración (transcripción)

General

Proveedor

Transcripción

Resumen

Chat

Cuenta

### Resumen

Plantilla

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis rhoncus nulla et felis pharetra, eu pretium quam faucibus. Curabitur et est tempus urna rutrum facilisis. Suspendisse feugiat sit amet

Modelo de lenguaje

Modelo: Llama3.2

Temperatura: 0.4

Tokens máximos: Llama3.2

Cancelar

10.4 - Vista de configuración (resumen)

General

Proveedor

Transcripción

Resumen

Chat

Cuenta

### Chat

Modelo de lenguaje

Modelo: Llama3.2

Temperatura: 0.4

Tokens máximos: Llama3.2

Cancelar

10.5 - Vista de configuración (chat)

General

Proveedor

Transcripción

Resumen

Chat

Cuenta

### Cuenta

Correo: xxxx@xxx.xx

Modificar correo electrónico

Cambiar contraseña

Cerrar sesión

Eliminar cuenta

Cancelar

10.6 - Vista de configuración (chat)



## Vistas de la interfaz de usuario

---

A continuación se muestran todas las vistas de la interfaz de usuario resultante del proceso de desarrollo iterativo visto a lo largo del Capítulo 5.

# Welcome to Memento

Your intelligent meeting assistant

Log in

Sign up

# Welcome to Memento

Your intelligent meeting assistant

Log in

Sign up

### Sign in to Memento

email

some@email.com

password

\*\*\*\*\*

Sign in

New to Memento? [Create an account](#)

### Sign in to Memento

email

some@email.com

password

\*\*\*\*\*

Sign in

New to Memento? [Create an account](#)

## Sign up to Memento

email

password

confirm password

Sign up

Already have an account? [Log in](#)

## Sign up to Memento

email

password

confirm password

Sign up

Already have an account? [Log in](#)

# Welcome to Memento

Your intelligent meeting assistant

# Welcome to Memento

Your intelligent meeting assistant

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

How can I help you?

Some message...



MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

How can I help you?

Some message...



MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

De qué trató la reunión de hoy?

No hay información disponible sobre las notas de la reunión de hoy. Por favor, revise los registros de la reunión o proporcione más detalles si necesita ayuda con algo específico.

de qué fecha es la reunión más reciente?

La reunión más reciente en las notas es "Reunión con el cliente" con fecha 2026-06-04.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

De qué trató la reunión de hoy?

No hay información disponible sobre las notas de la reunión de hoy. Por favor, revise los registros de la reunión o proporcione más detalles si necesita ayuda con algo específico.

de qué fecha es la reunión más reciente?

La reunión más reciente en las notas es "Reunión con el cliente" con fecha 2026-06-04.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

# Meetings

Search meetings...

Filter by date

Sort by  
Newest first

| #  | TITLE                      | DATE       |  |  |
|----|----------------------------|------------|--|--|
| 1  | Reunión con el cliente     | 04/06/2026 |  |  |
| 2  | Daily Standup              | 02/06/2026 |  |  |
| 3  | Entrevista técnica         | 21/05/2026 |  |  |
| 4  | Reunión con el cliente     | 21/05/2026 |  |  |
| 5  | Diseño de la nueva API     | 19/05/2026 |  |  |
| 6  | Reunión con el cliente     | 14/05/2026 |  |  |
| 7  | Reunión con el cliente     | 07/05/2026 |  |  |
| 8  | Retrospectiva del Sprint 3 | 07/05/2026 |  |  |
| 9  | Daily Standup              | 16/03/2026 |  |  |
| 10 | Daily Standup              | 13/03/2026 |  |  |

Prev

1

2

Next

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

# Meetings

Search meetings...

Filter by date

Sort by  
Newest first

| #  | TITLE                      | DATE       |  |  |
|----|----------------------------|------------|--|--|
| 1  | Reunión con el cliente     | 04/06/2026 |  |  |
| 2  | Daily Standup              | 02/06/2026 |  |  |
| 3  | Entrevista técnica         | 21/05/2026 |  |  |
| 4  | Reunión con el cliente     | 21/05/2026 |  |  |
| 5  | Diseño de la nueva API     | 19/05/2026 |  |  |
| 6  | Reunión con el cliente     | 14/05/2026 |  |  |
| 7  | Reunión con el cliente     | 07/05/2026 |  |  |
| 8  | Retrospectiva del Sprint 3 | 07/05/2026 |  |  |
| 9  | Daily Standup              | 16/03/2026 |  |  |
| 10 | Daily Standup              | 13/03/2026 |  |  |

Prev

1

2

Next



MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

Reunión con el cliente - 04/06/2026

Meeting Overview

- **Meeting Date and Duration:** No specified
- **Participants:** Speaker A, Speaker B (no other participants mentioned)
- **Main Objectives Discussed:** Not explicitly stated in the provided transcript. The conversation appears to be a casual discussion between two individuals sharing their thoughts on sports and exercise.

Key Decisions

- None mentioned in the provided transcript.
- No deadlines or timelines established.
- No budgets or resources allocated.

Action Items

- None specified in the provided transcript.

Discussion Topics

- **Sports and Exercise:** Speaker A and Speaker B discuss their opinions on various sports, including football, Pilates, and racing (motos and Fórmula 1). They share their personal experiences and preferences.
  - **Challenges/Risks:** No specific challenges or risks identified in the conversation.
  - **Unresolved Questions:** None mentioned.

Next Steps

- **Upcoming Milestones:** No specified milestones mentioned in the provided transcript.
- **Scheduled Follow-up Meetings:** No follow-up meetings scheduled.
- **Required Preparations for Next Discussion:** No preparations required, as this appears to be a casual conversation.

Summary

Transcription

Reunión con el cliente - 04/06/2026

Meeting Overview

- **Meeting Date and Duration:** No specified
- **Participants:** Speaker A, Speaker B (no other participants mentioned)
- **Main Objectives Discussed:** Not explicitly stated in the provided transcript. The conversation appears to be a casual discussion between two individuals sharing their thoughts on sports and exercise.

Key Decisions

- None mentioned in the provided transcript.
- No deadlines or timelines established.
- No budgets or resources allocated.

Action Items

- None specified in the provided transcript.

Discussion Topics

- **Sports and Exercise:** Speaker A and Speaker B discuss their opinions on various sports, including football, Pilates, and racing (motos and Fórmula 1). They share their personal experiences and preferences.
  - **Challenges/Risks:** No specific challenges or risks identified in the conversation.
  - **Unresolved Questions:** None mentioned.

Next Steps

- **Upcoming Milestones:** No specified milestones mentioned in the provided transcript.
- **Scheduled Follow-up Meetings:** No follow-up meetings scheduled.
- **Required Preparations for Next Discussion:** No preparations required, as this appears to be a casual conversation.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

Upload Meetings

| # | STATUS | TITLE                              | DATE                                    | FILE                                |
|---|--------|------------------------------------|-----------------------------------------|-------------------------------------|
| 1 | -      | <input type="text" value="Title"/> | <input type="text" value="14/06/2026"/> | <input type="text" value="Browse"/> |

+ Add meeting

Submit

Scheduled Follow-up Meetings:

No follow-up meetings scheduled.

Required Preparations for Next Discussion:

No preparations required, as this appears to be a casual conversation.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

Upload Meetings

| # | STATUS | TITLE                              | DATE                                    | FILE                                |
|---|--------|------------------------------------|-----------------------------------------|-------------------------------------|
| 1 | -      | <input type="text" value="Title"/> | <input type="text" value="14/06/2026"/> | <input type="text" value="Browse"/> |

+ Add meeting

Submit

Scheduled Follow-up Meetings:

No follow-up meetings scheduled.

Required Preparations for Next Discussion:

No preparations required, as this appears to be a casual conversation.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

General

Theme

Light

Language

English

Meetings per page

10

Scheduled Follow-up Meetings:

No follow-up meetings scheduled.

Required Preparations for Next Discussion:

No preparations required, as this appears to be a casual conversation.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

General

Theme

Dark

Language

English

Meetings per page

10

Scheduled Follow-up Meetings:

No follow-up meetings scheduled.

Required Preparations for Next Discussion:

No preparations required, as this appears to be a casual conversation.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

Provider

Ollama

OpenAI

Anthropic

Pull model

API key added

Remove API key

Add API key

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

Provider

Ollama

OpenAI

Anthropic

Pull model

API key added

Remove API key

Add API key

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

WhisperX

Model size

tiny

Compute type

int8

Device

cuda

If the GPU is unavailable or fails, transcription will automatically fall back to CPU using int8 precision

AssemblyAI

API key added

Remove API key

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

WhisperX

Model size

tiny

Compute type

int8

Device

cuda

If the GPU is unavailable or fails, transcription will automatically fall back to CPU using int8 precision

AssemblyAI

API key added

Remove API key

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

Summarization

Template

Analyze this meeting transcript and provide a structured summary with the following:

1. Meeting Overview

- Meeting date and duration

- List of participants (if mentioned)

- Main objectives discussed

2. Key Decisions

- Document all final decisions made

- Include any deadlines or timelines established

- Note any budgets or resources allocated

3. Action Items

- List each action item with:

1177 / 5000

Reset to default

Cancel

Save

Language Model

Model

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

Summarization

Template

Analyze this meeting transcript and provide a structured summary with the following:

1. Meeting Overview

- Meeting date and duration

- List of participants (if mentioned)

- Main objectives discussed

2. Key Decisions

- Document all final decisions made

- Include any deadlines or timelines established

- Note any budgets or resources allocated

3. Action Items

- List each action item with:

1177 / 5000

Reset to default

Cancel

Save

Language Model

Model

Scheduled Follow-up Meetings:

No follow-up meetings scheduled.

Required Preparations for Next Discussion:

No preparations required, as this appears to be a casual conversation.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

Chat

Chat Model

Model

OpenAI — gpt-5-mini

Temperature

0,7

Max tokens

2000

Cancel

Save

Retrieval Model

Model

OpenAI — gpt-5-mini

Temperature

0

Max tokens

Scheduled Follow-up Meetings:

No follow-up meetings scheduled.

Required Preparations for Next Discussion:

No preparations required, as this appears to be a casual conversation.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

Chat

Chat Model

Model

OpenAI — gpt-5-mini

Temperature

0,7

Max tokens

2000

Cancel

Save

Retrieval Model

Model

OpenAI — gpt-5-mini

Temperature

0

Max tokens

Scheduled Follow-up Meetings:

No follow-up meetings scheduled.

Required Preparations for Next Discussion:

No preparations required, as this appears to be a casual conversation.



MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

Account

Email: esteban.suarez.calvo@udc.es

Edit email

Change password

Log out

Delete account

Scheduled Follow-up Meetings:

No follow-up meetings scheduled.

Required Preparations for Next Discussion:

No preparations required, as this appears to be a casual conversation.

MEMENTO

New Chat

My Meetings

Upload Meetings

Chats

Qué se trató?

Hola, ¿Cómo estás?

Resumen reunión de hoy

Settings

Summary

Transcription

General

Provider

Transcription

Summarization

Chat

Account

Account

Email: esteban.suarez.calvo@udc.es

Edit email

Change password

Log out

Delete account

Scheduled Follow-up Meetings:

No follow-up meetings scheduled.

Required Preparations for Next Discussion:

No preparations required, as this appears to be a casual conversation.

**Apéndice F**

# **Guía de instalación**

---

# Guía de Instalación

Esta guía describe paso a paso cómo instalar y desplegar Memento, desde la configuración inicial del sistema hasta la puesta en marcha con dominio público y certificado SSL.

## Índice

- 1. Requisitos del sistema
  - 1.1 Hardware
  - 1.2 Software
- 2. Instalación de prerequisites
  - 2.1 Docker
  - 2.2 NVIDIA Container Toolkit
  - 2.3 Verificar instalación
- 3. Configuración del proyecto
  - 3.1 Clonar el repositorio
  - 3.2 Archivos de entorno
- 4. Despliegue
  - Verificar estado
- 5. Acceso a la aplicación
  - Primer uso
- 6. Dominio y acceso seguro (DuckDNS + Nginx Proxy Manager)
  - 6.1 Obtener un subdominio en DuckDNS
  - 6.2 Configurar el dominio en el proyecto
  - 6.3 Crear el certificado SSL
  - 6.4 Crear el Proxy Host
  - 6.5 Verificar el acceso HTTPS
  - 6.6 NAT loopback (acceso desde la misma red)
- 7. Configuración de APIs externas (opcional)
- 8. Comandos útiles

## 1. Requisitos del sistema

### 1.1 Hardware

Los requisitos de hardware mínimos son los siguientes:

- **GPU NVIDIA:** 4 GB de VRAM mínimo (para usar Ollama local).
- **RAM:** 16 GB.
- **Almacenamiento:** ~30 GB disponibles (imágenes Docker + modelos).

Si tu GPU tiene menos de 4 GB de VRAM, puedes usar APIs externas (OpenAI o Anthropic) en lugar de Ollama local. Esta alternativa se explica en la sección 7.

## 1.2 Software

- **Sistema Operativo:** Ubuntu 25.10.
- **CUDA:** 12.8.
- **Docker:** 29.1.5 o superior.
- **Docker Compose:** 2.0 o superior.
- **NVIDIA Container Toolkit.**

## 2. Instalación de prerequisites

### 2.1 Docker

```
sudo apt-get update
sudo apt-get install -y docker.io docker-compose-v2
sudo usermod -aG docker $USER
newgrp docker
```

### 2.2 NVIDIA Container Toolkit

Permite a los contenedores Docker acceder a la GPU NVIDIA.

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list \
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' \
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctl runtime configure --runtime=docker
sudo systemctl restart docker
```

### 2.3 Verificar instalación

```
nvidia-smi
docker run --rm --runtime=nvidia --gpus all nvidia/cuda:12.8.0-base-ubuntu22.04 nvidia-smi
```

Si ambos comandos muestran información de la GPU, la instalación es correcta.

## 3. Configuración del proyecto

### 3.1 Clonar el repositorio

```
git clone git@github.com:estebansuarezcalvoudc/Memento.git
cd memento
```

### 3.2 Archivos de entorno

El proyecto usa dos archivos `.env`.

**.env (raíz del proyecto)**

Crea un fichero denominado `.env` en la raíz del proyecto:

```
touch .env
```

Edítalo dando valores a las siguientes variables:

```
FRONTEND_PORT=5173

MONGO_HOST=mongo_db
MONGO_USER=mongo
MONGO_PASSWORD=mongo
MONGO_PORT=27017

OLLAMA_HOST=ollama
OLLAMA_PORT=11434

CHROMA_HOST=chromadb
CHROMA_PORT=8000
RAG_EMBEDDING_MODEL=nomic-embed-text
RAG_COLLECTION_NAME=meetings

DOMAIN=localhost
```

Cambia `MONGO_USER` y `MONGO_PASSWORD` en producción.

### Backend/.env

Crea el archivo:

```
touch Backend/.env
```

Edítalo con las siguientes variables:

```
HF_TOKEN=tu_huggingface_token

ENCRYPTION_KEY="genera_clave_aleatoria_base64"
SECRET_KEY="genera_clave_hex_64_caracteres"
ALGORITHM="HS256"
ACCESS_TOKEN_EXPIRE_MINUTES=1800000

ACCOUNT_DELETION_GRACE_DAYS=30
PRIVACY_CONTACT_EMAIL=privacidad@tu-dominio.com
ACCOUNT_PURGE_JOB_INTERVAL_SECONDS=3600
ACCOUNT_PURGE_JOB_ENABLED=true

OPENAI_KEY=sk-placeholder

MONGO_USER=mongo
MONGO_HOST=mongo_db
MONGO_PASSWORD=mongo
MONGO_PORT=27017
CHROMA_HOST=chromadb
CHROMA_PORT=8000
RAG_EMBEDDING_MODEL=nomic-embed-text
RAG_COLLECTION_NAME=meetings
```

Genera las claves de seguridad ejecutando cada comando por separado y copia su salida en la variable correspondiente.

**ENCRYPTION\_KEY:** se usa para cifrar datos sensibles (API keys de usuario) con Fernet. Debe ser una clave de 32 bytes en base64 url-safe.

```
python3 -c "import secrets, base64; print(base64.urlsafe_b64encode(secrets.token_bytes(32)).decode())"
```

**SECRET\_KEY:** se usa para firmar tokens JWT de autenticación. Debe ser una cadena hexadecimal de 64 caracteres (32 bytes).

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

En cuanto al API key de Hugging Face, puedes obtenerlo en [este enlace](#).

## 4. Despliegue

Construir las imágenes e iniciar los servicios por primera vez:

```
docker compose up -d --build
```

En ejecuciones posteriores:

```
docker compose up -d
```

`--build` solo es necesario cuando se modifican los `Dockerfile` o los archivos de dependencias (`requirements.txt`, `package.json`). El código fuente se sincroniza automáticamente gracias a los bind mounts, por lo que no requiere reconstruir las imágenes.

Al iniciar, el servicio de Ollama descarga automáticamente los modelos por defecto:

- `llama3.2:latest`: modelo de lenguaje seleccionado por defecto para todos los usuarios.
- `nomie-embed-text`: modelo de embeddings para RAG.

La primera ejecución puede tardar varios minutos mientras se descargan las imágenes y los modelos.

### Verificar estado

```
docker compose ps
```

Todos los servicios deben mostrar estado `Up` o `healthy`. Si un servicio no arranca, revisa los logs:

```
docker compose logs -f <nombre-servicio>
```

## 5. Acceso a la aplicación

| Servicio            | URL                                |
|---------------------|------------------------------------|
| Frontend            | <code>http://localhost:5173</code> |
| Nginx Proxy Manager | <code>http://localhost:81</code>   |

### Primer uso

1. Abre `http://localhost:5173`.
2. Registra una cuenta de usuario.
3. Configura los proveedores de IA en Ajustes.
4. Comienza a usar Memento.

## 6. Dominio y acceso seguro (DuckDNS + Nginx Proxy Manager)

Para acceder a Memento desde internet con HTTPS necesitas un dominio público. DuckDNS ofrece subdominios gratuitos, y Nginx Proxy Manager (incluido en el `docker-compose`) gestiona el proxy inverso y los certificados SSL.

Si utilizas Nginx Proxy Manager como entrada única a la aplicación, es recomendable eliminar el mapeo del puerto del frontend (`ports: - "5173:5173"`) en el `docker-compose.yaml`, de modo que solo Nginx quede expuesto al host y el resto de servicios se comuniquen exclusivamente por las redes internas de Docker.

### 6.1 Obtener un subdominio en DuckDNS

1. Regístrate en [duckdns.org](https://duckdns.org).

2. Crea un subdominio (ej. `midominio.duckdns.org`) y haz clic en **add domain**.

## 6.2 Configurar el dominio en el proyecto

Edita `.env` en la raíz del proyecto y cambia el valor de `DOMAIN` :

```
DOMAIN=midominio.duckdns.org
```

Reconstruye el frontend para que coja la nueva variable:

```
docker compose up -d --build frontend
```

## 6.3 Crear el certificado SSL

1. Accede a Nginx Proxy Manager en <http://localhost:81>.
2. Añade tus credenciales.
3. Ve a **Certificates > Add SSL Certificate > Let's Encrypt via DNS**.

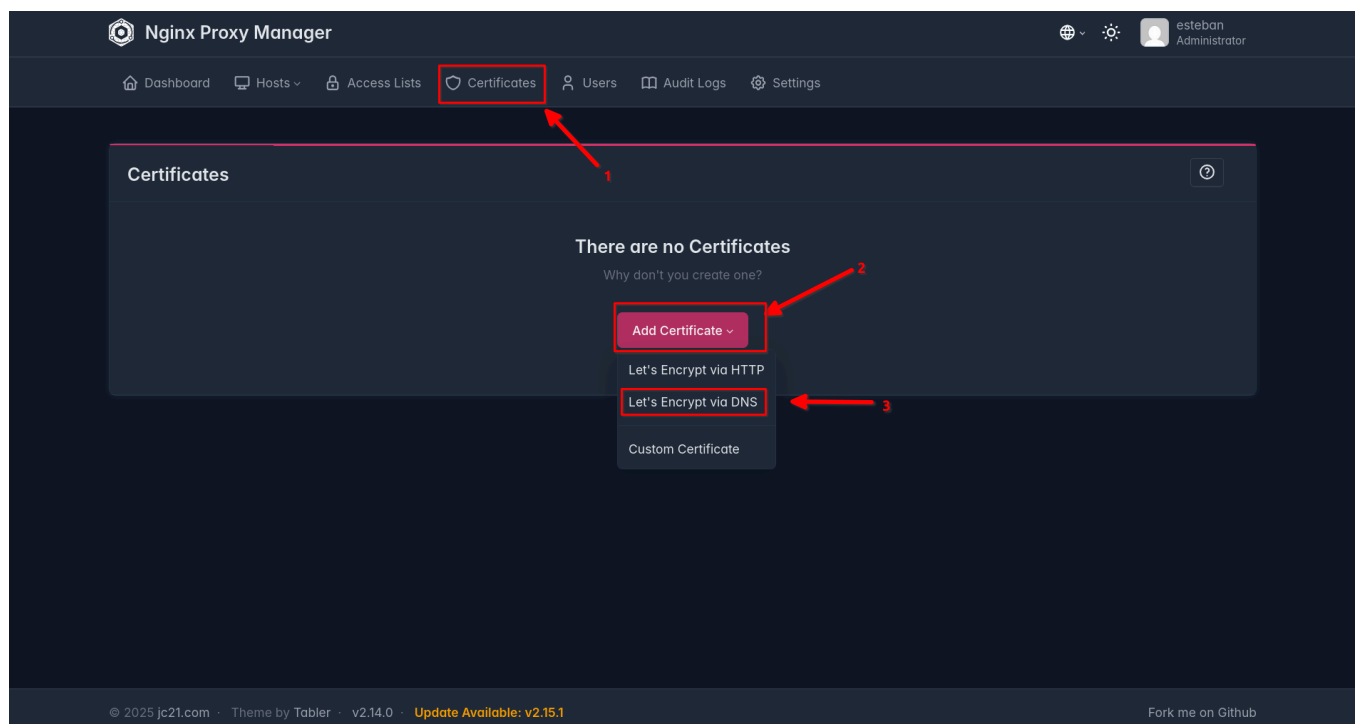


Figura 1: Selección del método de emisión de certificados.

4. Rellena los campos, sustituyendo el texto `your-duckdns-token` por tu token de DuckDNS, que puedes obtener en [este enlace](#):

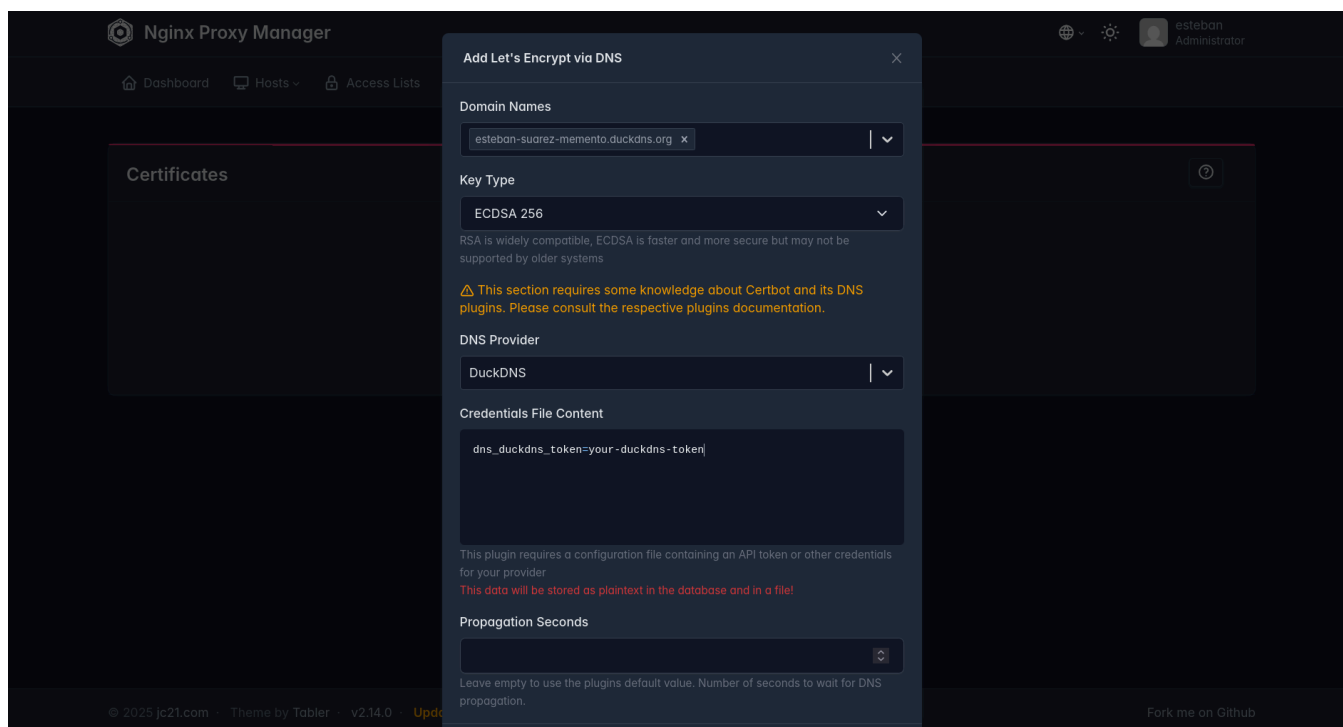


Figura 2: Formulario de solicitud del certificado.

5. Haz clic en **Save**. El certificado se generará en unos segundos.

## 6.4 Crear el Proxy Host

1. Ve a **Hosts > Proxy Hosts** y haz clic en **Add Proxy Host**.

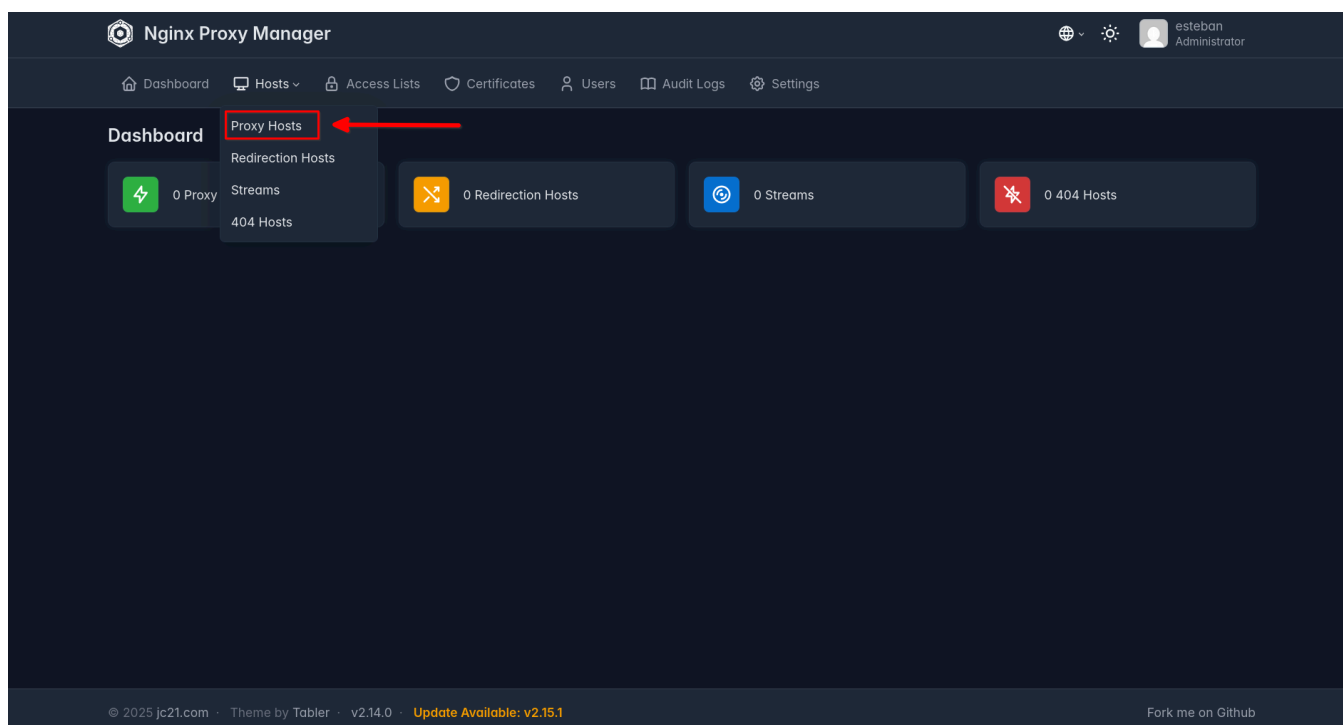


Figura 3: Acceso a la creación de un proxy host.



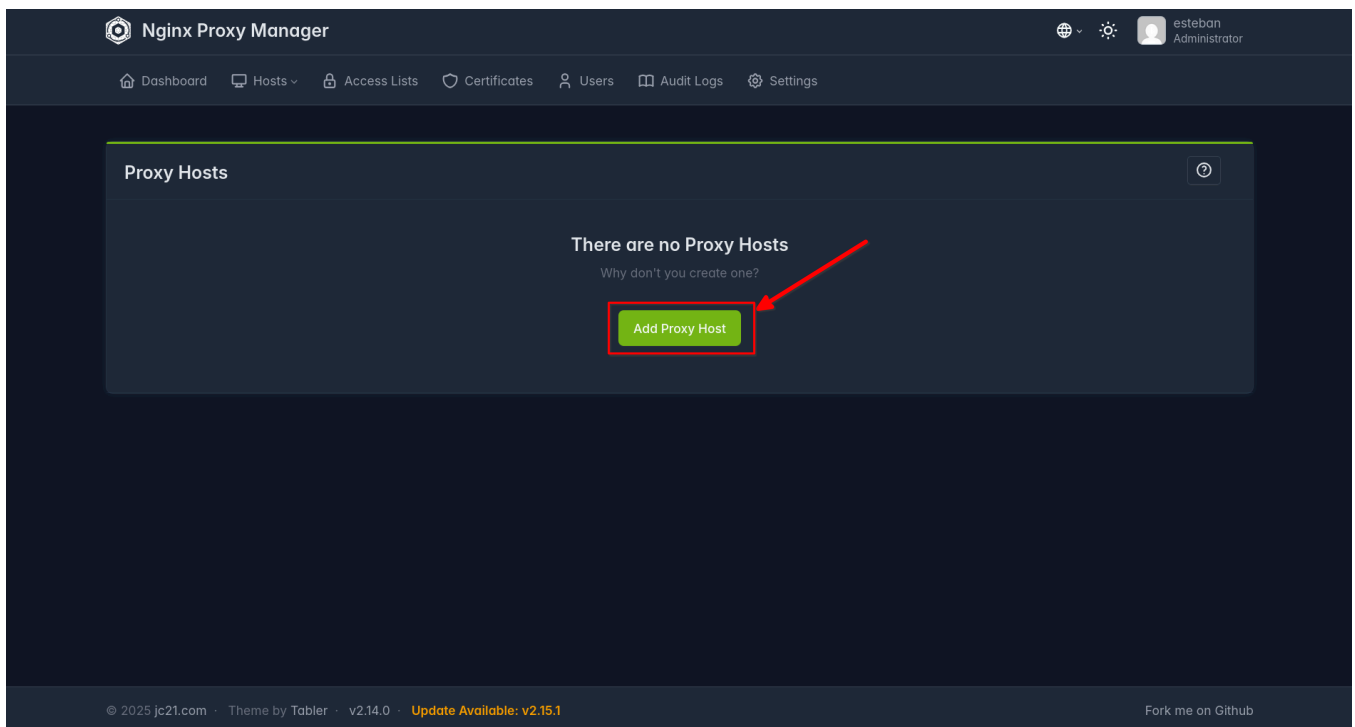


Figura 4: Formulario de creación del proxy host.

2. Rellena los detalles del proxy:

- **Domain Names:** midominio.duckdns.org
- **Scheme:** http
- **Forward Hostname / IP:** frontend
- **Forward Port:** 5173
- **Cache Assets:** Activado
- **Block Common Exploits:** Activado

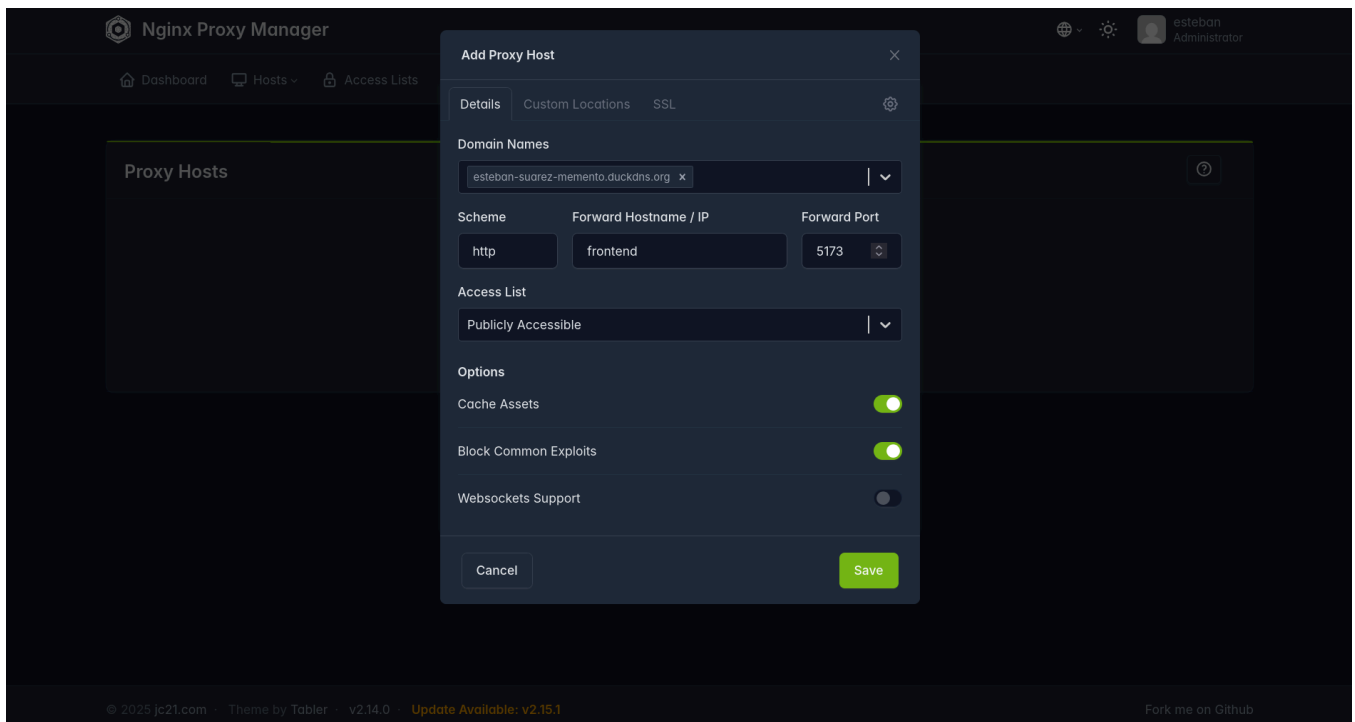


Figura 5: Detalles del proxy host.

3. Ve a la pestaña **SSL** y selecciona el certificado creado anteriormente, activando las opciones que se ven en la imagen:

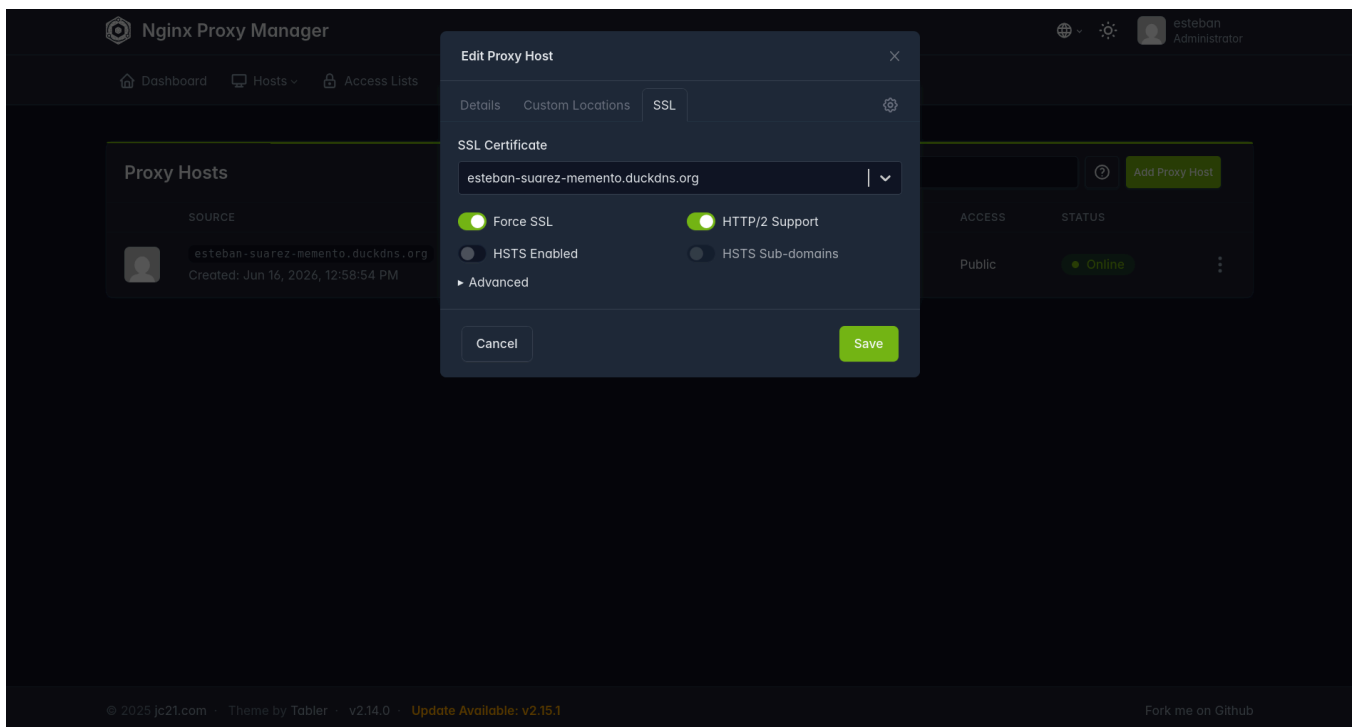


Figura 6: Configuración del certificado SSL en el proxy host.

4. Haz clic en **Save**.

## 6.5 Verificar el acceso HTTPS

Abre <https://midominio.duckdns.org> en un navegador. Deberías ver Memento con el candado verde. El certificado se renovará automáticamente mientras Nginx Proxy Manager esté corriendo.

## 6.6 NAT loopback (acceso desde la misma red)

Si accedes desde la misma red local donde está alojado el servidor, es posible que el dominio no resuelva correctamente y veas la página de bienvenida de NPM. Para evitarlo, añade una entrada en `/etc/hosts`:

```
echo "127.0.0.1 midominio.duckdns.org" | sudo tee -a /etc/hosts
```

Esto fuerza a tu máquina a resolver el dominio localmente. El resto de dispositivos de la red accederán sin problemas por su IP pública.

## 7. Configuración de APIs externas (opcional)

Si tu GPU no puede ejecutar modelos locales con el rendimiento deseado o simplemente deseas utilizar otros proveedores, puedes utilizar los modelos de OpenAI o Anthropic añadiendo la clave API correspondiente a través de la interfaz gráfica.

## 8. Comandos útiles

```
# Ver todos los logs en tiempo real
docker compose logs -f

# Logs de un servicio específico
docker compose logs -f backend

# Reiniciar un servicio
docker compose restart backend

# Detener todos los servicios
docker compose down

# Detener y eliminar volúmenes (borra datos)
docker compose down -v

# Reconstruir contenedores tras cambios en el docker-compose.yml
```

```
docker compose up -d --build
```

```
# Listar modelos disponibles en Ollama
```

```
docker compose exec ollama ollama list
```

Apéndice G

# Manual de usuario

---

# Manual de Usuario

Memento es una aplicación web para procesar grabaciones de reuniones. Permite subir archivos de audio, obtener transcripciones y resúmenes automáticos, y realizar consultas conversacionales sobre el contenido de las reuniones.

Este manual describe cada una de las funcionalidades de la aplicación y explica cómo utilizarlas paso a paso. Está organizado en torno a las tareas que puedes realizar: autenticarte, gestionar reuniones, consultar su contenido mediante el chat y configurar la aplicación a tu medida.

## Índice

- 1. Autenticación
  - 1.1 Iniciar sesión
  - 1.2 Crear una cuenta
- 2. Reuniones
  - 2.1 Subir una grabación
  - 2.2 Lista de reuniones
  - 2.3 Consultar transcripción y resumen
  - 2.4 Editar metadatos
  - 2.5 Eliminar una reunión
- 3. Chat
  - 3.1 Realizar una consulta
  - 3.2 Gestionar el historial de conversaciones
- 4. Ajustes
  - 4.1 Preferencias generales
  - 4.2 Plantilla de resumen
  - 4.3 Proveedores y claves API
  - 4.4 Modelos de lenguaje
  - 4.5 Cuenta

## 1. Autenticación

Para usar la aplicación es necesario iniciar sesión o crear una cuenta.

### 1.1 Iniciar sesión

Desde la vista inicial, haz clic en *Iniciar sesión*.

# Bienvenido a Memento

Tu asistente inteligente para reuniones

Iniciar sesión

Registrarse

Figura 1: Vista inicial de la aplicación.

Se abrirá el formulario de inicio de sesión, donde puedes introducir tu correo electrónico y contraseña.

## Iniciar sesión en Memento

correo electrónico

contraseña

Iniciar sesión

¿Nuevo en Memento? [Crear una cuenta](#)

Figura 2: Formulario de inicio de sesión.

## 1.2 Crear una cuenta

Si no tienes cuenta, haz clic en *Registrarse* desde la vista inicial.

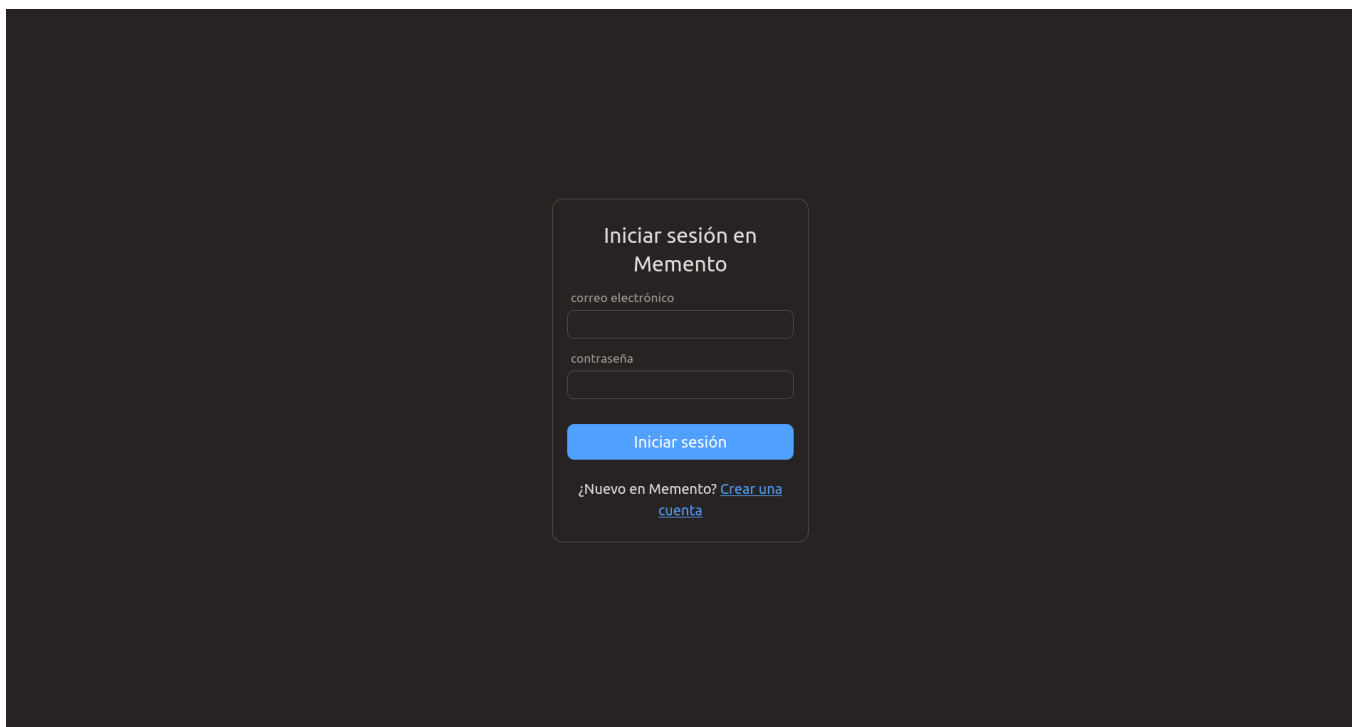


Figura 3: Botón de registro en la vista inicial.

En el formulario de registro, introduce tu correo electrónico y una contraseña (deberás confirmarla dos veces). Una vez completado, serás redirigido a la vista principal de la aplicación.

## 2. Reuniones

La gestión de reuniones es la funcionalidad principal de Memento. Las reuniones se crean subiendo archivos de audio; el sistema las procesa automáticamente para generar su transcripción y un resumen estructurado. Una vez procesadas, puedes consultar su contenido, editar sus metadatos o eliminarlas.

### 2.1 Subir una grabación

Para subir una o varias grabaciones, haz clic en *Subir reuniones* en el panel lateral.



Figura 4: Panel lateral con la opción "Subir reuniones".

Se abrirá el modal de subida. Para cada reunión debes indicar:

- **Título:** un nombre para identificar la reunión.

- **Fecha:** la fecha de la reunión (por defecto, la de hoy) .
- **Archivo:** el archivo de audio a procesar .

The screenshot shows the 'Subir Reuniones' modal in the MEMENTO application. The modal has a dark theme and a sidebar on the left with options like 'Nuevo Chat', 'Mis Reuniones', 'Subir Reuniones', 'Chats', 'Qué se trató?', 'Resumen reunión de hoy', and 'Configuración'. The main area contains a table with the following structure:

| # | ESTADO | TÍTULO                              | FECHA                                   | ARCHIVO                               |
|---|--------|-------------------------------------|-----------------------------------------|---------------------------------------|
| 1 | -      | <input type="text" value="Título"/> | <input type="text" value="15/06/2026"/> | <input type="text" value="Examinar"/> |

Below the table, there is a '+ Añadir reunión' button and a blue 'Enviar' button.

Figura 5: Modal de subida de reuniones.

Opcionalmente, puedes hacer clic en el botón situado a la derecha del selector de archivo para indicar el **idioma** de la reunión y el **número de hablantes** .

This screenshot shows the same 'Subir Reuniones' modal but with additional options. The table row now includes 'Idioma' and 'Participantes' columns:

| # | ESTADO | TÍTULO                              | FECHA                                   | ARCHIVO                               | Idioma                                  | Participantes                  |
|---|--------|-------------------------------------|-----------------------------------------|---------------------------------------|-----------------------------------------|--------------------------------|
| 1 | -      | <input type="text" value="Título"/> | <input type="text" value="15/06/2026"/> | <input type="text" value="Examinar"/> | <input type="text" value="Cualquiera"/> | <input type="text" value="1"/> |

The 'Idioma' column has a dropdown menu, and the 'Participantes' column has a numeric input field. The '+ Añadir reunión' and 'Enviar' buttons remain at the bottom.

Figura 6: Opciones adicionales: idioma y número de hablantes.

Si quieres subir más de una reunión, haz clic en + *Añadir reunión* tantas veces como necesites.

Cuando hayas completado el formulario, haz clic en *Enviar* para iniciar el procesamiento. La columna *Estado* muestra indicadores visuales del progreso:



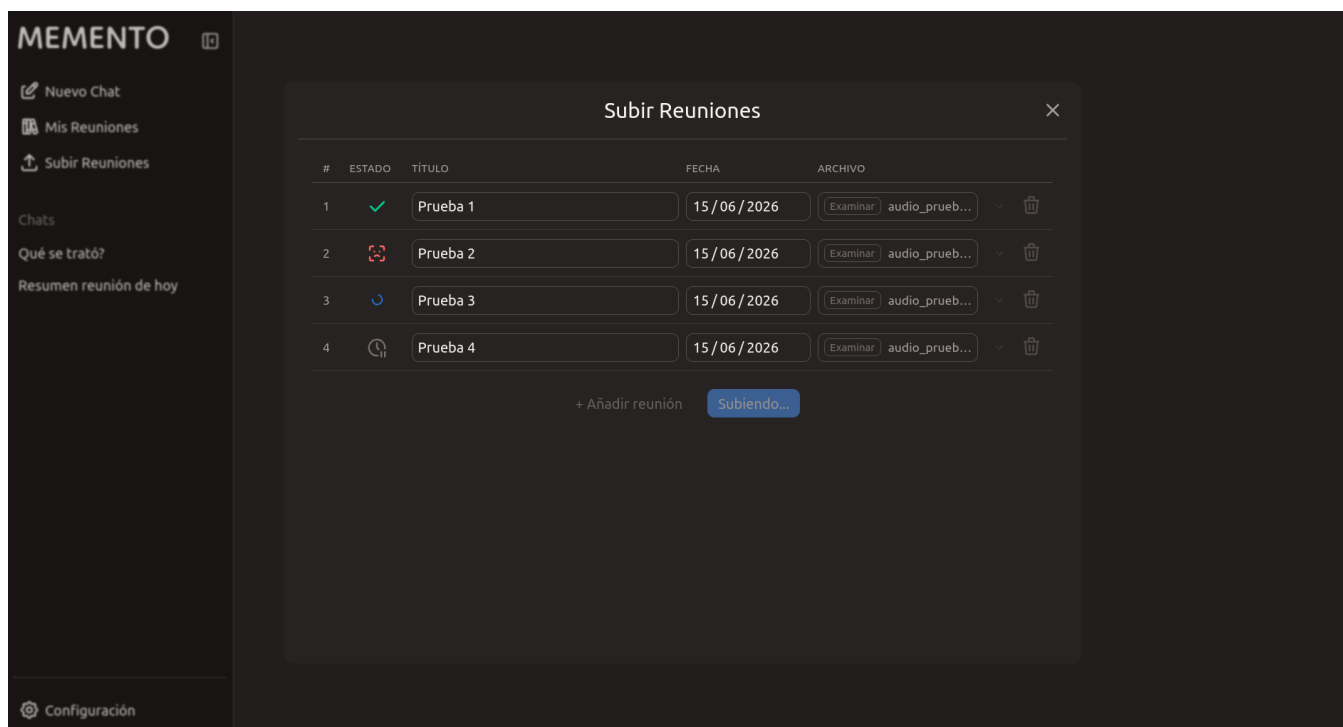


Figura 7: Estados de procesamiento: procesado con éxito, error, procesando y pendiente.

Una vez finalizado el proceso, el formulario se actualiza mostrando únicamente las reuniones que no pudieron procesarse correctamente, para que puedas reintentarlo si lo deseas.

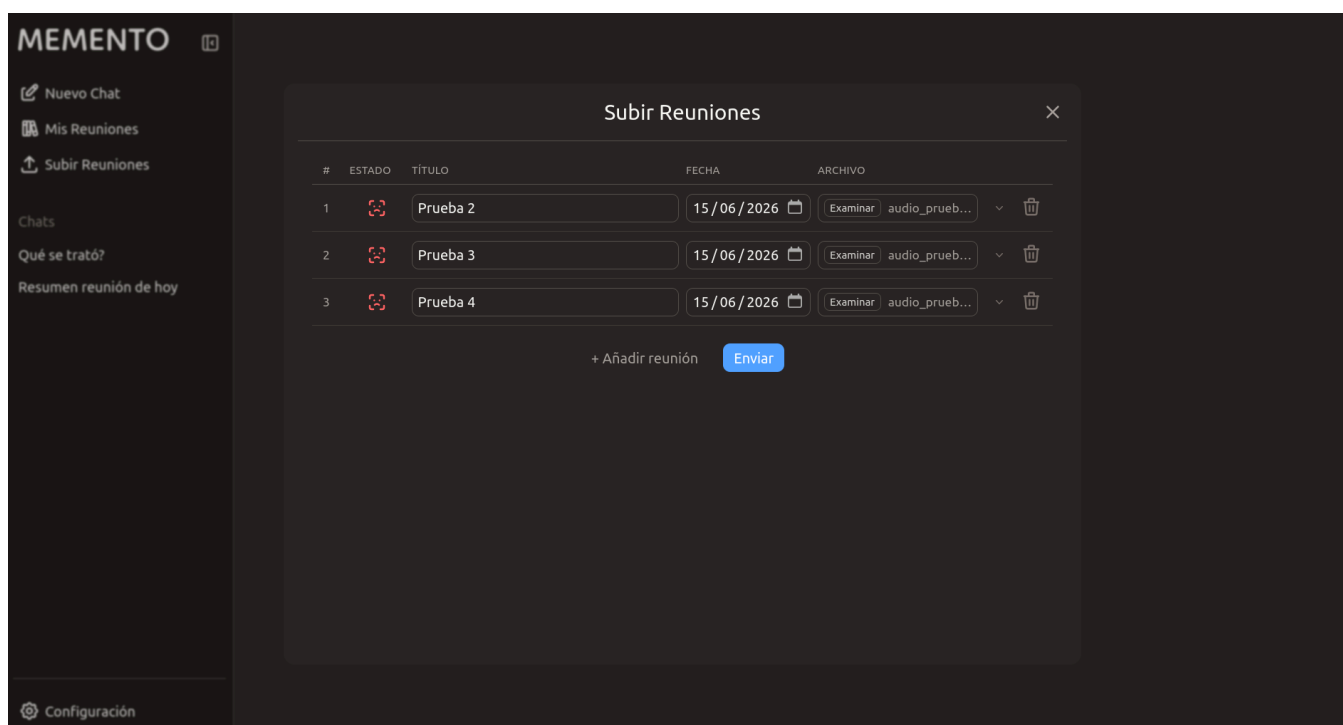


Figura 8: Reuniones que no se procesaron correctamente.

## 2.2 Lista de reuniones

Para ver tus reuniones procesadas, haz clic en *Mis Reuniones* en el panel lateral.



Figura 9: Panel lateral con la opción "Mis Reuniones".

Se mostrará la lista de todas tus reuniones con su título, fecha y estado.

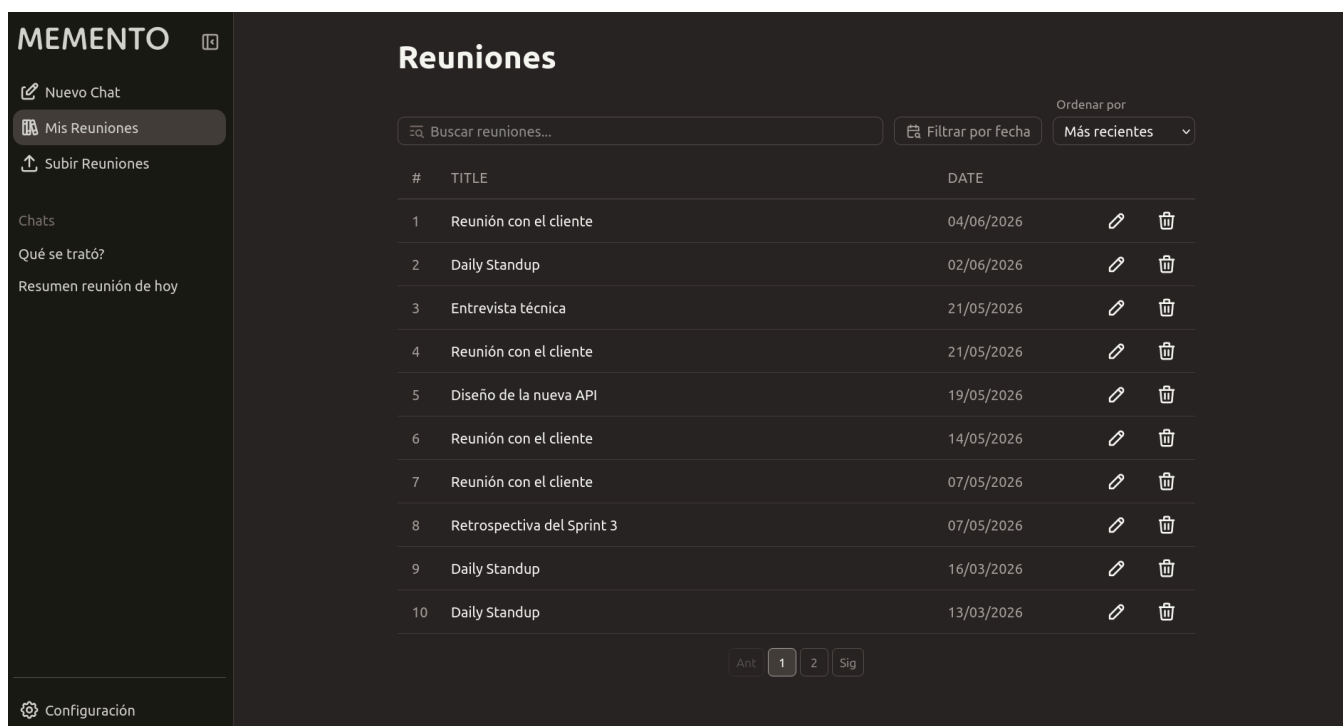


Figura 10: Lista de reuniones.

Puedes buscar reuniones introduciendo texto en el buscador, que filtra por título. También puedes acotar las reuniones por fecha haciendo clic en *Filtrar por fecha* y seleccionando un rango; la lista se actualizará automáticamente. Para eliminar los filtros de fecha, haz clic en *Limpiar*.

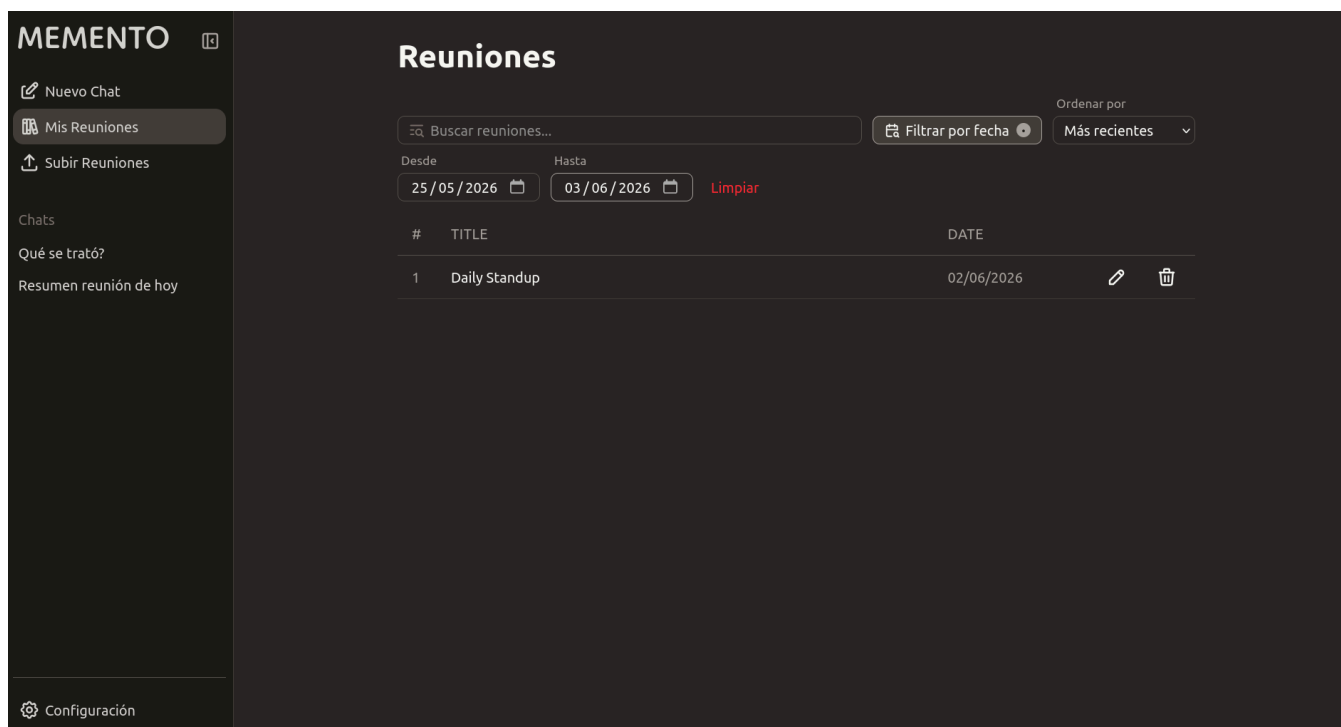


Figura 11: Filtro por rango de fechas.

Para cambiar el orden de las reuniones, usa el desplegable y selecciona el criterio deseado.

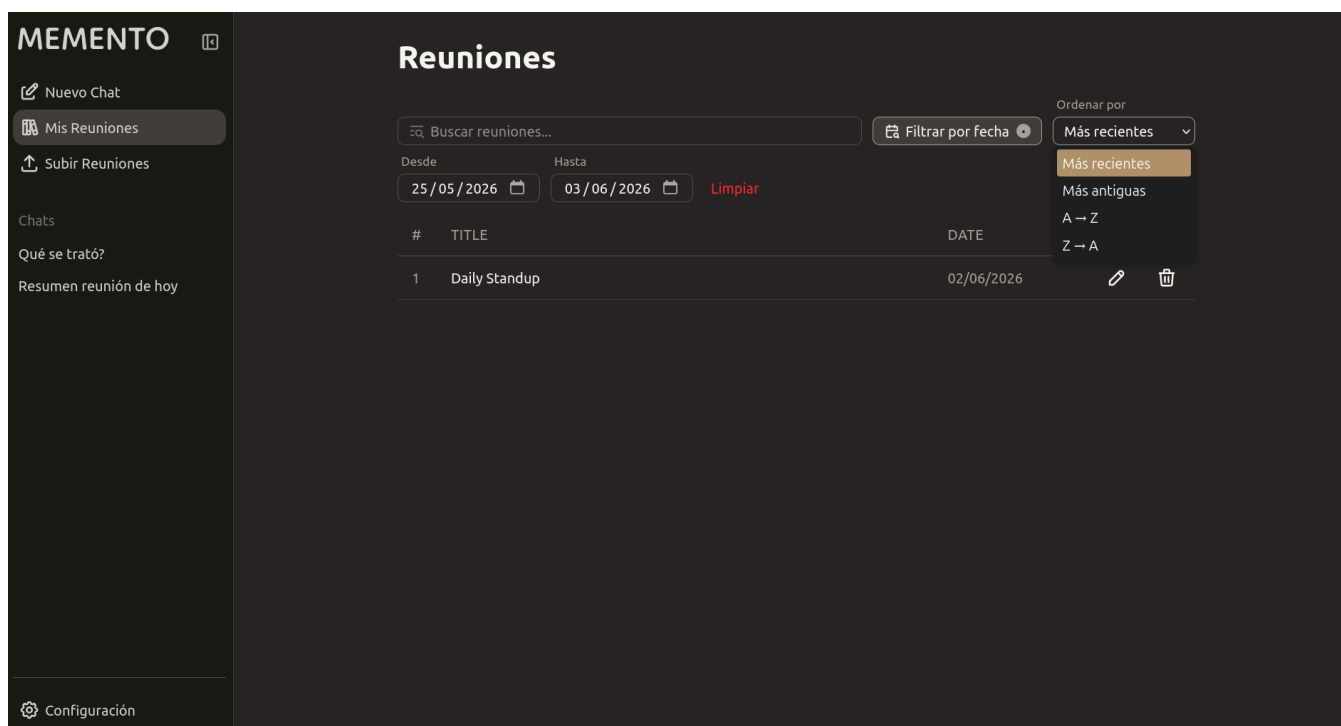


Figura 12: Desplegable de ordenación.

## 2.3 Consultar transcripción y resumen

Para ver los detalles de una reunión, haz clic sobre su título en la lista.

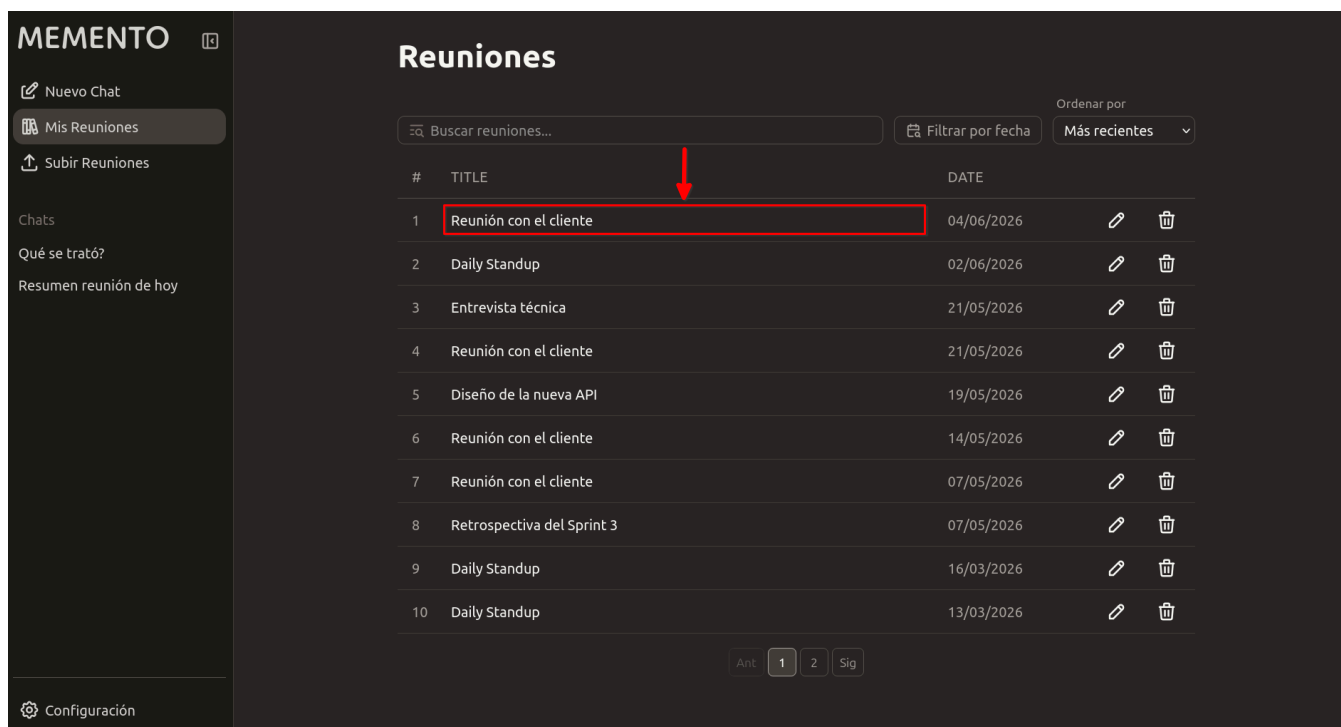


Figura 13: Acceso a los detalles de una reunión.

Se abrirá la vista de transcripción, donde puedes leer el contenido completo de la reunión.

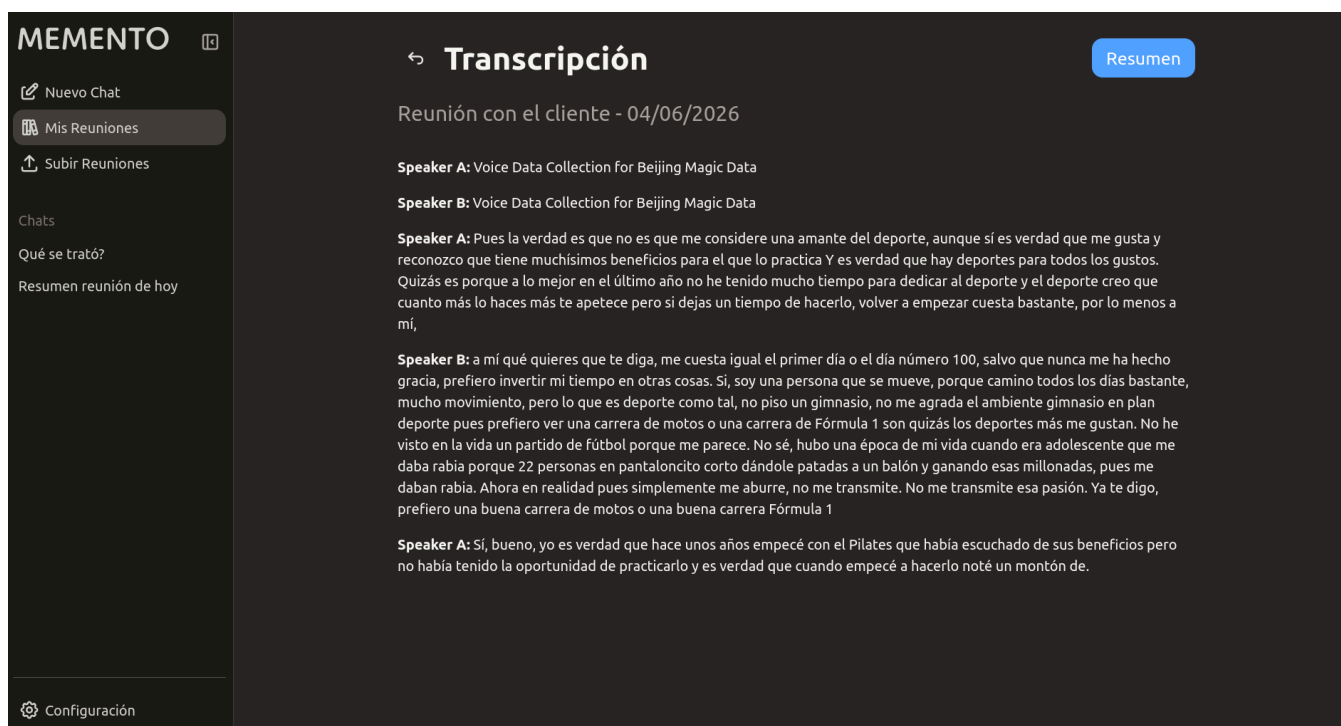


Figura 14: Vista de transcripción de la reunión.

Haciendo clic en la pestaña *Resumen*, puedes consultar el resumen estructurado generado automáticamente.

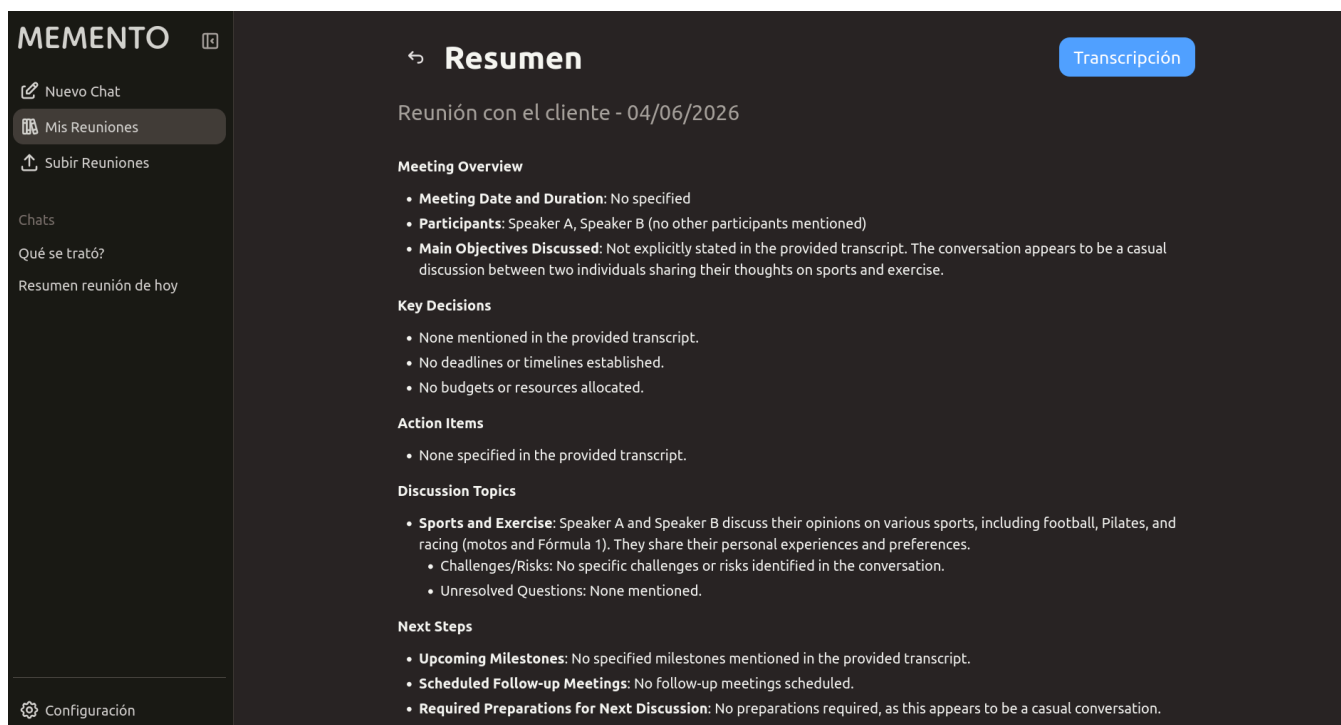


Figura 15: Vista del resumen de la reunión.

## 2.4 Editar metadatos

Desde la lista de reuniones, haz clic en el botón de editar de la reunión que quieras modificar.

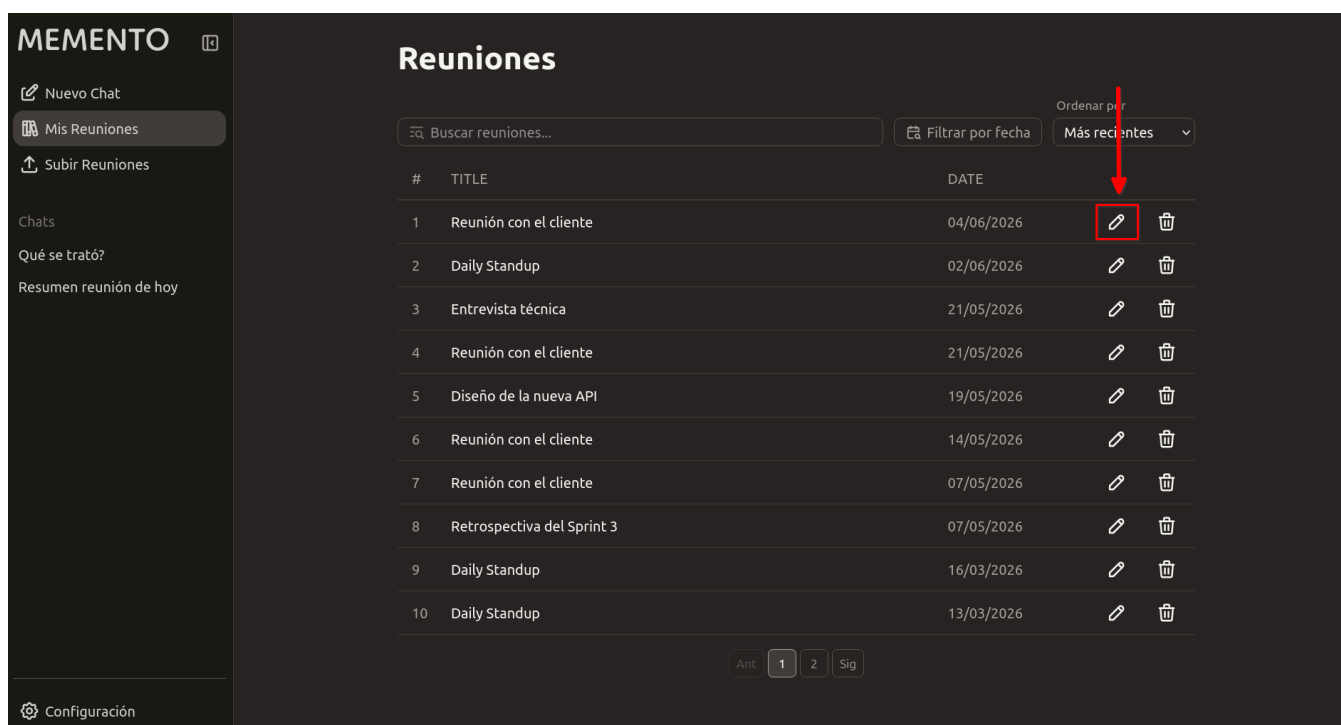


Figura 16: Botón de editar en la lista de reuniones.

Se habilitará el modo de edición, que te permite modificar el título y la fecha de la reunión.

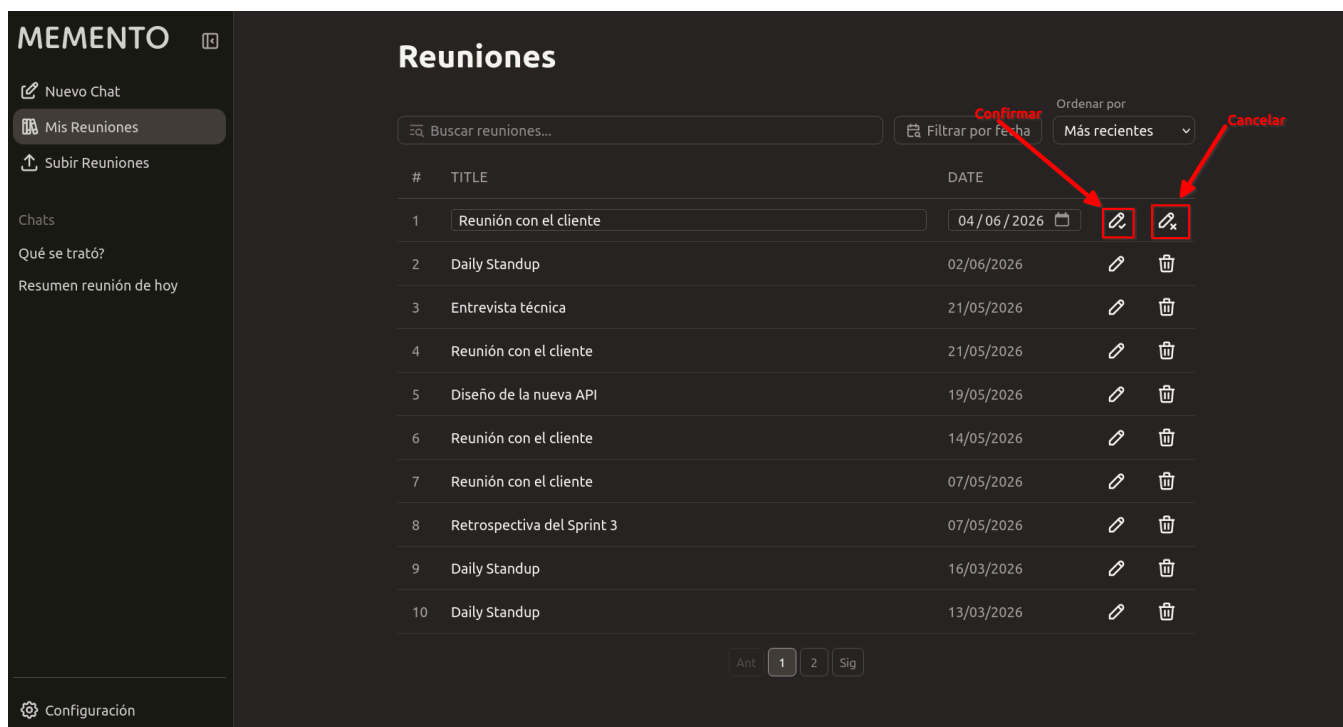


Figura 17: Modo de edición de metadatos.

Para confirmar los cambios, haz clic en el botón de confirmar. Si quieres descartarlos, haz clic en el botón de cancelar.

## 2.5 Eliminar una reunión

Desde la lista de reuniones, haz clic en el botón de eliminar de la reunión correspondiente.

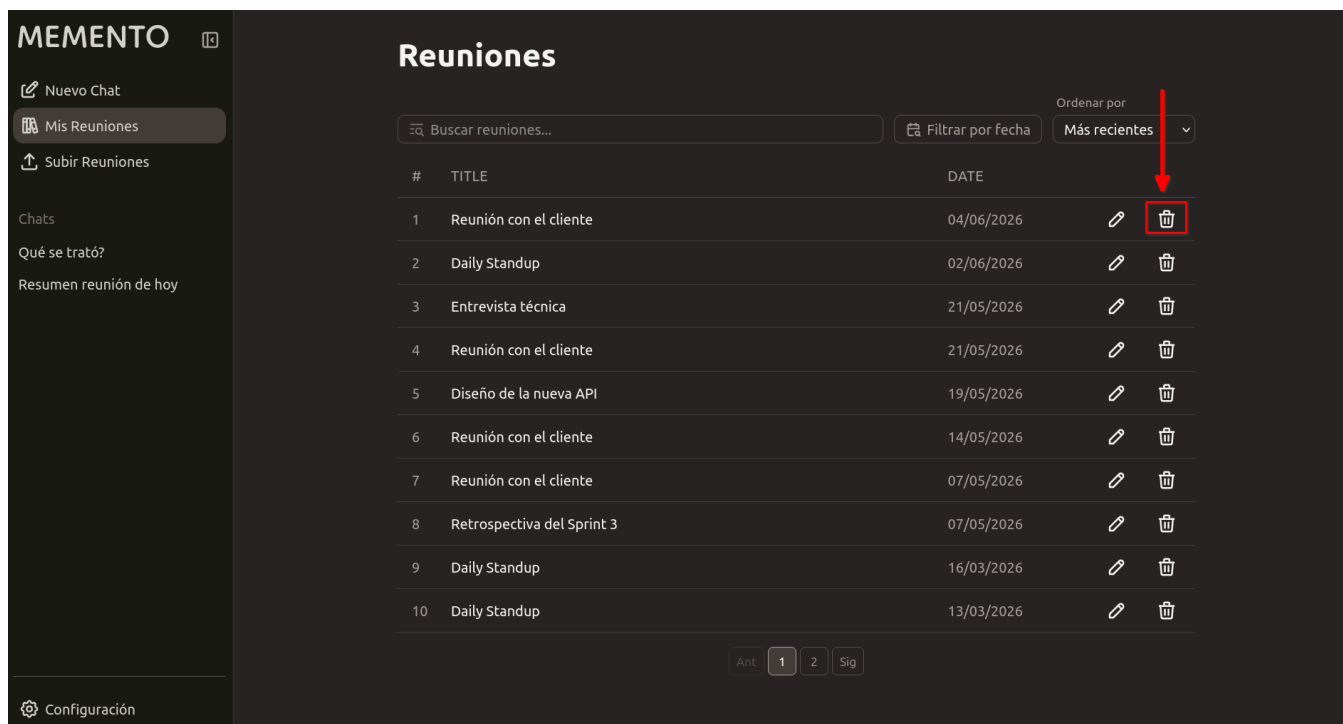


Figura 18: Botón de eliminar en la lista de reuniones.

Aparecerá un modal de confirmación. Haz clic en *Eliminar* para borrar la reunión definitivamente.

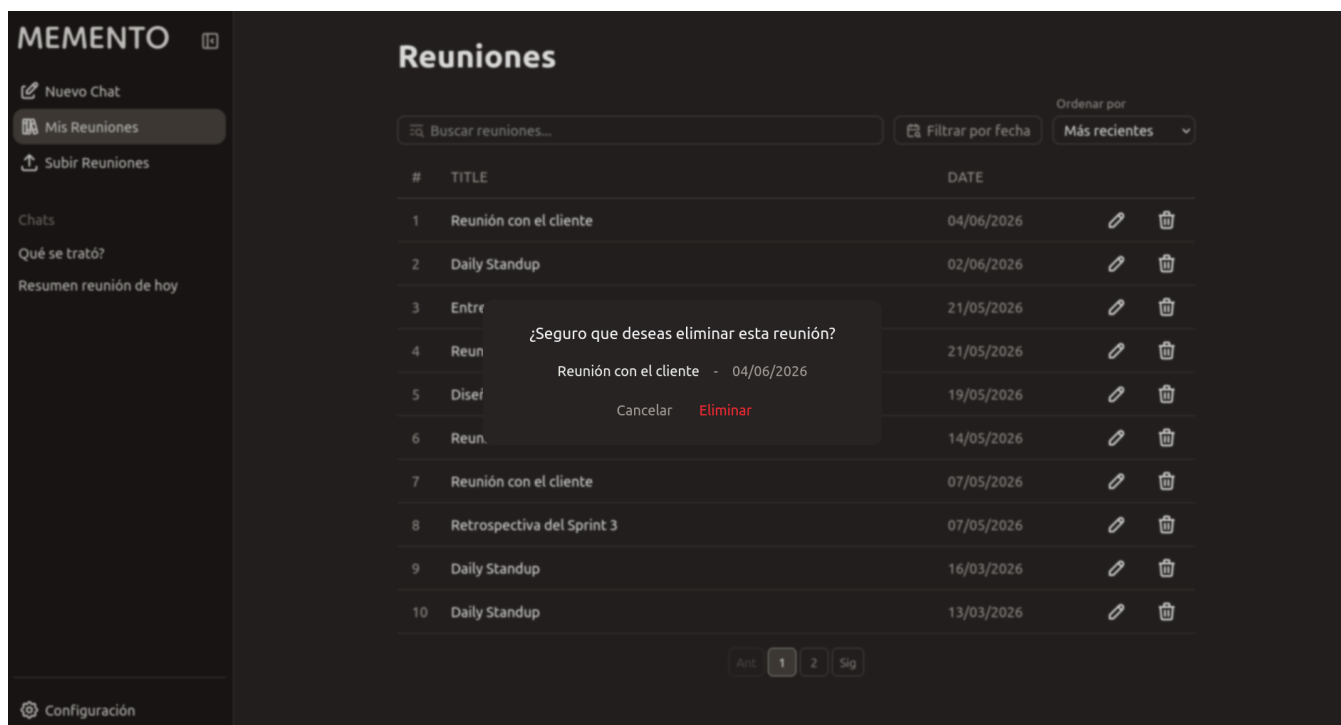


Figura 19: Modal de confirmación de eliminación.

### 3. Chat

El chat te permite hacer preguntas en lenguaje natural sobre el contenido de tus reuniones. El asistente utiliza los resúmenes de tus reuniones como contexto para generar respuestas.

#### 3.1 Realizar una consulta

Desde el panel lateral puedes navegar a la vista de chat. Tienes dos opciones:

- **Nueva conversación:** para iniciar un chat desde cero.
- **Conversación existente:** para reanudar un chat anterior.



Figura 20: Sección de chat en el panel lateral.

Si eliges *Nueva conversación*, verás una vista con un campo de texto donde puedes escribir tu consulta y enviarla con el botón o presionando Enter.

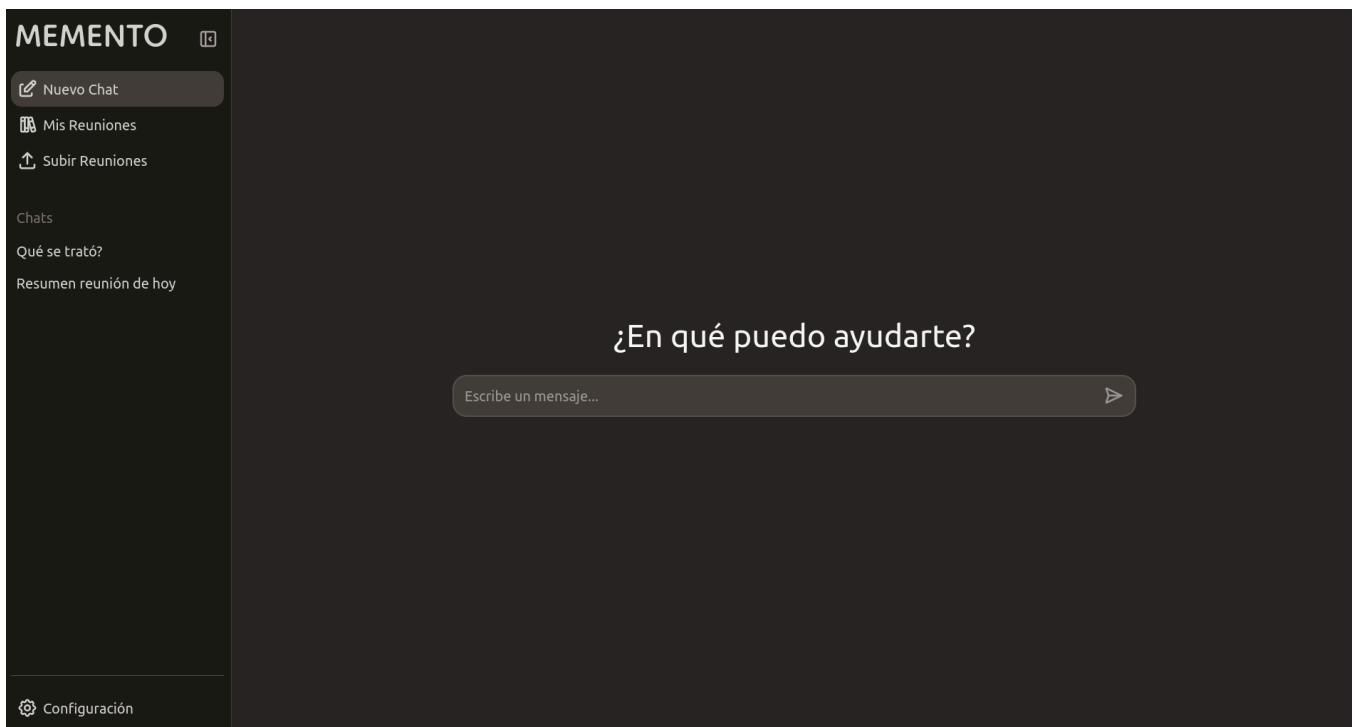


Figura 21: Vista de nueva conversación.

Si seleccionas una conversación existente, verás el historial de mensajes y podrás continuar la conversación de la misma forma.

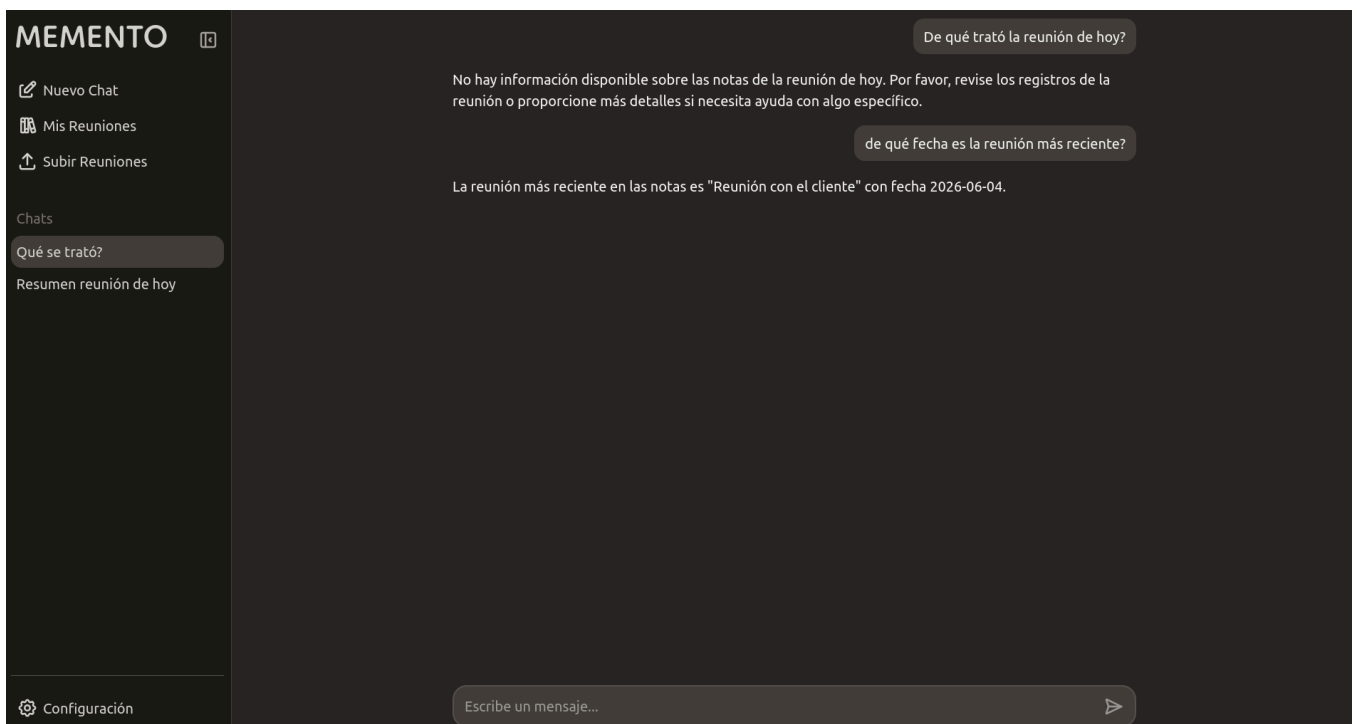


Figura 22: Vista de una conversación existente.

### 3.2 Gestionar el historial de conversaciones

Las conversaciones se listan en el panel lateral. Para gestionarlal, sitúa el cursor sobre una conversación y aparecerá un botón de opciones.



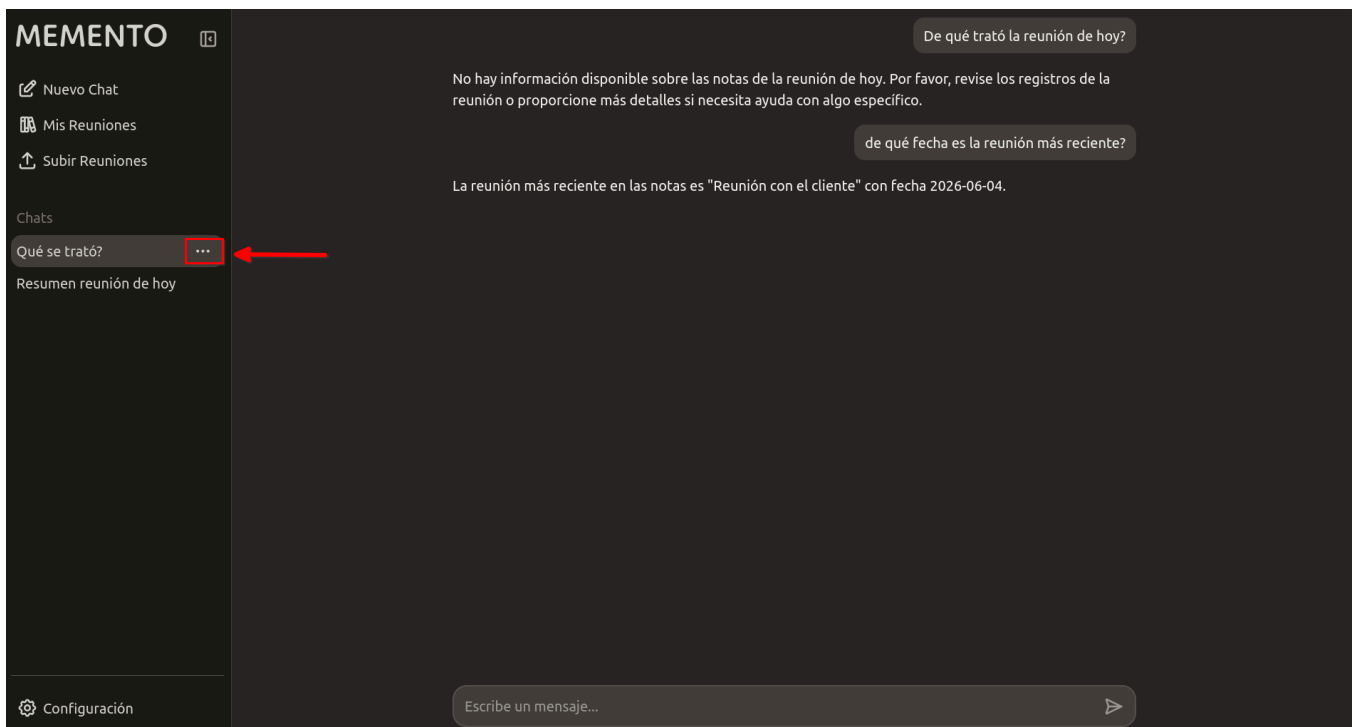


Figura 23: Botón de opciones al pasar el cursor sobre una conversación.

Al hacer clic, se despliega un menú con las siguientes acciones:

- **Renombrar:** cambia el nombre de la conversación.
- **Eliminar:** borra la conversación definitivamente.

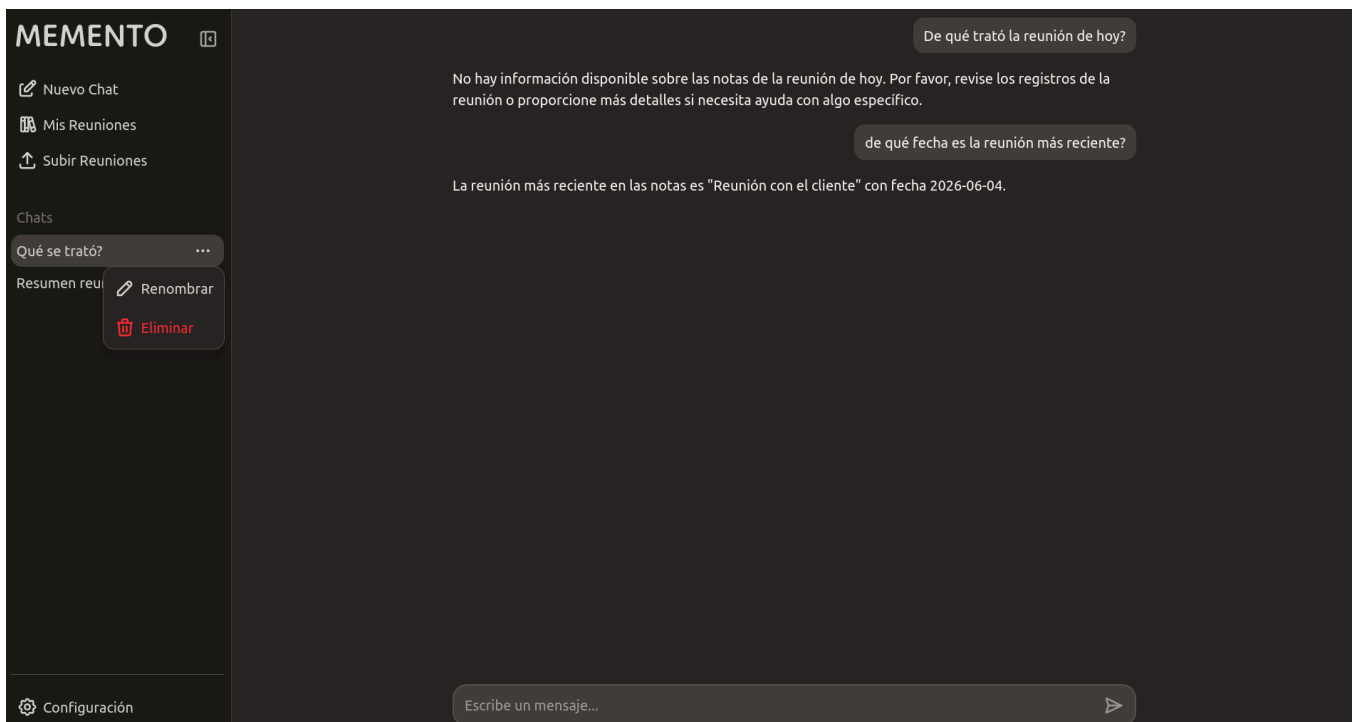


Figura 24: Menú de opciones de una conversación.

## 4. Ajustes

El modal de ajustes centraliza todas las opciones de configuración de la aplicación. Para abrirlo, haz clic en el botón de configuración del panel lateral.



Figura 25: Botón de configuración en el panel lateral.

## 4.1 Preferencias generales

La pestaña *General* te permite modificar:

- **Tema:** claro, oscuro o automático (según el sistema).
- **Idioma:** idioma de la interfaz.
- **Elementos por página:** número de reuniones mostradas por página en la lista.

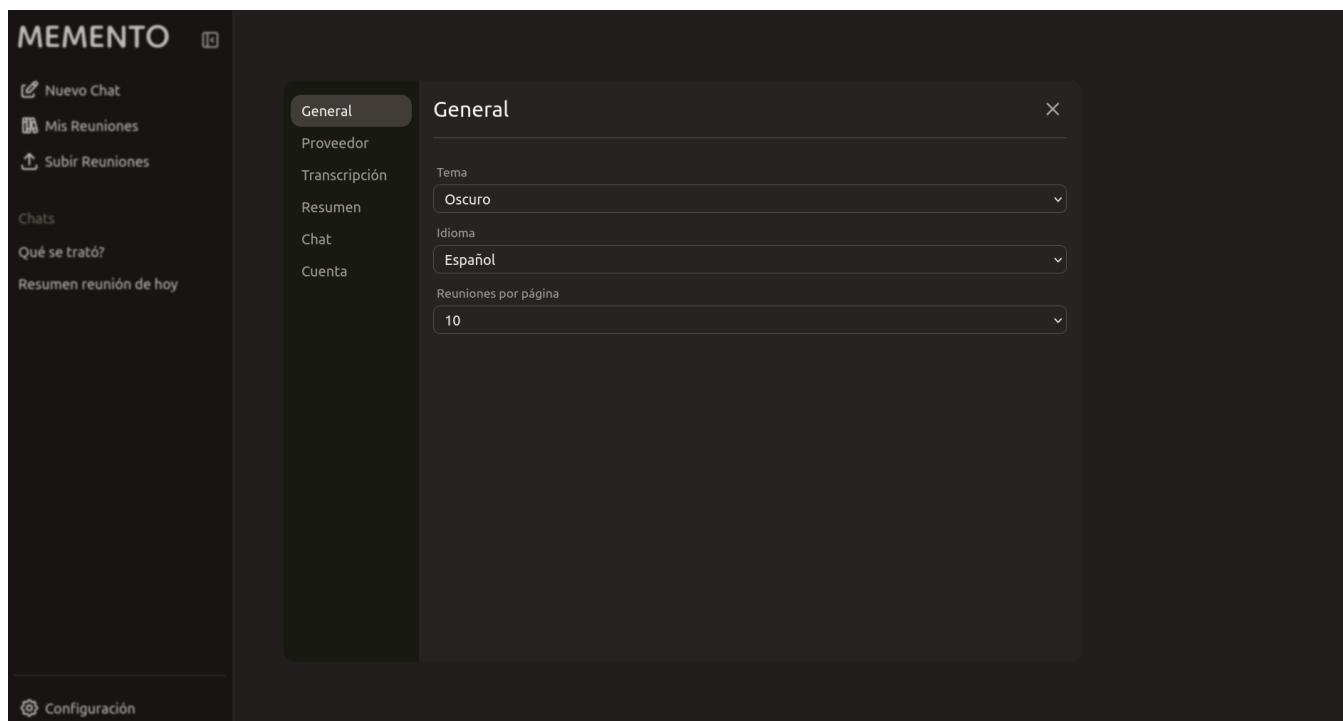


Figura 26: Ajustes generales de la aplicación.

## 4.2 Plantilla de resumen

En la pestaña *Resumen* puedes visualizar y editar la plantilla que se utiliza para generar los resúmenes de las reuniones. Si has modificado la plantilla y quieres volver a la original, puedes usar el botón *Restaurar plantilla por defecto*.

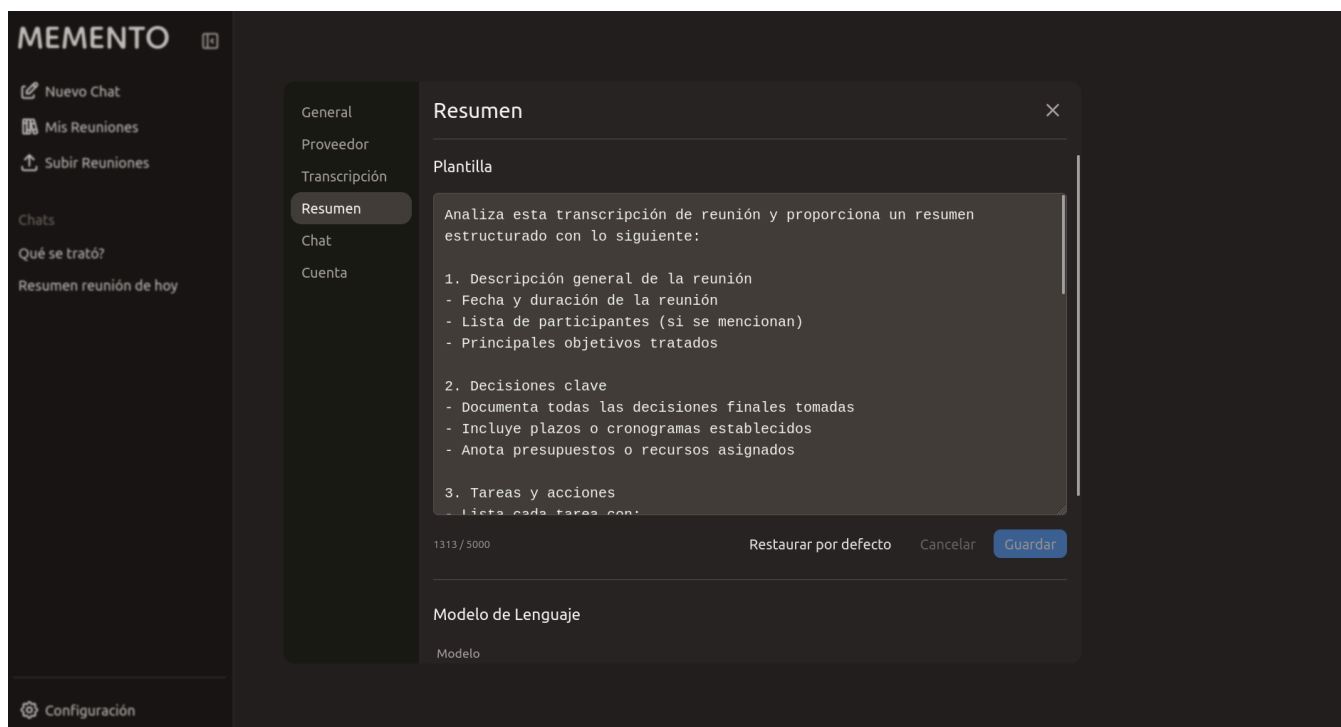


Figura 27: Ajustes de la plantilla de resumen.

### 4.3 Proveedores y claves API

En la pestaña *Proveedores* se muestran los proveedores de modelos de lenguaje disponibles (Ollama, OpenAI y Anthropic). Para usar los modelos de un proveedor, necesitas añadir tu clave API.

Haz clic en *Añadir clave API* e introduce la clave en el formulario.

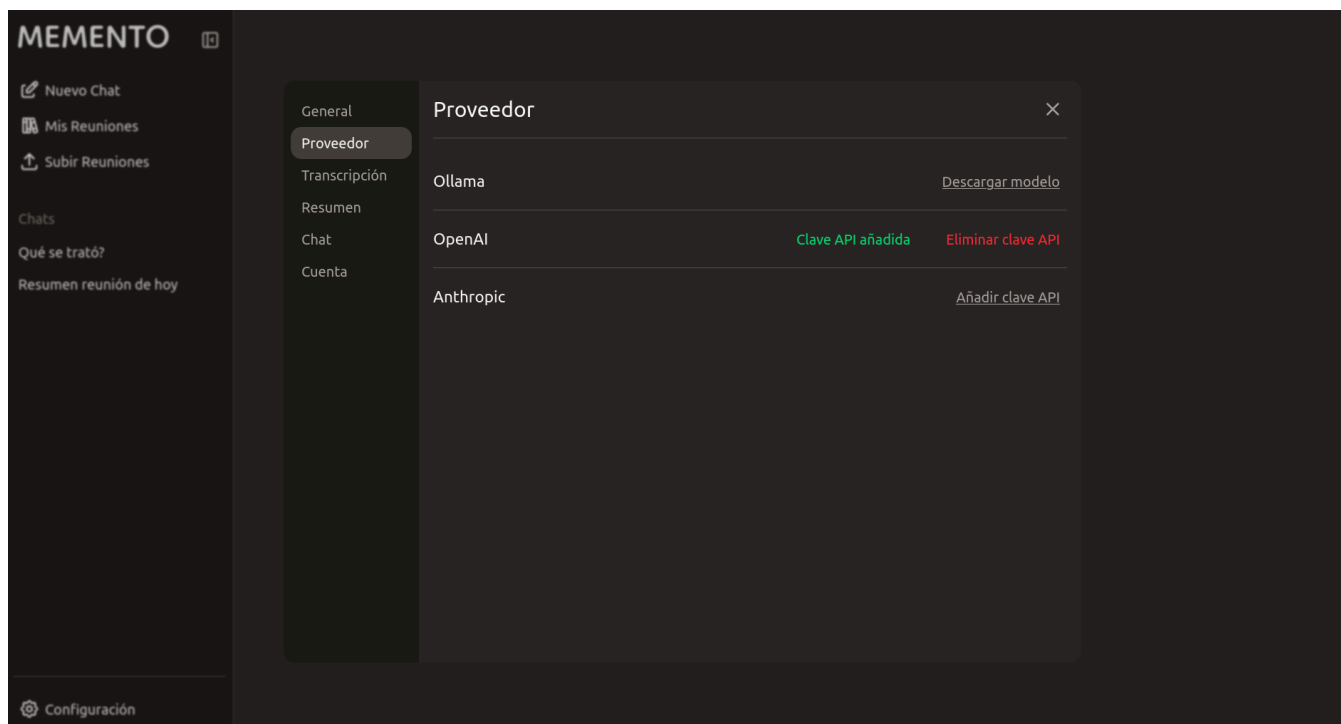


Figura 28: Lista de proveedores con botón para añadir clave API.

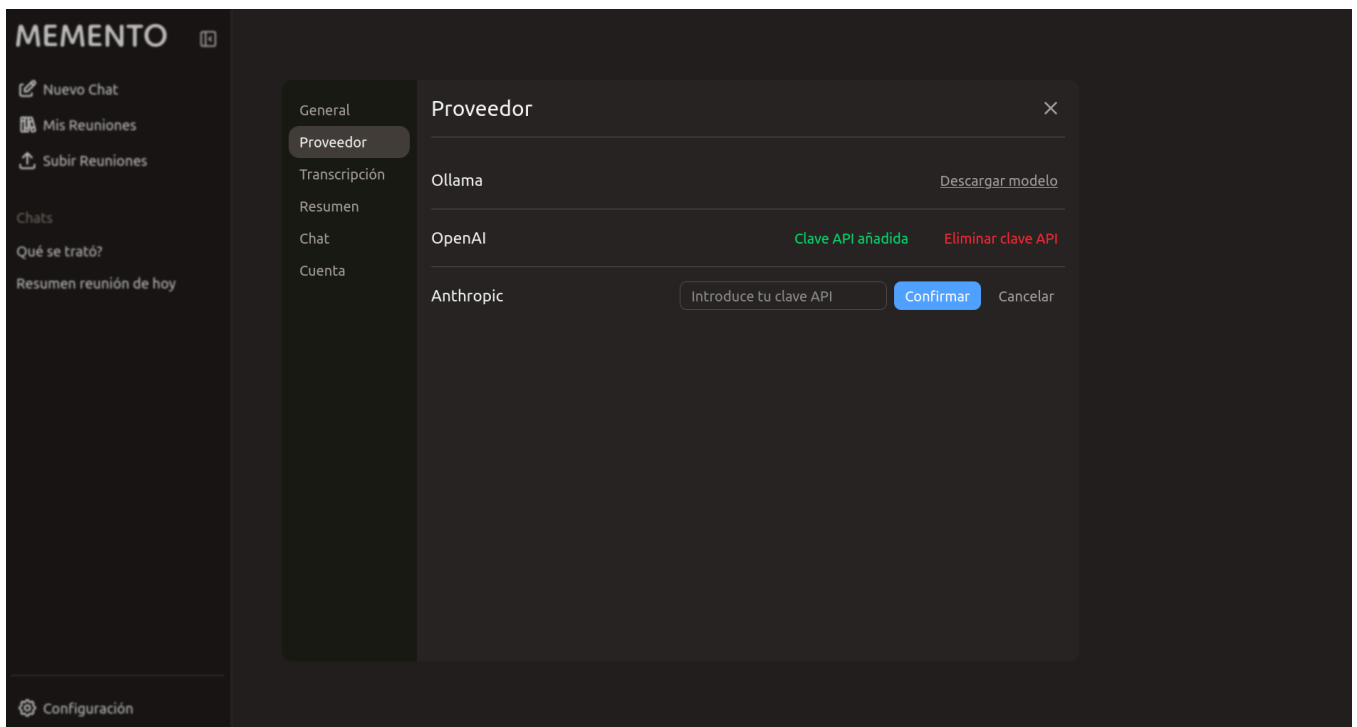


Figura 29: Formulario para añadir una clave API.

Para eliminar una clave que ya has añadido, haz clic en *Eliminar clave API*.

## 4.4 Modelos de lenguaje

Las pestañas *Chat* y *Resumen* te permiten seleccionar el modelo de lenguaje a utilizar en cada caso, así como ajustar dos hiperparámetros:

- **Temperatura:** controla la creatividad de las respuestas (valores más altos producen respuestas más variadas).
- **Tokens máximos:** limita la longitud máxima de la respuesta generada.

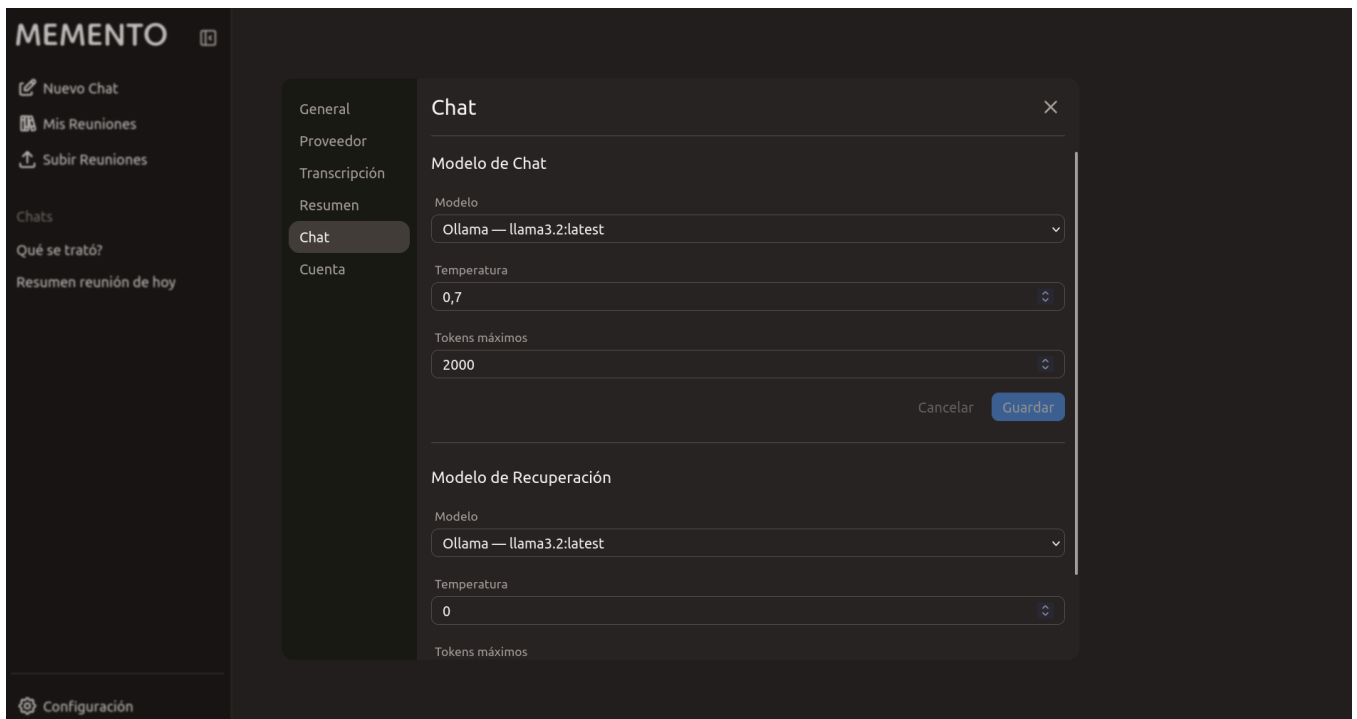


Figura 30: Configuración del modelo para el chat.

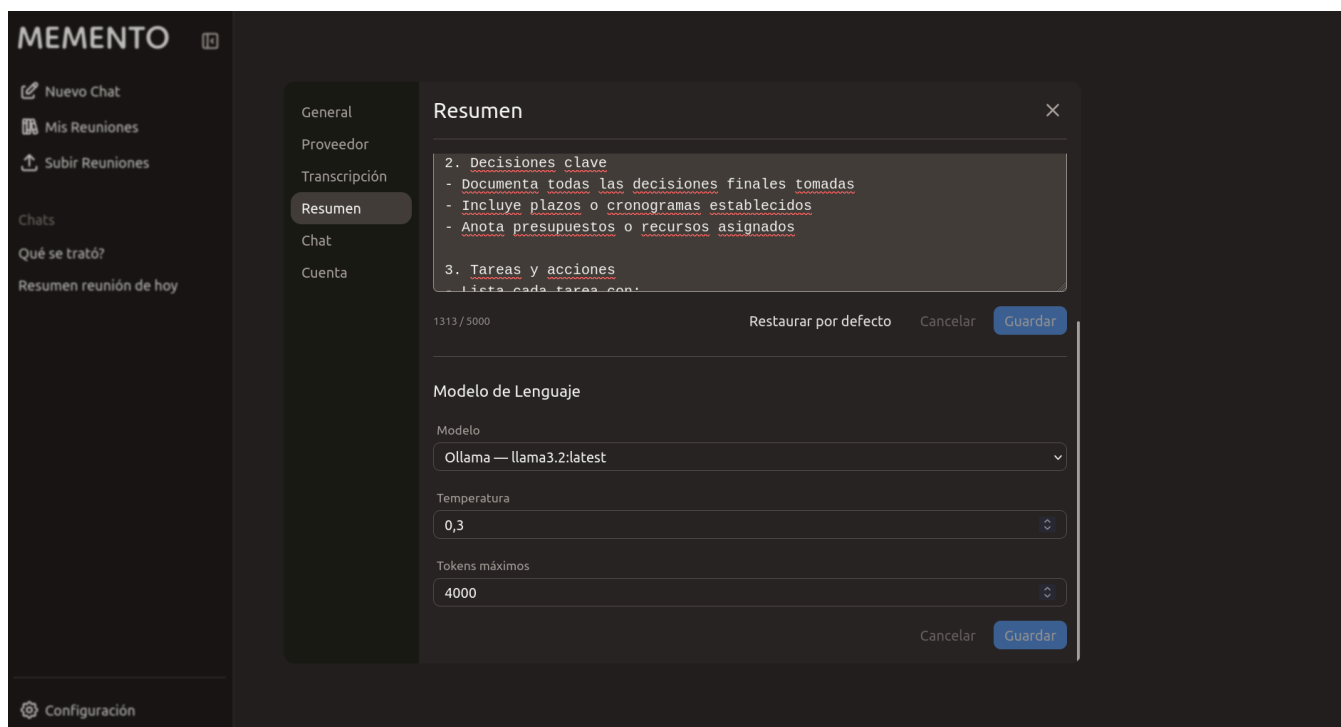


Figura 31: Configuración del modelo para el resumen.

## 4.5 Cuenta

La pestaña *Cuenta* muestra el correo electrónico con el que has iniciado sesión y ofrece las siguientes opciones:

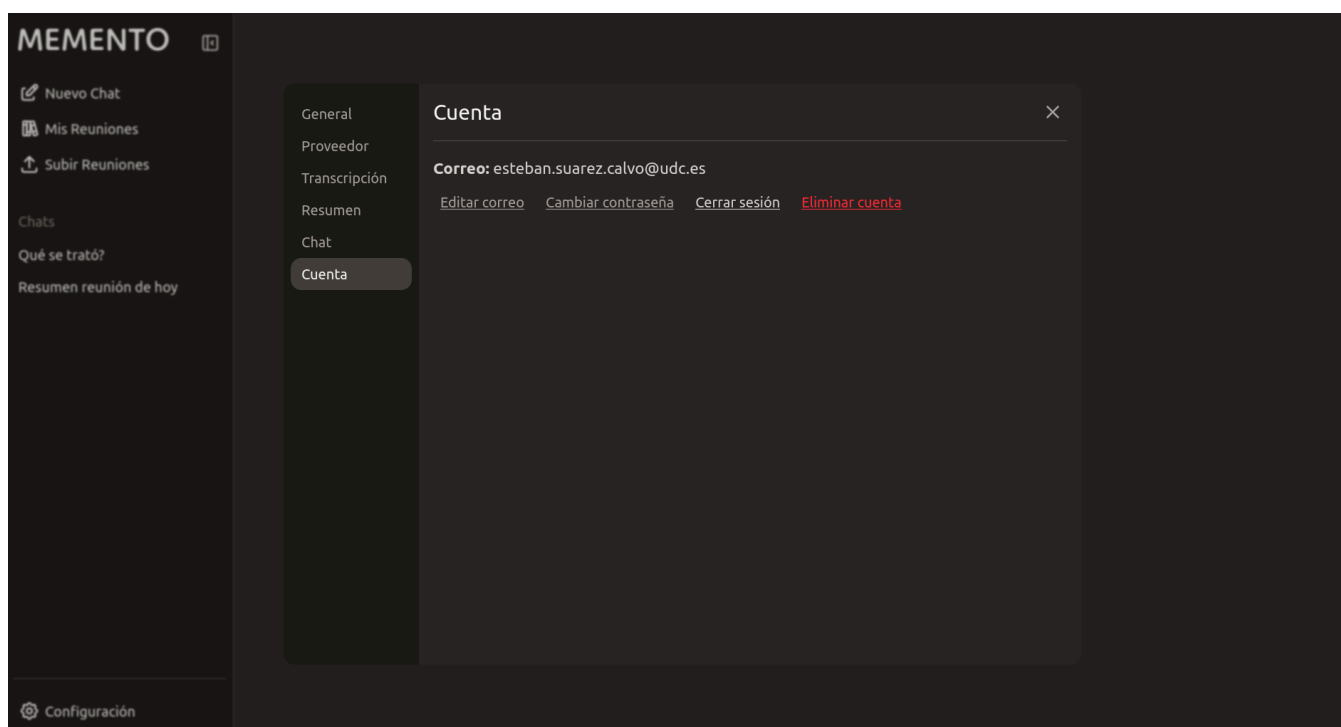


Figura 32: Ajustes de cuenta.

**Editar correo electrónico.** Haz clic en *Editar correo*, introduce el nuevo correo y tu contraseña actual para confirmar el cambio.

MEMENTO

Nuevo Chat

Mis Reuniones

Subir Reuniones

Chats

Qué se trató?

Resumen reunión de hoy

Configuración

General

Proveedor

Transcripción

Resumen

Chat

Cuenta

Cuenta

Correo: esteban.suarez.calvo@udc.es

Nuevo correo

Contraseña

Cancelar Guardar

Figura 33: Formulario de edición de correo electrónico.

**Modificar contraseña.** Haz clic en *Modificar contraseña*, introduce tu contraseña actual y la nueva (debes confirmarla dos veces).

MEMENTO

Nuevo Chat

Mis Reuniones

Subir Reuniones

Chats

Qué se trató?

Resumen reunión de hoy

Configuración

General

Proveedor

Transcripción

Resumen

Chat

Cuenta

Cuenta

Correo: esteban.suarez.calvo@udc.es

Contraseña actual

Nueva contraseña

Confirmar nueva contraseña

Cancelar Guardar

Figura 34: Formulario de cambio de contraseña.

**Eliminar cuenta.** Haz clic en *Eliminar cuenta*, lee el aviso sobre el periodo de conservación de datos e introduce tu contraseña para confirmar la eliminación.

MEMENTO

Nuevo Chat

Mis Reuniones

Subir Reuniones

Chats

Qué se trató?

Resumen reunión de hoy

Configuración

General

Proveedor

Transcripción

Resumen

Chat

Cuenta

Cuenta

Correo: **esteban.suarez.calvo@udc.es**

Al eliminar tu cuenta, tus datos se conservarán durante **30 días** antes de su eliminación definitiva.

Durante ese plazo puedes solicitar la eliminación inmediata de tus datos o la recuperación de tu cuenta escribiendo a **privacidad@tu-dominio.com**.

Contraseña

Cancelar

Eliminar

Figura 35: Formulario de eliminación de cuenta.

# Lista de acrónimos

---

**API** *Application Programming Interface*. 1, 2, 7, 15, 16, 17, 18, 20, 32, 35, 38, 40, 45, 57, 147, 149

**BSON** *Binary JSON*. 19

**CPU** *Central Processing Unit*. 19, 27, 36

**CRUD** *Create Retrieve Update Delete*. 35, 38

**DAFO** *Debilidades Amenazas Fortalezas Oportunidades*. i, vi, 2, 4, 9

**DNS** *Domain Name Service*. 21

**DOM** *Document Object Model*. 17

**GPU** *Graphical Processing Unit*. 9, 27, 36

**HTTP** *HyperText Transfer Protocol*. iv, 13, 17, 38, 69, 149, 150

**HTTPS** *Hyper Text Transfer Protocol Secure*. 20, 69

**IA** *Inteligencia Artificial*. 1, 6, 11, 12, 13, 17, 19, 72, 147

**IP** *Internet Protocol*. 21

**JSON** *JavaScript Object Notation*. 13, 53, 68

**JWT** *JSON Web Token*. 41, 42, 43, 49, 50, 66, 149

**LLM** *Large Language Model*. 1, 2, 7, 8, 10, 11, 12, 17, 18, 28, 36, 38, 39, 40, 51, 54, 62, 66, 67, 71, 72, 73, 149

**MCP** *Model Context Protocol*. 11, 18, 25, 28, 39, 72, 73



- MSW** *Mock Service Worker*. 17
- PLN** *Procesamiento del Lenguaje Natural*. 1, 17
- RAG** *Retrieval Augmented Generation*. 1, 2, 12, 18, 19, 28, 39, 40, 72
- RAM** *Random Access Memory*. 9, 27
- REST** *Representational State Transfer*. 13
- RF** *Requisito Funcional*. 30
- RNF** *Requisito No Funcional*. 30
- SaaS** *Software as a Service*. i, 5, 32, 74
- SDK** *Software Development Kit*. 18
- SO** *Sistema Operativo*. 27
- SPA** *Single Page Application*. 14, 72
- SSE** *Server Sent Events*. iii, 11, 13, 26, 64, 65
- SSL** *Secure Socket Layer*. 69
- STT** *Speech to Text*. 10
- SVG** *Scalable Vector Graphics*. 53
- UI** *User Interface*. 48
- URL** *Uniform Resource Locator*. 15, 21
- UUID** *Universally Unique Identifier*. 52
- VRAM** *Video Random Access Memory*. 27
- WS** *WebSockets*. 13, 26, 38, 55, 64, 66, 67

# Glosario

---

**API Key** (*Application Programming Interface Key*). Código único utilizado por un [API](#) para identificar y autenticar al usuario o aplicación que realiza una petición. [19](#), [45](#), [62](#)

**API REST** Interfaz de programación de aplicaciones que sigue los principios de la arquitectura REST (*Representational State Transfer*) y utiliza el protocolo HTTP para la comunicación. Permite el intercambio de datos entre aplicaciones de forma estructurada y desacoplada, facilitando que ambos componentes puedan evolucionar de manera independiente. [13](#), [32](#)

**backend** También conocido como *lado del servidor* (server-side), es la capa de lógica y persistencia de datos del software. Es el encargado de procesar la mayor parte de la lógica de negocio, gestionar la autenticación, interactuar con bases de datos (y en este caso con modelos de [IA](#)) y exponer puntos de enlace para que el [frontend](#) pueda comunicarse con él. [i](#), [ii](#), [iv](#), [13](#), [15](#), [16](#), [17](#), [18](#), [19](#), [20](#), [25](#), [32](#), [33](#), [34](#), [35](#), [36](#), [37](#), [38](#), [40](#), [41](#), [42](#), [43](#), [45](#), [46](#), [49](#), [53](#), [54](#), [55](#), [56](#), [58](#), [60](#), [62](#), [64](#), [65](#), [66](#), [67](#), [74](#)

**boilerplate** Secciones de código que deben incluirse en muchos lugares con poca o ninguna alteración para que el software funcione, a menudo consideradas como código repetitivo o redundante. [15](#), [16](#)

**chatbot** Programa informático diseñado para simular una conversación con usuarios humanos, ya sea a través de texto o de voz, utilizando habitualmente técnicas de procesamiento de lenguaje natural para entender y responder a las peticiones. [2](#), [6](#), [12](#)

**copyleft** Método de licenciamiento en el ámbito del software libre que permite la libre distribución y modificación de un programa, bajo la condición estricta de que cualquier obra derivada o versión extendida mantenga exactamente la misma licencia, impidiendo así que el código sea integrado en software privativo. [32](#)

**diarización** Proceso de partición de un flujo de audio en segmentos temporales según la identidad del hablante. [19](#)

**embeddings** Representaciones matemáticas de unidades lingüísticas (como palabras, frases o documentos) en un espacio vectorial de alta dimensionalidad. Su función principal es transformar el texto en vectores numéricos de manera que el sistema pueda procesar la similitud semántica; esto permite que fragmentos de texto con significados parecidos se sitúen matemáticamente cerca en dicho espacio. [i](#), [12](#), [40](#)

**endpoint** Punto final de comunicación a través del cual una API recibe peticiones de acceso a los recursos o servicios que ofrece. [iv](#), [13](#), [17](#), [18](#), [33](#), [35](#), [36](#), [38](#), [39](#), [40](#), [41](#), [42](#), [43](#), [44](#), [45](#), [47](#), [50](#)

**framework** Estructura conceptual y tecnológica de soporte, compuesta por un conjunto de librerías, herramientas y convenciones, que sirve de base para la organización y agilización del desarrollo de software (por ejemplo, FastAPI o React).. [14](#), [16](#), [17](#), [18](#), [42](#)

**frontend** También conocido como *lado del cliente* (client-side), es la capa del software con la que el usuario interactúa de manera directa. Engloba la interfaz gráfica de usuario y la lógica responsable de estructurar el contenido, aplicar estilos visuales y capturar las interacciones del usuario. En el contexto del desarrollo web, se ejecuta en el navegador y se comunica con el servidor para enviar y recibir datos. [i](#), [ii](#), [iv](#), [12](#), [13](#), [15](#), [16](#), [17](#), [20](#), [26](#), [28](#), [42](#), [45](#), [46](#), [49](#), [51](#), [54](#), [55](#), [58](#), [62](#), [64](#), [65](#), [66](#), [67](#), [68](#), [69](#), [72](#), [147](#), [150](#)

**hashing** Proceso mediante el cual se aplica un algoritmo matemático (conocido como función *hash*) a un conjunto de datos, como por ejemplo una contraseña, para transformarlo en una cadena de caracteres alfanuméricos. Es una operación unidireccional, es decir, no es posible obtener el dato original a partir del hash resultante. [42](#)

**hook** Función especial en React que permite "enganchar" el estado y otras características del ciclo de vida de la biblioteca en componentes funcionales sin necesidad de escribir clases. [15](#), [49](#), [58](#), [60](#)

**IA generativa** Subcampo de la inteligencia artificial centrado en modelos capaces de crear contenido original (texto, imágenes, audio o código) a partir de patrones y estructuras aprendidos de grandes conjuntos de datos existentes. [8](#)

**internacionalización** Proceso de diseño y desarrollo de una aplicación para que pueda adaptarse fácilmente a diferentes idiomas y regiones sin requerir cambios estructurales en el código. [16](#)

**issue** Unidad de trabajo en sistemas de gestión de proyectos que representa una tarea, una mejora, una duda o el reporte de un error. 21

**kanban** Metodología de gestión de proyectos ágil que se centra en la visualización del trabajo, la limitación del trabajo en curso y la mejora continua del flujo de entrega. 21

**licencia MIT** Licencia de software libre permisiva creada por el Instituto Tecnológico de Massachusetts. Permite la reutilización, copia, modificación y distribución del software con restricciones mínimas. 7

**media query** Técnica de CSS que permite aplicar reglas de estilo de manera condicional en función de las características del dispositivo o navegador del usuario, como la resolución de pantalla o las preferencias de color del sistema operativo. 68

**mockeado (mocking)** Técnica utilizada en pruebas de software que consiste en simular el comportamiento de dependencias, objetos o sistemas reales (como por ejemplo un [API](#) externa) mediante objetos simulados conocidos como *mocks*. Esto permite ejecutar pruebas de un componente específico de un sistema de manera totalmente aislada, predecible y sin consumir recursos externos. 53

**mockup** Prototipo visual de una interfaz de usuario. Se emplea en las fases de diseño de la ingeniería de software para representar la estructura, disposición de los elementos y el aspecto gráfico de una aplicación de forma previa a su implementación en código. 25, 31, 51, 53, 61

**modelo de pesos abiertos** (*Open-weights model*). Modelo de inteligencia artificial cuyos parámetros entrenados (pesos) están disponibles públicamente para su descarga y ejecución. A diferencia de los modelos cerrados, permiten su despliegue en infraestructuras propias y garantizan una mayor transparencia y personalización. 9

**payload** Cuerpo de datos que se transmiten en un paquete de red, petición [HTTP](#) o dentro de un [token](#) de autenticación (como un [JWT](#)), excluyendo las cabeceras y los metadatos de control del protocolo. 42, 43

**prompt** Texto, instrucción o conjunto de directrices que se proporciona como entrada (*input*) a un [LLM](#). 62

**prop drilling** Práctica en el desarrollo con componentes (común en React) que consiste en pasar datos a través de múltiples niveles de la jerarquía de componentes hasta llegar a un componente hijo profundamente anidado. 15

- provider nesting*** Situación que ocurre al utilizar múltiples proveedores de contexto (en React) que se envuelven unos a otros, dificultando la lectura y el mantenimiento del árbol de componentes. [15](#)
- proxy*** Servidor intermedio que actúa como punto de enlace entre un cliente y un servidor de destino, interceptando y gestionando las peticiones según reglas de seguridad o redirección. [iii](#), [20](#), [26](#), [64](#), [69](#)
- query*** Consulta o petición formulada a una base de datos o sistema de gestión de información para recuperar datos específicos. En el contexto del desarrollo *frontend*, también hace referencia a las peticiones asíncronas para la obtención y sincronización del estado del servidor. [62](#)
- router*** En el contexto de un backend desarrollado en FastAPI, un router es el componente encargado de mapear las peticiones [HTTP](#) entrantes y dirigir las a la función correspondiente. [35](#)
- Speech-to-Text*** Tecnología de reconocimiento del habla que permite la transcripción de audio a texto de forma automatizada. [19](#)
- sprint*** Ciclo de trabajo de duración fija —típicamente de entre una y cuatro semanas— utilizado en metodologías ágiles como Scrum. Durante este periodo, el desarrollador se compromete a completar un conjunto específico de tareas con el objetivo de obtener un incremento de software funcional y evaluable. [iv](#), [3](#), [22](#), [24](#), [25](#), [26](#), [28](#), [30](#), [32](#), [35](#), [36](#), [37](#), [38](#), [40](#), [45](#), [49](#), [53](#), [54](#), [55](#), [61](#), [64](#), [65](#), [66](#), [70](#)
- store*** En el contexto de la biblioteca Zustand, un *store* es un almacén centralizado donde se guarda y administra un estado de manera global global, permitiendo que múltiples componentes accedan y reaccionen a la misma información de manera coherente. [49](#), [51](#), [57](#), [60](#), [62](#), [63](#)
- superset*** Lenguaje de programación que contiene todas las características de otro lenguaje y lo extiende con funcionalidades adicionales, como es el caso de TypeScript respecto a JavaScript. [14](#)
- tipado estático*** Característica de los lenguajes de programación en la que la comprobación de los tipos de variables se realiza durante la fase de compilación y no en tiempo de ejecución. [14](#)

**token** En el contexto de la ciberseguridad, un *token* es una credencial generada por el servidor para autenticar a un usuario y autorizar su acceso a ciertas funcionalidades del sistema.

[42](#), [43](#), [149](#)

**wrapper** Capa de abstracción que envuelve una biblioteca, servicio o interfaz de nivel inferior para simplificar su uso y, frecuentemente, añadir funcionalidades adicionales. Además de proporcionar una interfaz más idiomática —como la librería de Ollama para Python, que abstrae las llamadas HTTP manuales—, un *wrapper* puede integrar lógica de control de errores, reintentos automáticos, preprocesamiento de parámetros o formateo de respuestas, aportando cierto valor sobre el recurso original. [17](#)

# Bibliografía

---

- [1] (2026) Microsoft teams. Consultado el 3 de abril de 2026. [En línea]. Disponible en: <https://www.microsoft.com/en-us/microsoft-teams/group-chat-software>
- [2] (2026) Zoom. Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://www.zoom.com/>
- [3] M. Yusuf and D. Iskandar Mulyana, “Implementation of ai-based natural language processing (nlp) for automatic meeting minutes and summarization using voice-to-text on mobile applications,” *International Journal Software Engineering and Computer Science (IJSECS)*, vol. 5, pp. 1117–1125, 12 2025.
- [4] G. Vadlamudi, N. Vemuru, S. Vangapalli, R. Surapaneni, and S. Nimmagadda, “Meeting summarizer using natural language processing,” 04 2022, pp. 1610–1614.
- [5] (2026) Fireflies. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://fireflies.ai/es/>
- [6] (2026) otter.ai. Consultado el 3 de abril de 2026. [En línea]. Disponible en: <https://otter.ai/>
- [7] (2026) Microsoft teams premium. Consultado el 3 de abril de 2026. [En línea]. Disponible en: <https://www.microsoft.com/es-es/microsoft-teams/premium>
- [8] (2026) Zoom ai companion. Consultado el 3 de abril de 2026. [En línea]. Disponible en: <https://www.zoom.com/es/products/ai-assistant/>
- [9] (2026) Google meet. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://meet.google.com/landing>
- [10] (2026) Google workspace. Consultado el 3 de abril de 2026. [En línea]. Disponible en: <https://workspace.google.com/>
- [11] (2026) Meetly. Consultado el 3 de abril de 2026. [En línea]. Disponible en: <https://meetily.ai/>

- [12] (2026) Openai api platform. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://developers.openai.com/api/docs>
- [13] (2026) Meetly repository. Consultado el 3 de abril de 2026. [En línea]. Disponible en: <https://github.com/Zackriya-Solutions/meetily?tab=readme-ov-file>
- [14] (2026) Openai's own forecast predicts \$14 billion loss in 2026 but nvidia-style \$100 billion revenues by 2029 according to new report. Consultado el 5 de abril de 2026. [En línea]. Disponible en: <https://finance.yahoo.com/news/openais-own-forecast-predicts-14-150445813.html>
- [15] (2025) Price hike for ai coding tools: The free lunch is over. Consultado el 5 de abril de 2026. [En línea]. Disponible en: <https://www.heise.de/en/background/Price-hike-for-AI-coding-tools-The-Free-Lunch-Is-Over-10511480.html>
- [16] A. Nassif, I. Shahin, I. Attili, M. Azzeh, and K. Shaalan, "Speech recognition using deep neural networks: A systematic review," *IEEE Access*, vol. PP, pp. 1–1, 02 2019.
- [17] M. Bain, J. Huh, T. Han, and A. Zisserman, "Whisperx: Time-accurate speech transcription of long-form audio," 2023. [En línea]. Disponible en: <https://arxiv.org/abs/2303.00747>
- [18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [En línea]. Disponible en: <https://arxiv.org/abs/1706.03762>
- [19] (2026) Model context protocol. Consultado el 20 de mayo de 2026. [En línea]. Disponible en: <https://modelcontextprotocol.io/docs/getting-started/intro>
- [20] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [En línea]. Disponible en: <https://arxiv.org/abs/2005.11401>
- [21] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," 2019. [En línea]. Disponible en: <https://arxiv.org/abs/1908.10084>
- [22] (1992) Http. Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://www.w3.org/Protocols/HTTP/HTTP2.html>
- [23] (2026) Sse mozilla docs. Consultado el 4 de junio de 2026. [En línea]. Disponible en: [https://developer.mozilla.org/en-US/docs/Web/API/Server-sent\\_events](https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events)



- [24] (2026) ¿qué es la api rest? Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/rest-apis>
- [25] (2026) Ws mozilla docs. Consultado el 4 de junio de 2026. [En línea]. Disponible en: [https://developer.mozilla.org/es/docs/web/api/websockets\\_api](https://developer.mozilla.org/es/docs/web/api/websockets_api)
- [26] (2026) React. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://react.dev/>
- [27] (2026) Angular. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://angular.dev/>
- [28] (2026) Vue. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://vuejs.org/>
- [29] (2026) Typescript. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://www.typescriptlang.org/>
- [30] (2026) Tailwind css. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://tailwindcss.com/>
- [31] (2026) react-markdown. Consultado el 9 de junio de 2026. [En línea]. Disponible en: <https://github.com/remarkjs/react-markdown>
- [32] (2026) Zustand. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://zustand-demo.pmnd.rs/>
- [33] (2026) Redux toolkit. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://redux-toolkit.js.org/>
- [34] (2026) React context api. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://react.dev/reference/react/createContext>
- [35] (2026) React router. Consultado el 9 de junio de 2026. [En línea]. Disponible en: <https://reactrouter.com/home>
- [36] (2026) Tanstack query. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://tanstack.com/query/latest>
- [37] (2026) camelcase\_keys. Consultado el 9 de junio de 2026. [En línea]. Disponible en: <https://github.com/sindresorhus/camelcase-keys>
- [38] (2026) React i18next. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://react.i18next.com/>

- [39] (2026) Vite. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://vite.dev/>
- [40] (2026) Vitest. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://vitest.dev/>
- [41] (2026) Jest. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://jestjs.io/>
- [42] (2026) React router. Consultado el 9 de junio de 2026. [En línea]. Disponible en: <https://testing-library.com/>
- [43] (2026) msw. Consultado el 9 de junio de 2026. [En línea]. Disponible en: <https://mswjs.io/>
- [44] Python Software Foundation. (2026) Python. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://www.python.org/>
- [45] (2026) Nodejs. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>
- [46] (2026) Flask. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://flask.palletsprojects.com/en/stable/>
- [47] (2026) Django. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://www.djangoproject.com/>
- [48] Sebastián Ramírez. (2026) Fastapi. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://fastapi.tiangolo.com/>
- [49] (2026) Swagger ui. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://swagger.io/tools/swagger-ui/>
- [50] (2026) Openapi. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://www.openapis.org/>
- [51] (2026) Langchain. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://www.langchain.com/>
- [52] (2026) Ollama. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://ollama.com/>
- [53] (2026) Claude platform. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://platform.claude.com/docs/en/home>
- [54] (2026) vllm. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://vllm.ai/>

- [55] (2026) MongoDB. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://www.mongodb.com/es>
- [56] (2026) Chromadb. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://www.mongodb.com/es>
- [57] (2026) Whisperx. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://github.com/m-bain/whisperx>
- [58] (2026) Whisper. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://github.com/openai/whisper>
- [59] (2026) Pyannote audio. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://github.com/pyannote/pyannote-audio>
- [60] (2026) Assemblyai. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://www.assemblyai.com/products/speech-to-text>
- [61] (2026) Pytest. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://docs.pytest.org/en/stable/>
- [62] (2026) Unittest. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://docs.python.org/3/library/unittest.html>
- [63] (2026) Junit. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://junit.org/>
- [64] (2026) Docker. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://www.docker.com/>
- [65] (2026) Docker compose. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://docs.docker.com/compose/>
- [66] (2026) Nginx proxy manager. Consultado el 1 de abril de 2026. [En línea]. Disponible en: <https://nginxproxymanager.com/>
- [67] (2026) Duckdns. Consultado el 12 de junio de 2026. [En línea]. Disponible en: <https://www.duckdns.org/>
- [68] (2026) Git. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://git-scm.com/>
- [69] (2026) Github. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://github.com/>

- [70] (2026) Github projects. Consultado el 31 de marzo de 2026. [En línea]. Disponible en: <https://docs.github.com/es/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>
- [71] (2026) Scrumban, the best of kanban and scrum. Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://kanbantool.com/scrumban-kanban-and-scrum>
- [72] (2026) Scrum.org. Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://www.scrum.org/>
- [73] (2026) The kanban method. Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://kanbantool.com/kanban-method>
- [74] (2026) Gitflow workflow. Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://kanbantool.com/scrumban-kanban-and-scrum>
- [75] (2026) Documento boe-a-2026-9024. Consultado el 4 de junio de 2026. [En línea]. Disponible en: [https://www.boe.es/diario\\_boe/txt.php?id=BOE-A-2026-9024](https://www.boe.es/diario_boe/txt.php?id=BOE-A-2026-9024)
- [76] Mit license. Consultado el 3 de junio de 2026. [En línea]. Disponible en: <https://opensource.org/license/mit>
- [77] (2007) Agpl license. Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://www.gnu.org/licenses/agpl-3.0.en.html>
- [78] (2007) Gpl license. Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://www.gnu.org/licenses/gpl-3.0.en.html>
- [79] (2024) Mastering rest apis: Layered system in rest. Consultado el 4 de junio de 2026. [En línea]. Disponible en: <https://medium.com/@bvsahane89/mastering-rest-apis-layered-system-in-rest-66447d6ae16d>
- [80] (2026) Repository design pattern. Consultado el 8 de junio de 2026. [En línea]. Disponible en: <https://www.geeksforgeeks.org/system-design/repository-design-pattern/>
- [81] E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, 1st ed. Addison-Wesley Professional, 1994. [En línea]. Disponible en: [http://www.amazon.com/Design-Patterns-Elements-Reusable-Object-Oriented/dp/0201633612/ref=ntt\\_at\\_ep\\_dpi\\_1](http://www.amazon.com/Design-Patterns-Elements-Reusable-Object-Oriented/dp/0201633612/ref=ntt_at_ep_dpi_1)
- [82] (2026) Oauth 2.0. Consultado el 6 de junio de 2026. [En línea]. Disponible en: <https://oauth.net/2/>

- [83] (2026) Fastapi: Oauth2 with password (and hashing), bearer with jwt tokens. Consultado el 6 de junio de 2026. [En línea]. Disponible en: <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>
- [84] (2026) React - the complete guide (incl. next.js, redux). Consultado el 15 de junio de 2026. [En línea]. Disponible en: <https://www.udemy.com/course/react-the-complete-guide-incl-redux/?couponCode=25BBPMXINACTIVE>